COMPLETE BEGINNER-FRIENDLY GUIDE

LedgerLine

Learn secure financial transaction systems from absolute zero



Ledgers • Atomicity • Idempotency • Concurrency • Immutable records
Authentication • Database design • Testing • Interview preparation

Teaching document — all amounts in Indian Rupees (₹). Designs marked PROPOSED are suggestions, not confirmed decisions.

Contents

How to Use This Guide . . . 5

1. Project Introduction . . . 6

1.1 What is LedgerLine? . . . 6
1.2 What problem does it solve? . . . 6
1.3 Why financial systems need special reliability . . . 7
1.4 How LedgerLine differs from a normal CRUD application . . . 7
1.5 What the final application should look like . . . 7
1.6 Real-world banking scenarios that use these concepts . . . 8

2. Visual System Architecture . . . 10

2.1 Every component and its responsibility . . . 10
2.2 Why separate backend and database responsibilities? . . . 11
2.3 The request funnel . . . 11

3. Financial Transaction Fundamentals . . . 12

3.1 Account . . . 12
3.2 Balance . . . 12
3.3 Transaction . . . 12
3.4 Debit and credit . . . 12
3.5 Ledger . . . 13
3.6 Double-entry bookkeeping . . . 13
3.7 Transaction ID . . . 15
3.8 Audit log . . . 15
3.9 Balance vs. transaction vs. ledger entry — in one table . . . 16

4. Complete Transaction Flow — a ₹1,000 Transfer . . . 17

4.1 What happens when the user clicks Transfer . . . 19
4.2 The request the frontend sends . . . 19
4.3 What the backend validates . . . 19
4.4 How authorization is checked . . . 19
4.5 How the available balance is checked — safely . . . 20
4.6 How the database updates the accounts, creates entries and commits . . . 20
4.7 The response the frontend receives . . . 20
4.8 Before and after . . . 21
4.9 What happens if an error occurs? . . . 21

5. Financial Transaction Rules . . . 23

5.1 Atomicity . . . 23
5.2 Double-Entry Ledger . . . 25
5.3 Idempotency . . . 26

5.4 Concurrency . . . 29
5.5 Immutable Records . . . 31
5.6 Balance Conservation . . . 33
5.7 Isolation . . . 34
5.8 Consistency . . . 36

6. Authentication and Security . . . 38

6.1 Authentication versus authorization . . . 38
6.2 Login and password hashing . . . 38
6.3 JWT or session-based authentication . . . 39
6.4 Role-based access control (RBAC) . . . 40
6.5 Account ownership . . . 41
6.6 Admin and Auditor permissions . . . 42
6.7 Backend security checklist . . . 42
6.8 Input validation . . . 42
6.9 API security . . . 43
6.10 Secrets and environment variables . . . 43
6.11 Preventing unauthorized transactions — practical scenarios . . . 44

7. Database Design . . . 45

7.1 Database vocabulary from zero . . . 45
7.2 How the tables connect . . . 45
7.3 Balance vs. transaction record vs. ledger entry . . . 46
7.4 Money as integers, time as UTC . . . 47
7.5 Complete schema (PROPOSED, PostgreSQL) . . . 47
7.6 Table-by-table explanation with example records . . . 48
7.7 Indexes — why and which . . . 49

8. Technology Stack . . . 50

8.1 Why each choice (and how it communicates) . . . 51
8.2 Decisions to confirm before coding . . . 52

9. Development Phases . . . 53

Phase 1 — Problem Analysis and Architecture . . . 54
Phase 2 — Database Schema and Backend Foundation . . . 54
Phase 3 — Authentication, Roles and Account Management . . . 54
Phase 4 — Financial Transaction Engine and Ledger . . . 55
Phase 5 — Idempotency and Concurrency Protection . . . 55
Phase 6 — Security and Financial Integrity Testing . . . 56
Phase 7 — Frontend Development and Integration . . . 56
Phase 8 — Deployment, Documentation and Final Verification . . . 56

10. Testing and Failure Scenarios . . . 57

10.1 Scenario table . . . 57

10.2 Example test code (sketches) . . . 58
10.3 How to know your tests are good . . . 60

11. Engineering Workflow . . . 61

11.1 Git commits and incremental development . . . 61
11.2 Code review . . . 62
11.3 AI coding assistance, PROMPTS.md and THREATS.md . . . 62
11.4 README and documentation . . . 63
11.5 Test before you commit — a daily routine . . . 63

12. Complete Project Demonstration . . . 64

12.1 Login page . . . 64
12.2 Dashboard . . . 64
12.3 Account management . . . 65
12.4 Deposit and withdrawal screens . . . 65
12.5 Transfer screen . . . 66
12.6 Transaction history . . . 66
12.7 Admin interface . . . 67
12.8 Auditor interface . . . 67

13. Interview Preparation . . . 69

13.1 Architecture . . . 69
13.2 Transaction processing and double-entry . . . 69
13.3 Atomicity and database transactions . . . 70
13.4 Idempotency . . . 70
13.5 Concurrency and race conditions . . . 71
13.6 Immutability, conservation and consistency . . . 71
13.7 Authorization and security . . . 72
13.8 Testing and engineering . . . 73

14. Final Learning Roadmap . . . 74

14.1 What you must understand BEFORE coding . . . 74
14.2 What you will learn DURING implementation . . . 74
14.3 What you must be able to explain AFTER completion . . . 75
14.4 A beginner-to-developer learning sequence . . . 75

Appendix A — Glossary . . . 76

Appendix B — Quick Reference Cheat Sheet . . . 77

How to Use This Guide

This guide teaches **LedgerLine** from absolute zero. It assumes you have never built a banking app, a payment system or an accounting ledger. Every technical word is explained the first time it appears, and every idea is shown with simple numbers in Indian Rupees (₹).

Three kinds of statements — never confuse them

Label in this guide	Meaning	Example
CONFIRMED REQUIREMENT	Stated in the LedgerLine problem statement you provided. These are fixed.	“Duplicate requests must not execute the same financial operation twice.”
BUSINESS RULE	What must be true about the money, independent of technology.	“An account may not be overdrawn.”
TECHNICAL IMPLEMENTATION	One way to make a business rule true in code.	`SELECT … FOR UPDATE` row locking in PostgreSQL.
PROPOSED DESIGN	My suggestion. NOT decided by the project. You may change it.	React + FastAPI + PostgreSQL; table and column names; the CLEARING system account.

WHAT IS ACTUALLY CONFIRMED

The problem statement confirms: authentication and role-based access; account management; balance inquiry; deposits, withdrawals and transfers; transaction history and statements; admin account management; auditor access; audit logging; atomic transactions; a double-entry ledger; idempotency; concurrency protection; immutable records with compensating corrections; server-side authorization; input validation; protected secrets; the eight development phases; automated tests; deployment; AI-assist documentation (PROMPTS.md) and THREATS.md.

Not confirmed anywhere: the programming language, framework, database product, hosting platform, exact table/column names, exact API URLs, whether Admin may move money, and how deposits are funded. This guide proposes answers and always labels them.

Suggested reading order

- **First pass (2–3 hours):** sections 1–4. Goal: you can explain LedgerLine and a ₹1,000 transfer to a friend.
- **Second pass:** section 5 (the eight financial rules). This is the technical heart of the project — read it slowly with a notebook.
- **Third pass:** sections 6–8 (security, database, stack), then 9–10 (phases and tests) as you start coding.
- **Before any interview:** sections 13 and 14.

1. Project Introduction

1.1 What is LedgerLine?

LedgerLine is a **secure financial transaction management system**. Think of a small, simplified bank. Customers can create accounts, check balances, deposit money, withdraw money, transfer money to other accounts and view a history of everything that happened. Administrators manage accounts. Auditors review the records.

The key word is **secure and correct**. Anyone can build a screen with a “Transfer” button. The real engineering challenge — and the whole point of the project — is guaranteeing that **every rupee is always accounted for**, even when computers crash, networks retry requests, users double-click, or thousands of people act at the same moment.

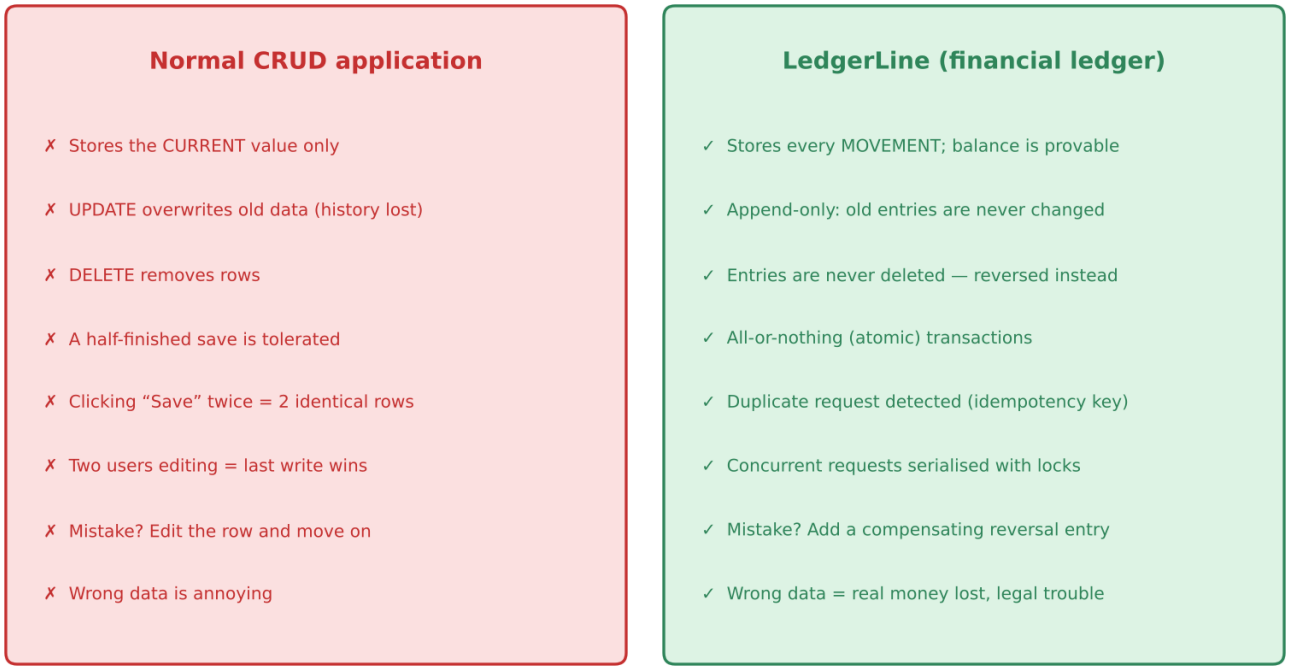


Figure 1.1 *— A normal CRUD application tolerates small errors. A financial ledger cannot.*

1.2 What problem does it solve?

Money systems fail in five classic ways. LedgerLine exists to show that you know how to stop each one.

Failure	What goes wrong (in ₹)	Technique that stops it
Partial failure	Server crashes after taking ₹1,000 from A but before giving it to B. ₹1,000 vanishes.	Atomic transactions
Duplicate requests	User double-clicks “Pay”, or the phone retries after a timeout. B receives ₹2,000 instead of ₹1,000.	Idempotency keys
Race conditions	Two ₹800 withdrawals hit a ₹1,000 balance at the same instant. Both succeed. Bank loses ₹600.	Concurrency control / row locks
Unauthorized access	Asha sends a request pretending to be Bharat and drains his account.	Authentication + server-side authorization
Silent history edits	Someone edits a past entry to hide a theft. Nobody can prove what happened.	Immutable, append-only ledger + audit log

1.3 Why financial systems need special reliability

- **Mistakes are irreversible in the real world.** If a social network loses a “like”, nobody cares. If a bank loses ₹10,000, a person may miss rent.
- **The law demands traceability.** Banks must be able to explain every rupee: who moved it, when, why. This is called an **audit trail**.
- **Trust is the product.** Customers only use a bank they believe will never lose or invent money.
- **Scale multiplies rare bugs.** A bug that happens once in a million requests will happen daily when you process millions.

1.4 How LedgerLine differs from a normal CRUD application

CRUD means Create, Read, Update, Delete — the four basic things most apps do with data (a to-do app, a blog, a student list). A CRUD app treats data as *the current state*: if you change your name, the old name is overwritten.

A ledger system treats data as *a permanent history of events*. Your balance is not “just a number someone typed”; it is the **result of adding up every deposit, withdrawal and transfer that ever touched your account**. The number can always be re-proven from the history.

	Normal CRUD app	LedgerLine
Core idea	Store the latest value	Store every movement; value is derived and verifiable
Update	Overwrite freely	Never rewrite history; add a correcting entry
Delete	Allowed	Not allowed for financial records
Failure handling	Show error, move on	Roll back everything; never leave half-done state
Duplicates	Annoying	Dangerous — must be detected
Concurrency	Last write wins	Must be serialised so no money is created/lost
Access control	Often just “logged in”	Role + account ownership checked on the server for every action
Testing	Does the page work?	Prove invariants: balances match ledger, totals conserved

1.5 What the final application should look like

From the problem statement, the finished project should let a user: log in; see their accounts and balances; deposit and withdraw; transfer between accounts; read transaction history and statements; and let admins and auditors do their roles. It must be **deployed and accessible**, **proven by automated tests**, and **documented so another developer can clone and run it**. Section 12 shows screen mockups for every role.



Figure 1.2 *— LedgerLine at a glance: a database holding trusted records, protected by security and traceable documentation. (Illustration supplied by you.)*

1.6 Real-world banking scenarios that use these concepts

Real-world situation	Concept it needs
You pay a shop by UPI; the app shows “processing” and you tap Pay again.	Idempotency — you must not be charged twice.
An EMI auto-debit and an ATM withdrawal happen in the same second on a ₹10,000 balance.	Concurrency control — balance can never go negative.
A bank transfer fails midway between two banks.	Atomicity and compensating (reversal) entries.
Your salary is credited; your passbook shows one more line.	Ledger entries are appended, never edited.
A wrong transfer is corrected by the bank two days later.	Reversal transaction, not deletion.
An RBI auditor asks “prove these accounts balance”.	Double-entry bookkeeping + audit log.
A bank employee tries to look at a celebrity’s account without reason.	Role-based access control + audit logging.

The two pictures below show how real payments move between banks. **Important scope note:** these diagrams involve *several* banks and settlement networks. LedgerLine is much smaller — it models the *inside* of one bank (or one wallet provider): accounts, balances and a trustworthy ledger.

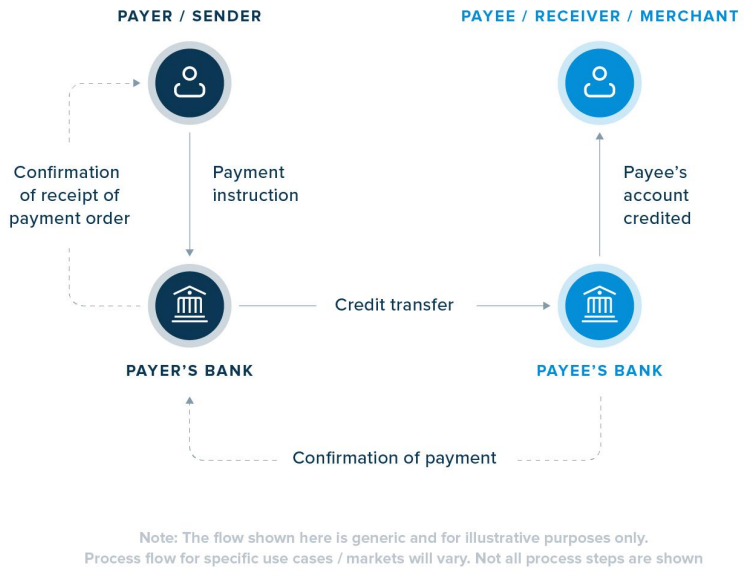


Figure 1.3 *— Generic payment flow between a payer, their bank, the payee's bank and the payee. LedgerLine implements the "bank keeps accurate books" part, not the inter-bank network. (Supplied image.)*

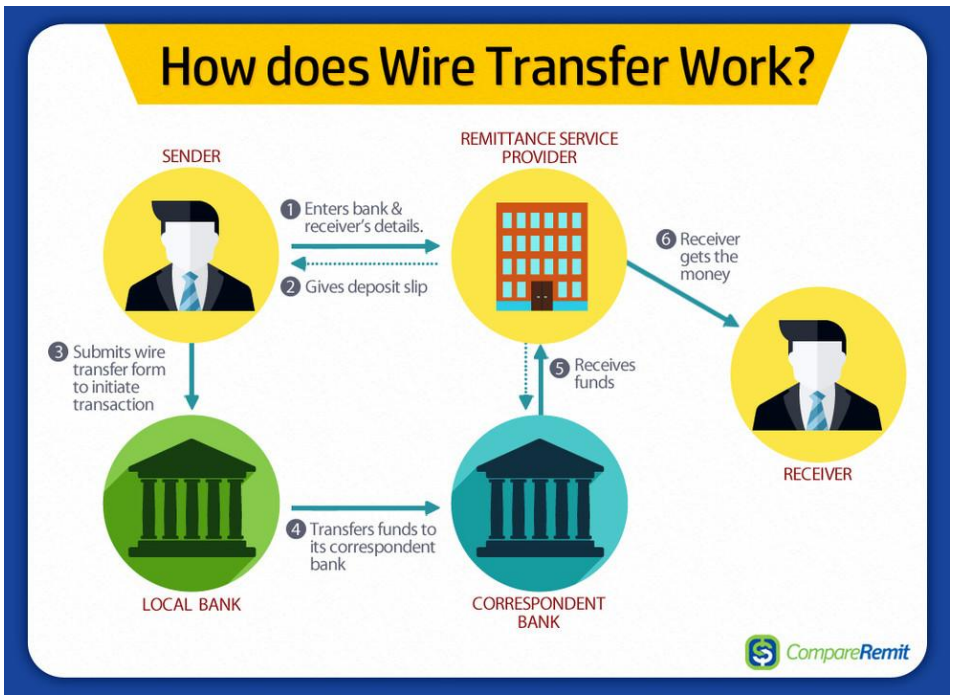


Figure 1.4 *— A wire transfer passes through several parties. Every party must keep a correct ledger of its own side — exactly what LedgerLine practises. (Supplied image.)*

2. Visual System Architecture

Architecture means “which parts does the system have, and who is responsible for what?” Before writing code, you should be able to draw this picture from memory.

> **PROPOSED DESIGN (not a confirmed requirement)**
> The tiers (client, server, data) follow your problem statement’s description of frontend, backend and database. The specific technologies named inside the boxes (React, PostgreSQL) are proposals.

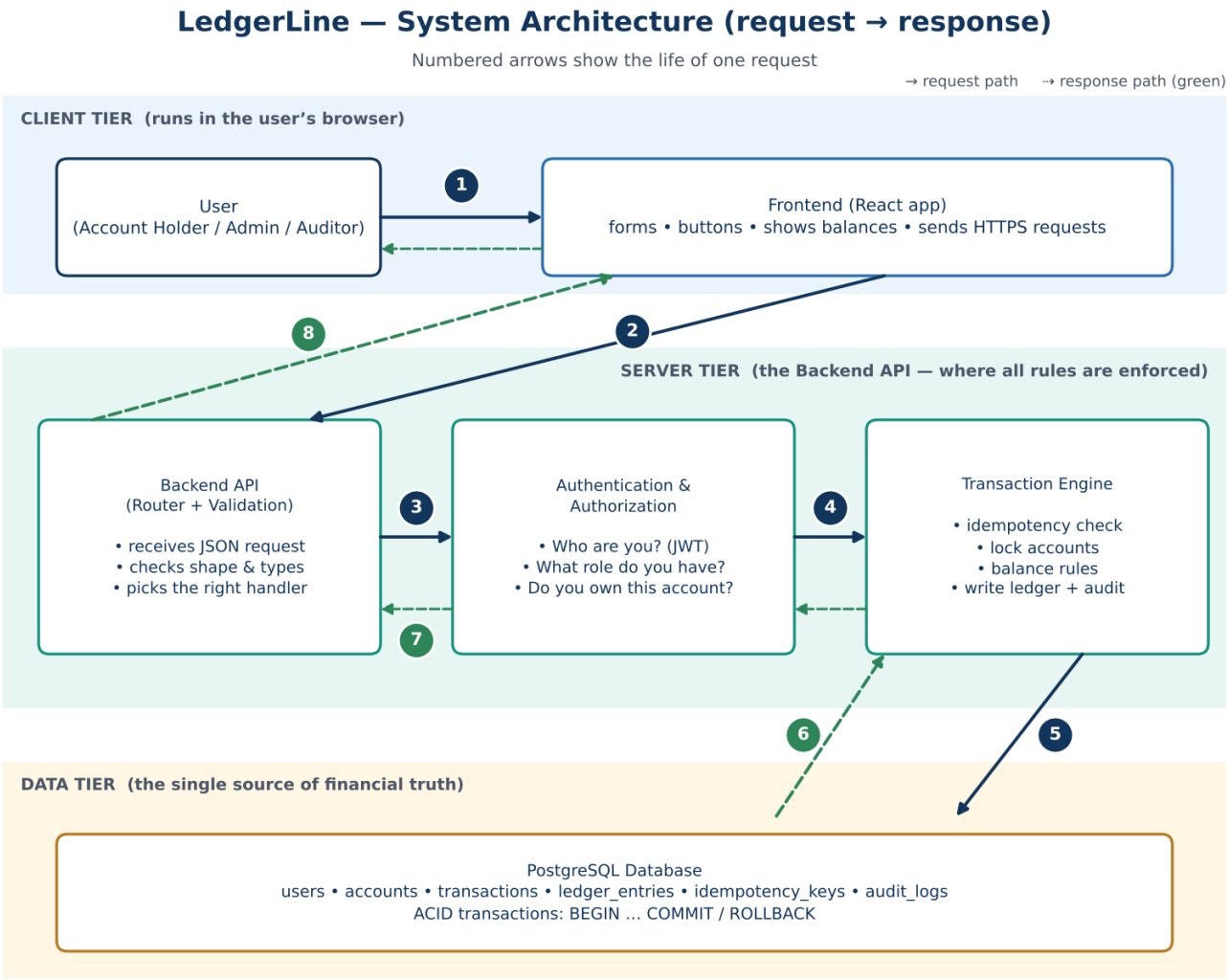


Figure 2.1 *— The journey of one request: User → Frontend → Backend API → Authentication & Authorization → Transaction Engine → Database → Response to User.*

2.1 Every component and its responsibility

Component	Responsibility	Must NOT do
User	Human with a role: Account Holder, Admin or Auditor. Decides what to do.	—
Frontend (browser app)	Shows screens, collects input, sends HTTPS requests, displays results and errors. Generates the `Idempotency-Key` for each money action.	Enforce rules. The browser can be modified by anyone, so frontend checks are only for convenience.
Backend API	Receives requests, checks the JSON shape and types (validation), routes to the correct handler, formats responses and error codes.	Trust any data from the client without checking.

Component	Responsibility	Must NOT do
Authentication	Proves identity: verifies the password at login and the token on every later request.	Store passwords in plain text.
Authorization	Decides what an identified user may do: role check + “does this account belong to you?”.	Rely on the frontend hiding buttons.
Transaction Engine	The heart: idempotency check, locking, balance rules, balance updates, ledger entries, audit entry, all inside one database transaction.	Perform partial updates; swallow errors.
Database	Stores truth permanently. Enforces constraints and atomic commit/rollback.	Hold business logic that only exists in the UI.
Response	Success (with transaction ID and new balance) or a clear error code.	Leak internal details (stack traces, SQL).

2.2 Why separate backend and database responsibilities?

Rules like “no overdraft” live in **both** the backend (to give a friendly error) and the database (as a last line of defence, e.g. `CHECK (balance_paise >= 0)`). This is called **defence in depth**: if one layer has a bug, another stops the damage. You will meet this idea repeatedly.

2.3 The request funnel

A money request does not go straight to the database. It passes six checkpoints, cheapest first. Any failure stops the request immediately and returns an error code.

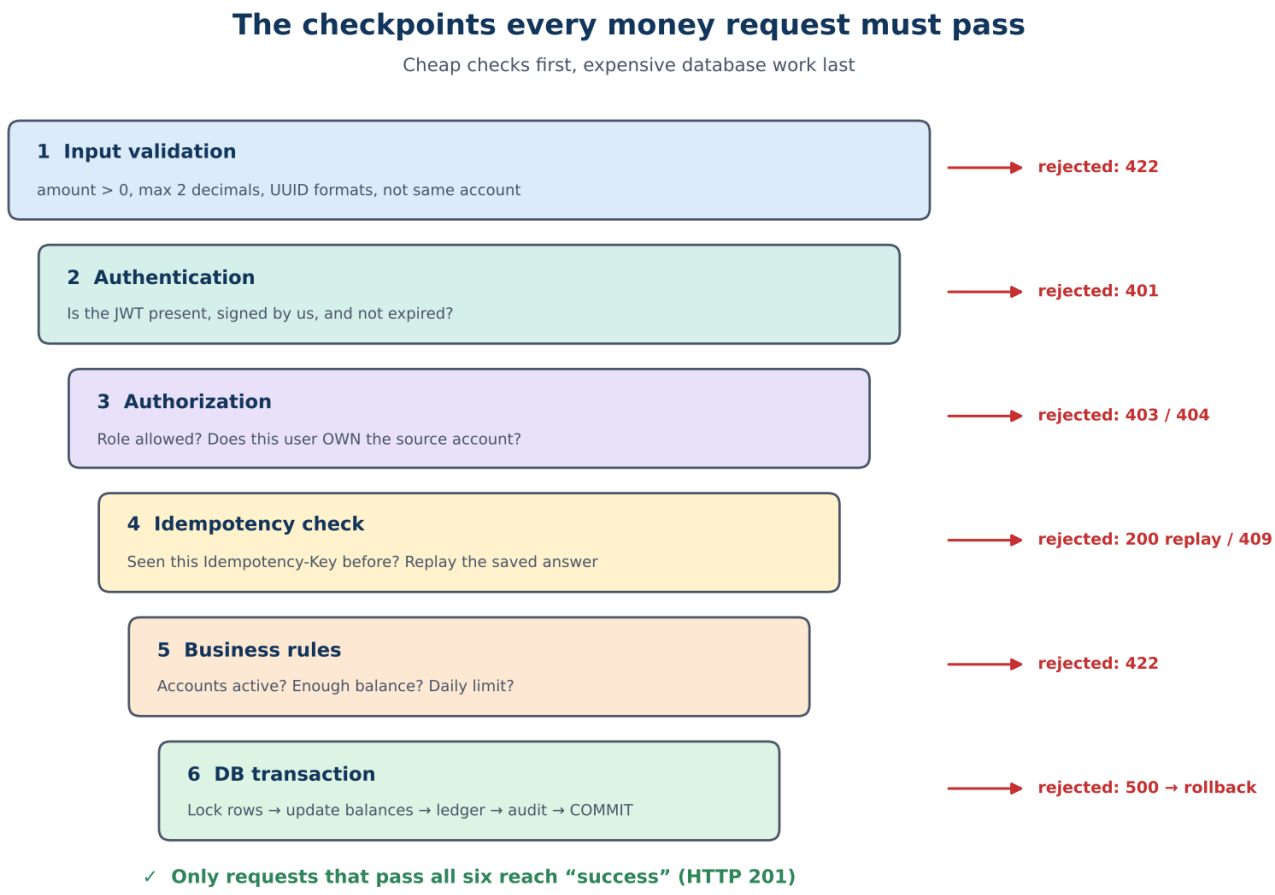


Figure 2.2 *— The six checkpoints. Notice that the database is touched only after everything cheaper has passed.*

3. Financial Transaction Fundamentals

Here are the eight words every banking engineer uses. Read them in order — each builds on the previous one.

3.1 Account

> **DEFINITION**
> An **account** is a named container that tracks how much money belongs to one holder. It has an owner, an identifier (account number) and a status such as ACTIVE or FROZEN.

Example: Asha owns account `LL0001`. Bharat owns `LL0002`. One person can own several accounts (savings, current).

3.2 Balance

> **DEFINITION**
> A **balance** is the amount of money currently in an account.

Asha’s balance is ₹5,000. The subtle question is: *where does that number come from?* In a naive app it is just a stored number you edit. In LedgerLine it is **the sum of all credits minus the sum of all debits** on that account — which means it can be *recomputed and checked* from the ledger. We store a copy of it in the account row for speed, and constantly verify that the copy matches the history.

3.3 Transaction

> **DEFINITION**
> A **transaction** (in the business sense) is one complete money event: a deposit, a withdrawal, a transfer or a reversal. It has an ID, a type, an amount, who initiated it and when.

Do not confuse it with a **database transaction** (a technical all-or-nothing wrapper around several SQL statements — see Atomicity). One *financial* transaction is implemented *inside* one *database* transaction. To reduce confusion this guide writes “financial transaction” and “database transaction” when it matters.

3.4 Debit and credit

These two words confuse almost everyone, because everyday speech uses them differently (“my card was debited” vs. “I have credit”). In accounting they are just **the two sides of a ledger**: left side = debit, right side = credit. What they *mean* depends on the type of account.

> **WHICH WAY DOES DEBIT GO?**
> Sources you may read online use different sign conventions, and the three pictures you supplied show different ones. This is normal, not a mistake:
> • In a **company’s own books** (Figure 3.3), “Cash” is an asset: a debit increases it and a credit decreases it.
> • In a **bank’s books**, a customer’s balance is money the bank *owes* the customer (a liability): a **credit increases** it and a **debit decreases** it.
> **LedgerLine uses the bank’s view for customer accounts** — which also matches Figure 3.2: the paying account is debited (money leaves) and the receiving account is credited (money enters).

In LedgerLine (customer accounts)	Effect on balance	Plain meaning
DEBIT	balance goes DOWN	money leaves this account
CREDIT	balance goes UP	money enters this account

3.5 Ledger

> **DEFINITION**
> A **ledger** is the permanent, ordered book of all money movements. Each line is a **ledger entry**: which account, debit or credit, how much, when, and which transaction caused it.

Your bank passbook or statement is a ledger *filtered for you*. The bank's master ledger contains everyone's lines.

Entry	Txn	Account	Direction	Amount	Balance after
1	T1 deposit	CLEARING	DEBIT	₹5,000	−₹5,000
2	T1 deposit	LL0001 (A)	CREDIT	₹5,000	₹5,000
3	T2 deposit	CLEARING	DEBIT	₹2,000	−₹7,000
4	T2 deposit	LL0002 (B)	CREDIT	₹2,000	₹2,000

> **PROPOSED DESIGN (not a confirmed requirement)**
> "CLEARING" is a **system account** representing money coming in from or going out to the outside world (cash counter, UPI top-up, bank rail). Using it lets deposits and withdrawals also be double-entry. This is a proposed modelling choice; the problem statement only requires debit and credit entries for transfers.

3.6 Double-entry bookkeeping

> **DEFINITION**
> **Double-entry bookkeeping**: every financial event is recorded **twice**, as equal amounts on two sides — a debit somewhere and a credit somewhere else. Therefore **total debits always equal total credits**.

Why bother? Because it makes mistakes **detectable**. If you ever add up the whole ledger and debits ≠ credits, you *know* something is broken, even if nobody noticed it happen. Single-entry records (just editing a balance) give you no such alarm. It has been used by merchants since the 1400s for exactly this reason.

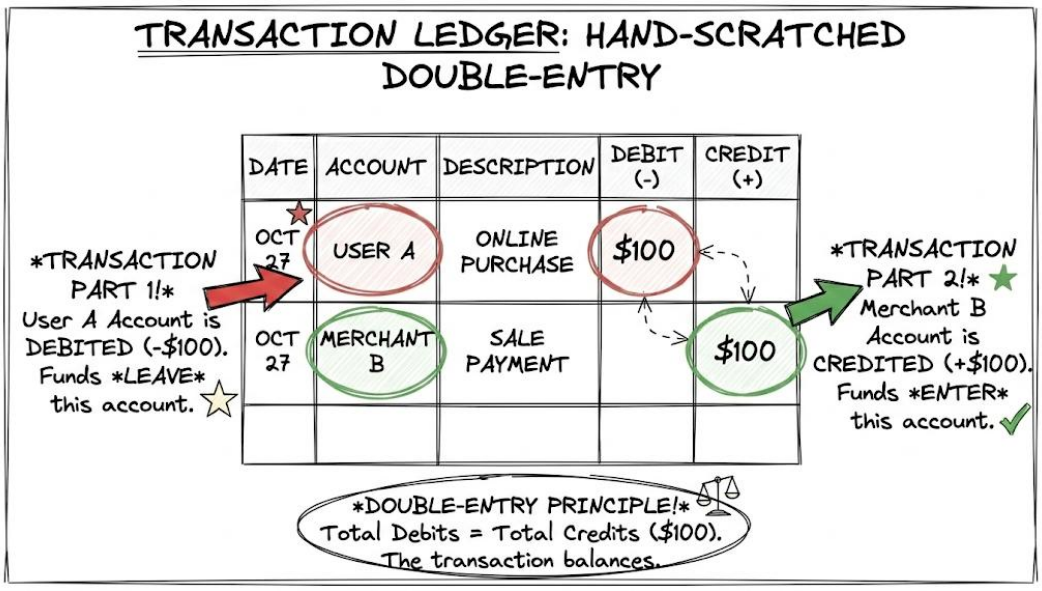


Figure 3.1 *— Hand-drawn double-entry: User A is debited, Merchant B credited, and the totals match. (Supplied image; it uses $ — in this guide we use ₹.)*

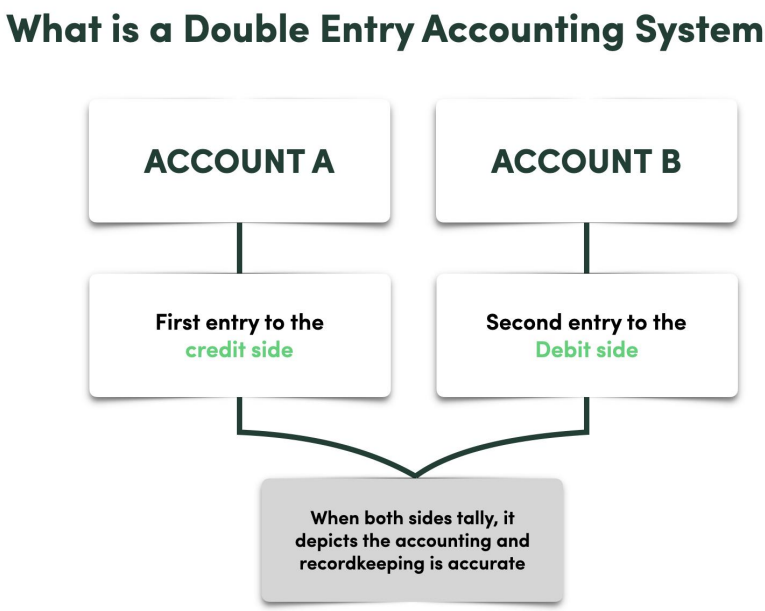


Figure 3.2 *— Two accounts, two entries, and the check that both sides tally. (Supplied image. Its generic labelling puts credit on Account A and debit on Account B; which side increases or decreases depends on the account type — see the warning box above.)*

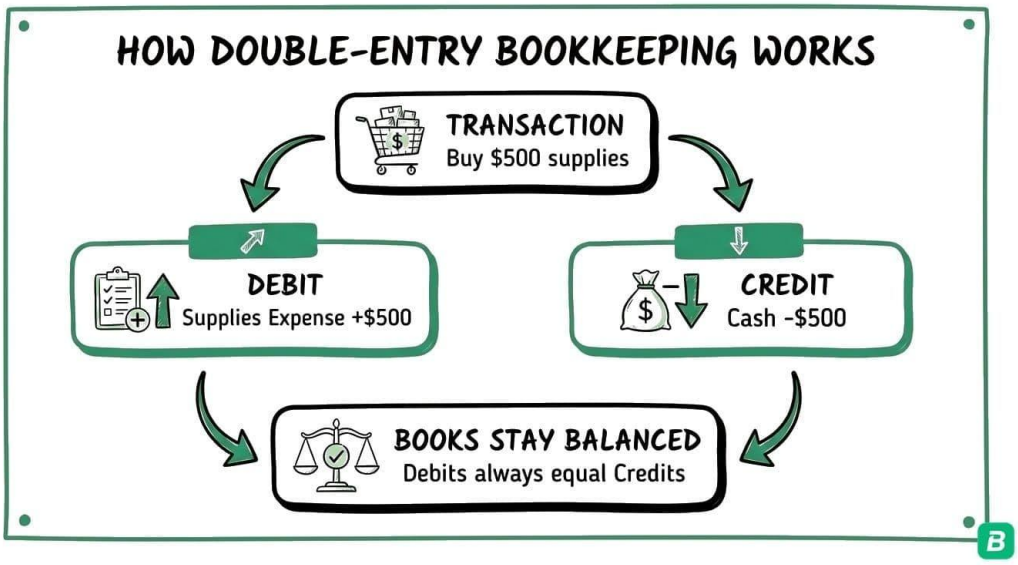


Figure 3.3 *— A company's view: buying ₹500 of supplies debits an Expense and credits Cash. The books stay balanced. (Supplied image; $ replaced by ₹ in your head. Here "Cash" is the company's own asset, so credit = decrease.)*

Bagel.co Activity	Debits	Credits
Charge Customer 1	(100,000)	
Pay Customer 2		100,000
Charge Customer 3	(500,000)	
Pay Customer 2		500,000
Customer 2 deposits to bank		600,000
Cash on hand	(600,000)	
Total	(1,200,000)	1,200,000

An example double-entry ledger of Bagel.co transactions.

Figure 3.4 *— A business ledger with many activities: the totals of both columns match (1,200,000). (Supplied image; some tools show debits in brackets.)*

LedgerLine numerical example: a deposit and a transfer

NUMERICAL EXAMPLE

Asha deposits ₹5,000 → debit CLEARING ₹5,000, credit A ₹5,000. Debits ₹5,000 = Credits ₹5,000 ✓
Asha transfers ₹1,000 to Bharat → debit A ₹1,000, credit B ₹1,000. Debits ₹1,000 = Credits ₹1,000 ✓
Whole ledger: debits ₹5,000 + ₹2,000 + ₹1,000 = ₹8,000; credits ₹5,000 + ₹2,000 + ₹1,000 = ₹8,000 ✓

3.7 Transaction ID

DEFINITION

A **transaction ID** is a unique label for one financial transaction. Every ledger entry created by that transaction carries the same ID, which ties the two sides of a transfer together.

Example: transfer T3 produced ledger entries #5 (debit A) and #6 (credit B). If an auditor asks “why did A lose ₹1,000?”, the transaction ID leads straight to entry #6 and to the audit log line saying who initiated it. In the database this is usually a **UUID** (a long random unique value such as `7b0c…`) and often a human-friendly **reference** like `TXN-000003` is shown on screens.

3.8 Audit log

DEFINITION

An **audit log** is an append-only diary of **who did what, when, and with what outcome** — including actions that failed or were denied.

The ledger answers “where did the money go?”. The audit log answers “who tried to do what?”. Example lines:

Time	Actor	Action	Outcome	Details
20:11:02	asha	LOGIN	SUCCESS	ip 49.x.x.x
20:12:40	asha	TRANSFER	SUCCESS	T3, ₹1,000 LL0001→LL0002
20:13:15	asha	TRANSFER	DENIED	tried to debit LL0002 (not owner)
20:14:50	meera (admin)	REVERSE	SUCCESS	T3 reversed — reason: wrong beneficiary

Ledger	Audit log
Money movements only	Security-relevant actions, including failures
Used to compute balances	Used for investigation and compliance
One per account side of a transaction	One per significant action

3.9 Balance vs. transaction vs. ledger entry — in one table

Thing	What it is	Example	Changes over time?
Account balance	Current total for one account (a cached summary)	A = ₹4,000	Yes — updated on every transaction
Transaction record	One header per financial event	T3: transfer ₹1,000, A→B, by Asha	No — never edited
Ledger entry	One line per account affected	#5 DEBIT A ₹1,000; #6 CREDIT B ₹1,000	No — never edited or deleted

4. Complete Transaction Flow — a ₹1,000 Transfer

We will follow one transfer from the mouse click to the final database commit. **Starting state:** Account A (Asha, `LL0001`) = **₹5,000**; Account B (Bharat, `LL0002`) = **₹2,000**. Asha transfers **₹1,000** to Bharat.

> **PROPOSED DESIGN (not a confirmed requirement)**
> URLs, field names and status codes below are PROPOSED API design. The *sequence of checks* is what the problem statement requires.

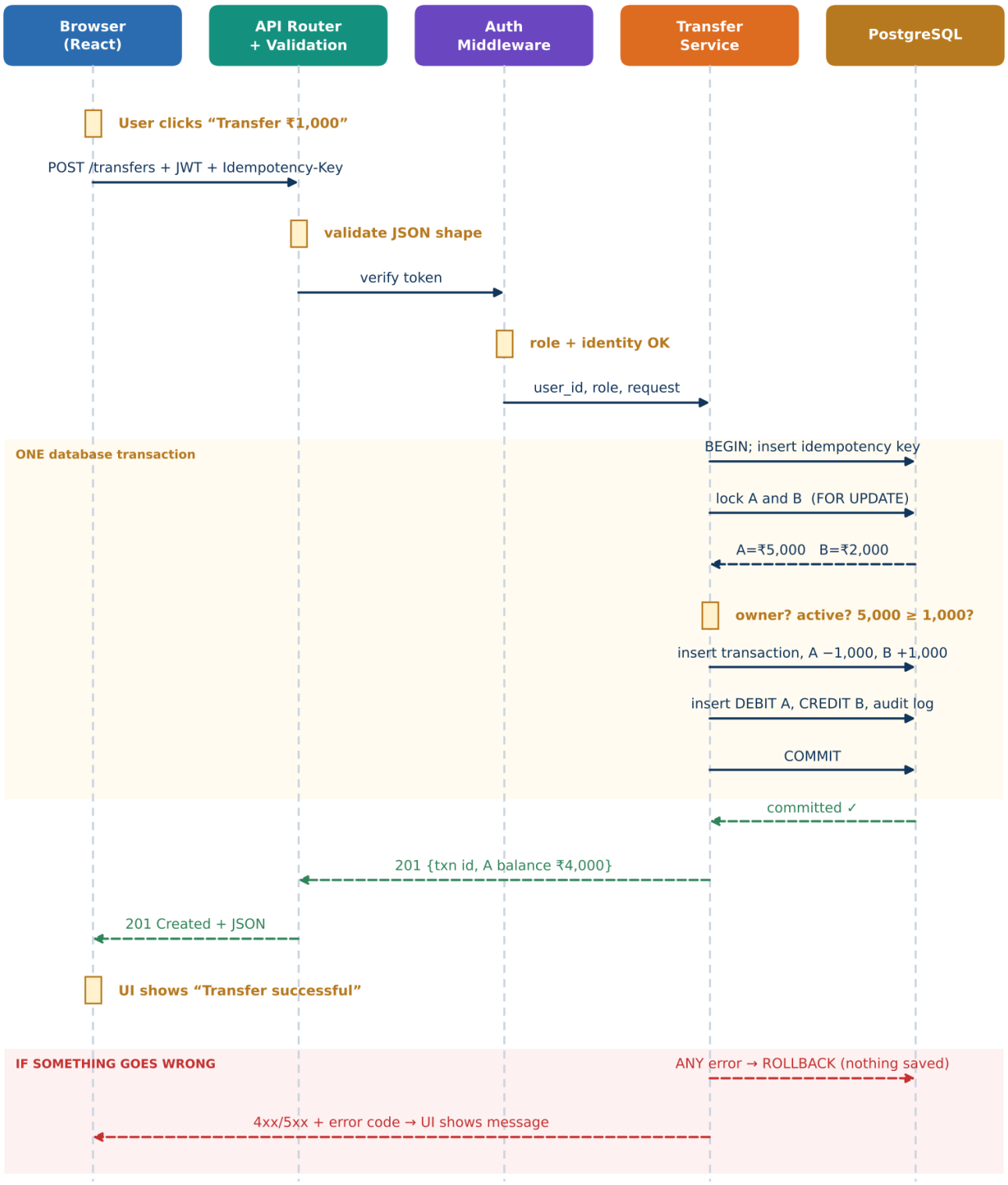


Figure 4.1 *— Sequence diagram of the whole transfer. Read it top to bottom; time flows downward.*

4.1 What happens when the user clicks Transfer

1. The frontend validates what it can for fast feedback (amount is a positive number, “to” account is filled). Remember: this is only a convenience.
2. The frontend generates a fresh random **Idempotency-Key** (a UUID) for this *intention to transfer*. If the click is repeated or the request retried, the same key is reused.
3. The Transfer button is disabled to discourage double-clicks (again, only a convenience — the server defends for real).
4. The frontend sends an HTTPS request to the backend (next section).

4.2 The request the frontend sends

```
POST /api/v1/transfers HTTP/1.1
Host: ledgerline.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...      <- proves who Asha is
Idempotency-Key: 4e1f6c0a-2b8d-4a57-9c1e-0f6f1f5b7a10              <- unique per intention
Content-Type: application/json

{
  "from_account_id": "acc_A",
  "to_account_id":   "acc_B",
  "amount":          "1000.00",
  "description":     "Rent share"
}
```

Notice the amount is the **string** `"1000.00"`, not a floating-point number. Computers store decimals like 0.1 inexactly in floating point; money must never use it (see Section 7.4). The backend converts the string to an exact integer number of **paise** (₹1 = 100 paise): ₹1,000.00 = `100000` paise.

4.3 What the backend validates

#	Check	Fails with	Why
1	JSON is well-formed; required fields exist	422	Malformed input must never reach business logic
2	Amount is a positive decimal with ≤ 2 decimal places; within a sane maximum	422	Blocks −1000, 0, 0.001, NaN, 1e999
3	Source ≠ destination account	422	Self-transfer is meaningless and can mask bugs
4	IDs are valid UUIDs; `Idempotency-Key` header present	422	Retry safety requires a key
5	JWT valid, signed by us, not expired → we know the user	401	Authentication
6	User role may transfer (Account Holder)	403	Authorization — role
7	User **owns** the source account	403 (or 404)	Authorization — ownership
8	Both accounts exist and are ACTIVE (not frozen/closed)	422 / 404	Business rule

4.4 How authorization is checked

The token proves *who* Asha is (`user_id = u_asha`). The server then **loads the source account from the database** and compares `account.owner_id` with `u_asha`. If they differ, the request is rejected, however well-formed it looks. The ownership check uses data from our database and token — **never** an “owner” value supplied by the client.

> **INCORRECT IMPLEMENTATION**
>
> Wrong: the request body contains `"user_id": "u_asha"` and the server believes it. An attacker simply writes another user’s ID.

> **CORRECT APPROACH**
> Right: take the user identity only from the verified token, then compare with the owner stored in the database.

4.5 How the available balance is checked — safely

Checking “is there enough money?” is only safe if nobody can change the balance between *checking* and *updating*. So the engine first **locks** both account rows (`SELECT … FOR UPDATE`), always in the same order (lowest ID first) to prevent deadlocks, and only then reads the balance. While we hold the lock, concurrent transfers touching those accounts **wait**. Now: A = ₹5,000 ≥ ₹1,000, so the rule passes.

4.6 How the database updates the accounts, creates entries and commits

All of the following happens inside one database transaction:

```
BEGIN;
-- 1. claim the idempotency key (unique index makes duplicates impossible)
INSERT INTO idempotency_keys (user_id, key, request_hash)
VALUES ('u_asha', '4e1f6c0a-...', 'sha256-of-body') ON CONFLICT DO NOTHING;

-- 2. lock both accounts in a fixed order
SELECT id, balance_paise, status, owner_id FROM accounts
WHERE id IN ('acc_A','acc_B') ORDER BY id FOR UPDATE;

-- (application code now checks owner, status, 500000 >= 100000)

-- 3. the financial transaction header
INSERT INTO transactions (id, type, amount_paise, from_account_id, to_account_id, initiated_by)
VALUES ('T3','TRANSFER',100000,'acc_A','acc_B','u_asha');

-- 4. update balances
UPDATE accounts SET balance_paise = balance_paise - 100000 WHERE id = 'acc_A';  -- 500000 -> 400000
UPDATE accounts SET balance_paise = balance_paise + 100000 WHERE id = 'acc_B';  -- 200000 -> 300000

-- 5. double-entry ledger lines
INSERT INTO ledger_entries (transaction_id, account_id, direction, amount_paise, balance_after_paise)
VALUES ('T3','acc_A','DEBIT', 100000, 400000),
       ('T3','acc_B','CREDIT',100000, 300000);

-- 6. audit + store the response for future replays
INSERT INTO audit_logs (actor_user_id, action, entity_type, entity_id, outcome) VALUES (...);
UPDATE idempotency_keys SET response_status=201, response_body='{...}', transaction_id='T3' WHERE ...;
COMMIT;      -- everything becomes permanent together, or nothing does
```

At `COMMIT`, the database makes all of these changes visible and permanent **at once**. Other users can never observe “A has lost money but B hasn’t received it”.

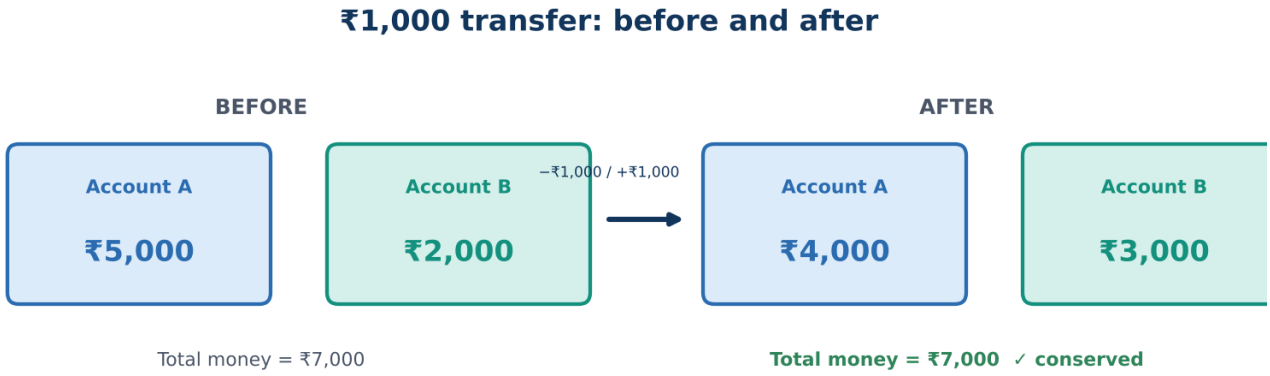
4.7 The response the frontend receives

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "transaction_id": "T3",
  "reference": "TXN-000003",
  "type": "TRANSFER",
  "amount": "1000.00",
  "from_account": { "id": "acc_A", "balance_after": "4000.00" },
  "to_account_id": "acc_B",
  "created_at": "2026-11-06T14:42:10Z"
}
```

The frontend updates the balance shown on screen using `balance_after` from the server (never by subtracting locally) and shows “Transfer successful”.

4.8 Before and after



entry	transaction	account	direction	amount	balance after
#5	T3 (TRANSFER)	A	DEBIT	₹1,000	₹4,000
#6	T3 (TRANSFER)	B	CREDIT	₹1,000	₹3,000

New ledger entries added by this transfer. Debits ₹1,000 = Credits ₹1,000 ✓ Balanced

Figure 4.2 *— Balances and ledger entries before and after the transfer. The total stays ₹7,000 and the new entries balance.*

Item	Before	After
Account A	₹5,000	₹4,000
Account B	₹2,000	₹3,000
A + B	₹7,000	₹7,000 ✓
Transactions	T1, T2	T1, T2, **T3**
Ledger entries	#1 – #4	#1 – #4, **#5 DEBIT A ₹1,000, #6 CREDIT B ₹1,000**
Audit log	…	+ “TRANSFER SUCCESS T3”

4.9 What happens if an error occurs?

When it fails	Example	Result	HTTP
Before DB work	Amount is −500; token expired	Rejected immediately; nothing written (a DENIED/failed attempt can still be audit-logged)	422 / 401
Authorization	Asha tries to debit Bharat’s account	Rejected; audit log records DENIED	403 / 404
Business rule	Asha has ₹500 but sends ₹1,000	Transaction rolled back; clear message “Insufficient balance”	422
Mid-transaction	Ledger insert fails or server crashes after debiting A	Database ROLLBACK: A still ₹5,000, B still ₹2,000, no entries	500
Network lost after commit	Transfer committed but the response never reached the phone	Client retries with the **same** Idempotency-Key; server replays the saved 201 — no second transfer	201 (replayed)
Duplicate click	Two identical requests	One executes; the other receives the same stored response	201 / 201

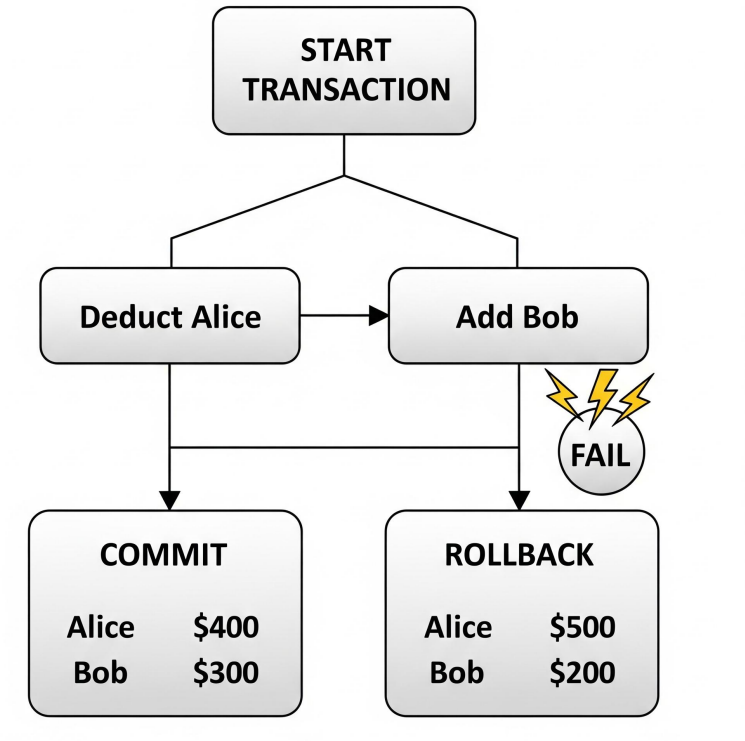


Figure 4.3 *— Commit vs. rollback: if "Add Bob" fails, the earlier deduction is undone. (Supplied image; Alice/Bob in $ — read as ₹ and A/B.)*

5. Financial Transaction Rules

This is the technical core of LedgerLine. Eight rules, each explained the same way: **definition → real-world example → incorrect implementation → correct implementation → diagram → numbers → code**. Rules 1–5 are explicitly listed as requirements in your problem statement; 6–8 are the deeper principles that those requirements rely on and that your tests will verify ("balance conservation" and "ledger consistency" appear in the verification strategy).

#	Rule	One-line meaning	Status
1	Atomicity	All steps happen or none do	CONFIRMED
2	Double-entry ledger	Every transfer = equal debit + credit	CONFIRMED
3	Idempotency	Same request twice = same effect once	CONFIRMED
4	Concurrency	Simultaneous requests cannot corrupt balances	CONFIRMED
5	Immutable records	History is never edited; corrections are new entries	CONFIRMED
6	Balance conservation	Money is never created or lost by accident	CONFIRMED (problem + tests)
7	Isolation	Overlapping transactions do not see each other's half-done work	Technical foundation of 4
8	Consistency	Constraints keep the database in a valid state	Technical foundation of all

5.1 Atomicity

DEFINITION

Atomicity means a group of operations is treated as one indivisible unit: either **every** step succeeds and is saved, or **none** is. The word comes from "atom" — originally thought indivisible.

REAL-WORLD EXAMPLE

Bank counter: the clerk takes your ₹1,000 note *and* credits your friend's passbook. If the power fails after taking your note but before writing the passbook, the sensible bank gives you your note back. The two actions are one act.

INCORRECT IMPLEMENTATION

Code runs the steps as separate, independent saves:
`UPDATE accounts SET balance = balance - 1000 WHERE id = A;` ← saved immediately
… server crashes …
`UPDATE accounts SET balance = balance + 1000 WHERE id = B;` ← never runs
Result: A = ₹4,000, B = ₹2,000, total ₹6,000 — **₹1,000 has vanished**, and no ledger entry explains it.

CORRECT APPROACH

Open a **database transaction** (`BEGIN`), do *all* steps — both balance updates, the transaction header, both ledger entries, the audit line, the idempotency record — and then `COMMIT`. If any step throws an error, or the process dies, the database performs `ROLLBACK` and undoes everything done since `BEGIN`.

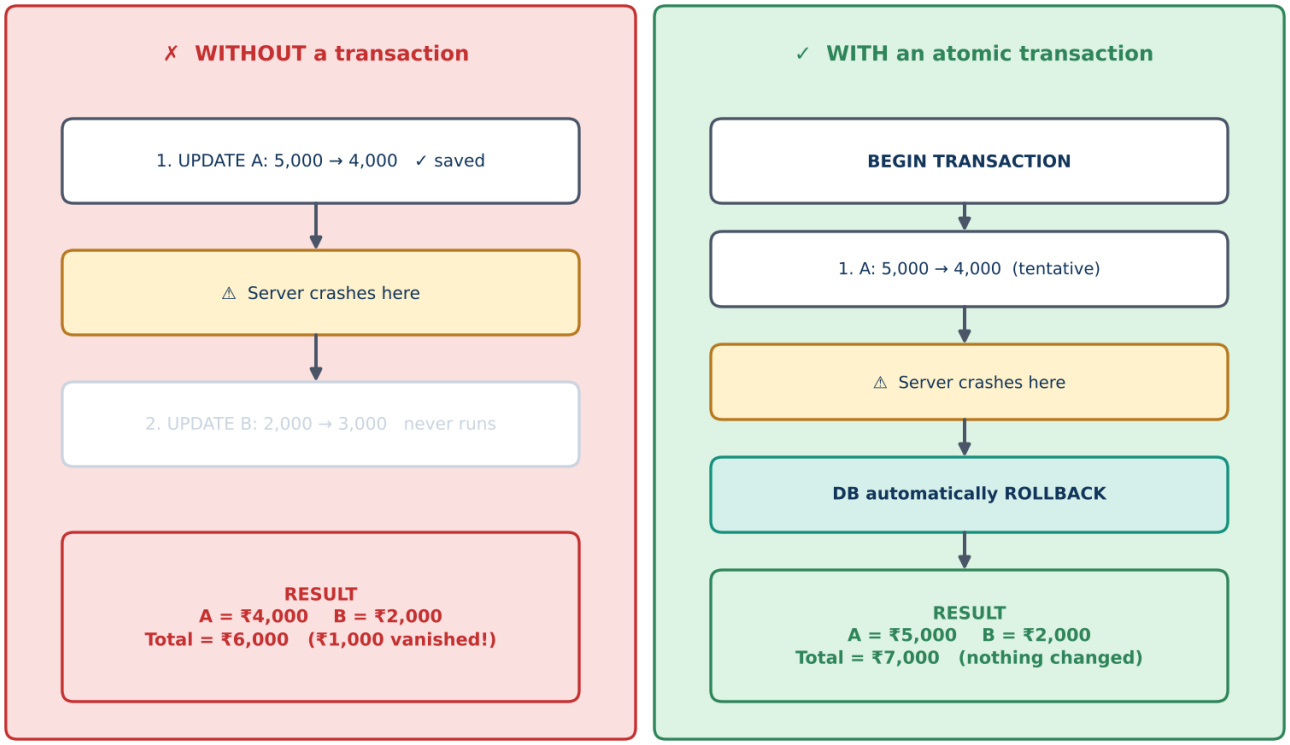


Figure 5.1 *— Without a transaction the crash leaves a half-done transfer; with a transaction the database restores the starting state.*

NUMERICAL EXAMPLE

Start: A = ₹5,000, B = ₹2,000 (total ₹7,000). Transfer ₹1,000 then crash while crediting B.
No transaction: A = ₹4,000, B = ₹2,000 → total ₹6,000 ✗
With transaction: rolled back → A = ₹5,000, B = ₹2,000 → total ₹7,000 ✓

How rollback works internally (simplified)

PostgreSQL does not overwrite rows in place. Each change creates a *new version* of the row tagged with the ID of the transaction that made it. Other sessions ignore versions belonging to transactions that have not committed. On `COMMIT` the transaction is marked committed, instantly making all its versions visible together; on `ROLLBACK` (or a crash) it is marked aborted and its versions are simply ignored and later cleaned up. A write-ahead log (WAL) written to disk before commit makes this survive power loss — that property is called **durability**.

Atomicity, **C**onsistency, **I**solation and **D**urability together are known as the **ACID** properties of database transactions. Sections 5.1, 5.7 and 5.8 cover A, I and C.

Backend implementation

```
def transfer(session, actor, req, idem_key):
    # "with session.begin()" = BEGIN ... COMMIT, and ROLLBACK if ANY exception escapes
    with session.begin():
        claim_idempotency_key(session, actor.id, idem_key, req)
        a, b = lock_accounts(session, req.from_id, req.to_id)      # SELECT ... FOR UPDATE
        check_owner_active_and_funds(actor, a, b, req.amount_paise)
        txn = insert_transaction(session, "TRANSFER", req, actor.id)
        move_balance(session, a, -req.amount_paise)
        move_balance(session, b, +req.amount_paise)
        insert_ledger_entries(session, txn.id, [
            (a.id, "DEBIT",  req.amount_paise),
            (b.id, "CREDIT", req.amount_paise)])
        insert_audit(session, actor.id, "TRANSFER", txn.id, "SUCCESS")
        save_idempotent_response(session, actor.id, idem_key, 201, txn)
    # leaving the block normally = COMMIT. An exception = ROLLBACK, nothing persisted.
```

WARNING

Common mistake: calling `commit()` after every single statement (or using "autocommit"), or catching an exception inside the block and carrying on. Either one silently breaks atomicity.

5.2 Double-Entry Ledger

DEFINITION

A **double-entry ledger** records every financial event as at least two entries — **debits** and **credits** — whose totals are equal. The ledger is "balanced" when Σ debits = Σ credits.

REAL-WORLD EXAMPLE

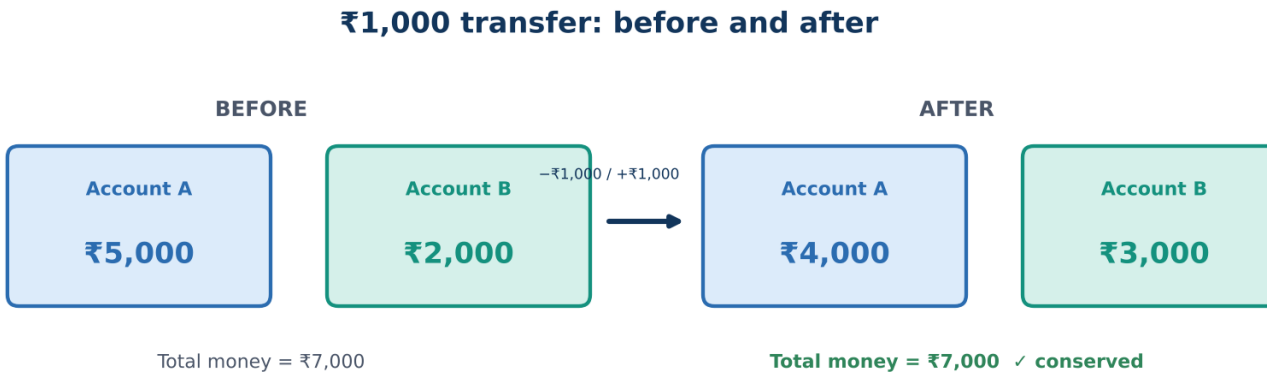
Asha pays Bharat ₹1,000: Asha's side goes down (debit), Bharat's side goes up (credit). A passbook for each shows one line; the bank's master ledger shows both lines linked by one transaction ID.

INCORRECT IMPLEMENTATION

Only updating balances: `balance_A -= 1000; balance_B += 1000;` with **no entries**.
Problems: (1) no history — nobody can tell *why* the balance changed; (2) no way to detect corruption; (3) bugs such as `balance_B += 1100` are invisible; (4) statements cannot be produced.
Also wrong: a *single* entry row "A→B ₹1,000". One row per transfer cannot represent deposits/withdrawals consistently and cannot be summed per account easily.

CORRECT APPROACH

For every transaction insert one **ledger_entries** row per affected account, with `direction` DEBIT/CREDIT and the same `transaction_id`. Enforce **Σ debits = Σ credits per transaction** in code *and* in the database (a deferred constraint trigger, see below). The account balance is a *cache* of `Σ credits − Σ debits`; an integrity job proves they agree.



entry	transaction	account	direction	amount	balance after
#5	T3 (TRANSFER)	A	DEBIT	₹1,000	₹4,000
#6	T3 (TRANSFER)	B	CREDIT	₹1,000	₹3,000

New ledger entries added by this transfer. Debits ₹1,000 = Credits ₹1,000 ✓ Balanced

Figure 5.2 *— The same picture as Section 4: entries #5 and #6 are the two halves of transaction T3.*

NUMERICAL EXAMPLE

Transfer ₹1,000: entries = DEBIT A 1,000 + CREDIT B 1,000 → debits 1,000 = credits 1,000 ✓
Bug example: code accidentally credits B ₹1,100: debits 1,000 ≠ credits 1,100 → the trigger raises an error → whole database transaction rolls back → the bug is caught immediately instead of silently creating ₹100.
Ledger replay: Σ credits − Σ debits for A = 5,000 − 1,000 = **4,000** = stored balance ✓

Database enforcement of balanced entries (PROPOSED, PostgreSQL)

```
CREATE FUNCTION check_txn_balanced() RETURNS trigger AS $$
DECLARE d BIGINT; c BIGINT;
BEGIN
  SELECT COALESCE(SUM(amount_paise) FILTER (WHERE direction = 'DEBIT'),  0),
         COALESCE(SUM(amount_paise) FILTER (WHERE direction = 'CREDIT'), 0)
    INTO d, c FROM ledger_entries WHERE transaction_id = NEW.transaction_id;
  IF d <> c THEN
    RAISE EXCEPTION 'Transaction % unbalanced: debits=% credits=%', NEW.transaction_id, d, c;
  END IF;
  RETURN NULL;
END; $$ LANGUAGE plpgsql;

-- DEFERRABLE INITIALLY DEFERRED = checked at COMMIT, after ALL entries of the txn were inserted
CREATE CONSTRAINT TRIGGER ledger_must_balance
AFTER INSERT ON ledger_entries
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION check_txn_balanced();
```

Why *deferred*? Inserting the debit row alone is momentarily unbalanced. By delaying the check until `COMMIT`, we validate the finished set of entries.

Application-level check

```
def insert_ledger_entries(session, txn_id, lines):         # lines = [(account_id, "DEBIT"|"CREDIT", paise)]
    debits  = sum(p for _, d, p in lines if d == "DEBIT")
    credits = sum(p for _, d, p in lines if d == "CREDIT")
    if debits != credits or debits <= 0:
        raise LedgerImbalance(f"debits={debits} credits={credits}")   # -> ROLLBACK
    for account_id, direction, paise in lines:
        ...  # INSERT INTO ledger_entries (...)
```

5.3 Idempotency

DEFINITION

An operation is **idempotent** if doing it once or many times has the same final effect as doing it once. In APIs, an **idempotency key** is a unique token the client attaches to a request so the server can recognise “this is a retry of the same request”.

REAL-WORLD EXAMPLE

You tap “Pay ₹500” at a shop. The screen hangs. You tap again. A well-built payment app charges you **once** and shows the same receipt both times. Without protection you would pay ₹1,000 for a ₹500 item.

Why duplicates happen

- **Double-click / double-tap** by an impatient user.
- **Network timeout:** the server completed the transfer but the response got lost; the app, not knowing, retries.
- **Automatic retries** by browsers, proxies, load balancers or client libraries.
- **Message re-delivery** by queues in larger systems.

INCORRECT IMPLEMENTATION

Server simply processes every incoming transfer request.
Request 1: A −₹1,000, B +₹1,000. Response lost. Request 2 (retry): A −₹1,000, B +₹1,000 again.
Result: A = ₹3,000, B = ₹4,000. The user wanted one transfer, got two. **Money moved twice.**

CORRECT APPROACH

1. Client generates a UUID `Idempotency-Key` per user intention and sends it in a header; retries reuse the **same** key.
2. Server stores `(user_id, key, request_hash, response)` in `idempotency_keys` with a **UNIQUE (user_id, key)** index.
3. First arrival → process and save the response. Later arrival with same key and same payload → return the saved response, **do not touch balances**.
4. Same key but a *different* payload (e.g., ₹9,000 instead of ₹1,000) → reject with an error: the key is being misused.

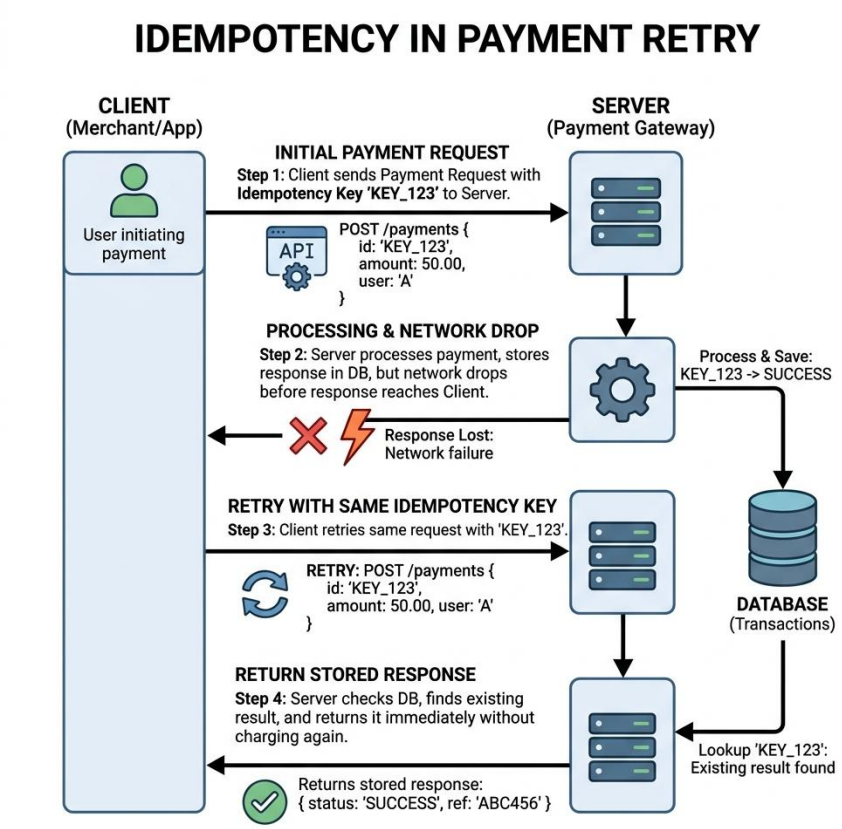


Figure 5.3 *— Retry with the same idempotency key returns the stored result without charging again. (Supplied image.)*

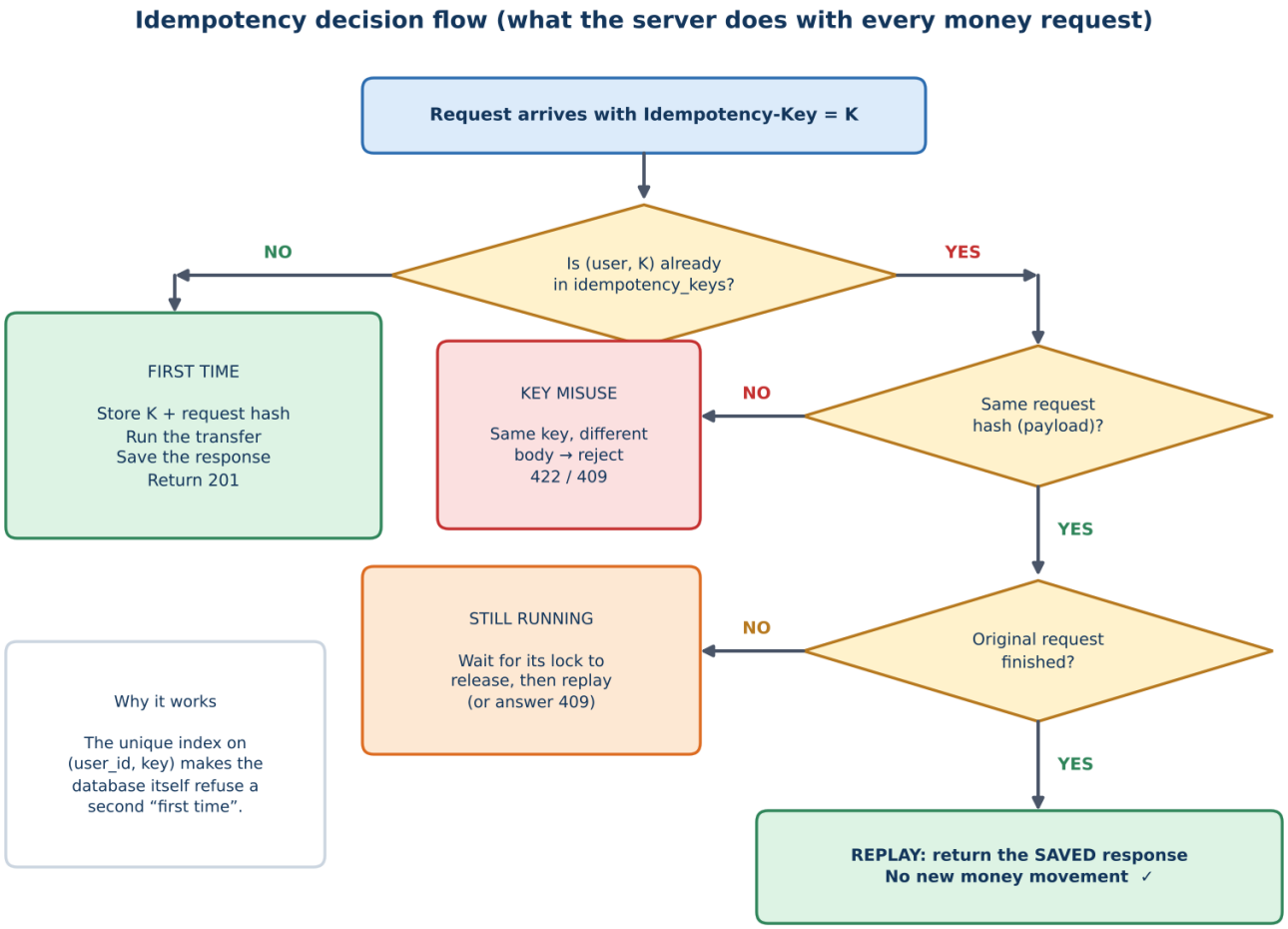


Figure 5.4 *— The server's decision flow for every money request.*

NUMERICAL EXAMPLE

Balance A = ₹5,000. Request with key K transfers ₹1,000.
Attempt 1 (key K): executes → A = ₹4,000. Response lost.
Attempt 2 (key K): key found, payload hash identical → stored 201 replayed. A stays **₹4,000** ✓
Attempt 3 (key K, amount ₹2,500): same key, different hash → **422 "Idempotency-Key reused with different request"**; A stays ₹4,000.

Backend / database implementation (PROPOSED)

```
-- unique per user + key: the DATABASE refuses a second "first time"
ALTER TABLE idempotency_keys ADD CONSTRAINT uq_idem UNIQUE (user_id, key);

-- inside the money transaction, first statement:
INSERT INTO idempotency_keys (user_id, key, request_hash)
VALUES (:uid, :key, :hash)
ON CONFLICT (user_id, key) DO NOTHING
RETURNING id;          -- row returned => we are the first; no row => someone has used this key
```

```
def claim_idempotency_key(session, user_id, key, request) -> Replay | None:
    h = sha256(canonical_json(request)).hexdigest()
    row = session.execute(text("""INSERT INTO idempotency_keys (user_id, key, request_hash)
                                  VALUES (:u, :k, :h) ON CONFLICT (user_id, key) DO NOTHING
                                  RETURNING id"""), dict(u=user_id, k=key, h=h)).first()
    if row:                       # first time: caller continues with the transfer
        return None
    old = session.execute(text("SELECT request_hash, response_status, response_body "
                               "FROM idempotency_keys WHERE user_id=:u AND key=:k"), dict(u=user_id, k=key)).one()
    if old.request_hash != h:
        raise IdempotencyKeyReuse()           # HTTP 422
    return Replay(old.response_status, old.response_body)   # HTTP replay of the saved answer
```

TECHNICAL IMPLEMENTATION (how we make it true)

Race between two identical requests: because both try to insert the same unique key, PostgreSQL makes the second `INSERT` **wait** until the first transaction commits or rolls back. When the first commits, the second does nothing (`ON CONFLICT DO NOTHING`) and then reads the saved response. So even two *simultaneous* duplicates are safe.
If the first attempt fails and rolls back, its key row disappears too, so a later retry is a fresh attempt — correct, because no money moved.
Alternative design used by large payment providers: commit the key as IN_PROGRESS first and finish later; it gives better crash handling but needs extra recovery code. The simpler design above is enough for LedgerLine.

NOTE

Keys are normally stored for a limited time (e.g., 24 hours) and then cleaned up. Do **not** use a key based on the amount or time — it must be random and generated per user intention, so two *genuine* identical transfers (e.g., two separate ₹500 payments) are not mistaken for duplicates.

5.4 Concurrency

DEFINITION

Concurrency means several operations are in progress at overlapping times. A **race condition** is a bug where the result depends on the unlucky timing of those operations. A **lock** forces operations that touch the same data to take turns.

REAL-WORLD EXAMPLE

A joint account holds ₹1,000. Husband withdraws ₹800 at an ATM while wife pays ₹800 online — the same second. The bank must let exactly one succeed.

INCORRECT IMPLEMENTATION

“Read–check–write” without protection:
`balance = SELECT balance … if balance >= 800: UPDATE … SET balance = balance - 800`
Request 1 and Request 2 both read ₹1,000, both pass the check, both write. The bank has paid ₹1,600 from ₹1,000. This is the **lost update / check-then-act race**.

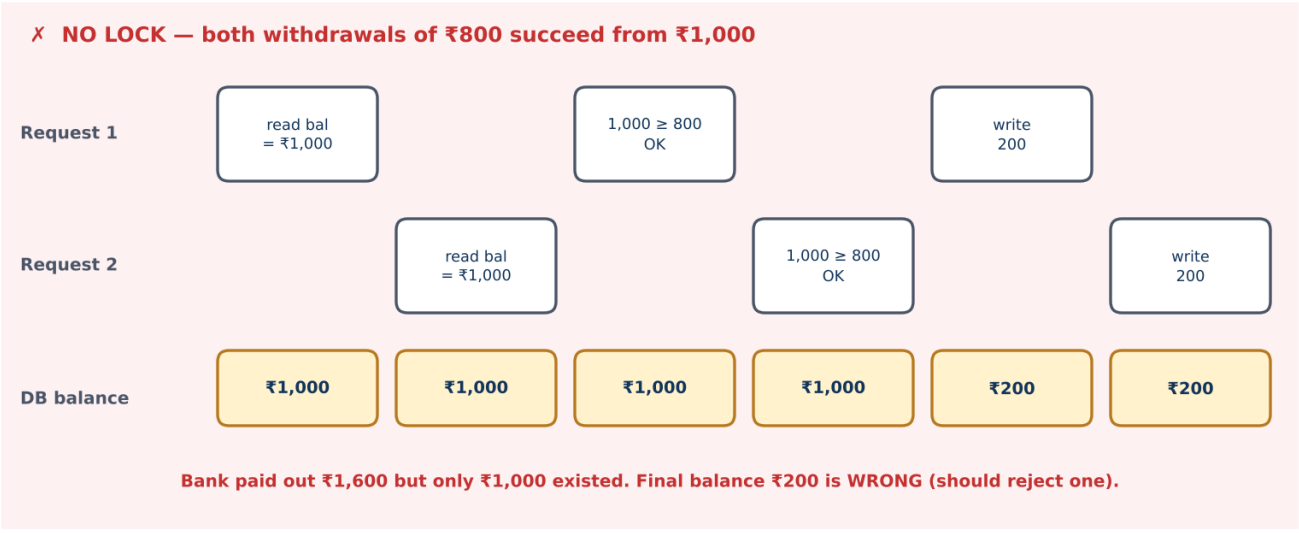


Figure 5.5 *— Top: both withdrawals succeed (bug). Bottom: the row lock makes the second request wait, then see the real balance and reject.*

NUMERICAL EXAMPLE

Balance ₹1,000; two simultaneous withdrawals of ₹800.
No lock: final balance ₹200 but ₹1,600 left the bank ✗ (₹600 created from nothing).
Row lock: first → ₹200. Second waits, then reads ₹200 < ₹800 → rejected ✓. Total paid out ₹800.

Ways to make balance updates safe

Technique	How it works	Pros	Cons
Pessimistic locking `SELECT … FOR UPDATE`	Lock the row *first*; others wait until commit.	Simple to reason about; ideal for money hot-spots.	Waiting can reduce throughput; risk of deadlocks if lock order differs.
Atomic conditional UPDATE	`UPDATE … SET b = b - :x WHERE id=:id AND b >= :x`; check rows-affected.	One statement; no explicit lock code; very fast.	Harder when many rules must be checked together.
Optimistic locking (version column)	Read with version; update only `WHERE version = :v`; retry if 0 rows.	No waiting; good when conflicts are rare.	Retries; frequent conflicts on popular accounts waste work.
SERIALIZABLE isolation	DB detects conflicting patterns and aborts one transaction; app retries.	Strongest guarantee, least custom logic.	Requires retry loop; more aborts.

PROPOSED DESIGN (not a confirmed requirement)

Proposed for LedgerLine: **pessimistic row locks (`FOR UPDATE`) as the main mechanism**, plus the `CHECK (balance_paise >= 0)` constraint as a safety net. They are easy to explain in an interview and easy to test.

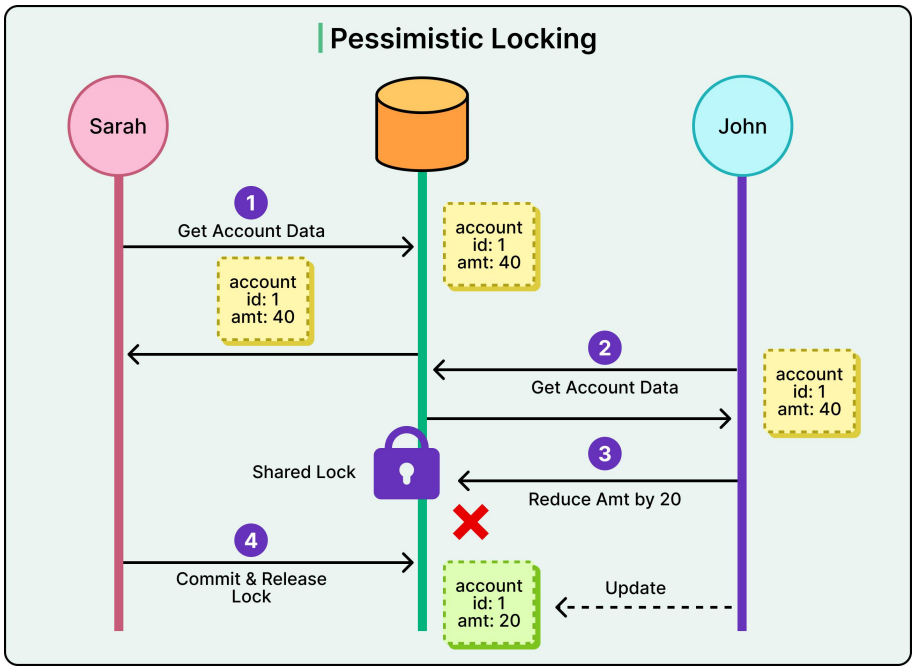


***Figure 5.6** — Pessimistic locking: Sarah's transaction holds a lock, so John's update cannot proceed until she commits. (Supplied image. It labels the lock "Shared"; for read-modify-write on balances we take an **exclusive** row lock with `FOR UPDATE`, which also makes the second *reader* wait.)*

Deadlocks — the trap when locking two accounts

A transfer must lock **two** rows. Suppose transfer 1 is A→B and transfer 2 is B→A at the same moment. If T1 locks A and T2 locks B, then T1 waits for B while T2 waits for A — forever. That is a **deadlock**. PostgreSQL detects it after about a second and aborts one transaction, but you should *prevent* it: **always lock accounts in the same global order** (for example sorted by account ID).

```
def lock_accounts(session, id1, id2):
    ids = sorted([id1, id2])                    # same order for every transfer, whatever the direction
    rows = session.execute(text("""SELECT id, owner_id, balance_paise, status
                                   FROM accounts WHERE id = ANY(:ids) ORDER BY id FOR UPDATE"""),
                           dict(ids=ids)).all()
    if len(rows) != 2: raise AccountNotFound()
    by_id = {r.id: r for r in rows}
    return by_id[id1], by_id[id2]
```

```
-- withdrawal using the atomic conditional update (alternative / extra safety)
UPDATE accounts
   SET balance_paise = balance_paise - :amt
 WHERE id = :id AND status = 'ACTIVE' AND balance_paise >= :amt;
-- rows affected = 0  ->  insufficient funds (or frozen); nothing changed
```

5.5 Immutable Records

DEFINITION

A record is **immutable** if, once written, it is never changed or deleted. The ledger is **append-only**: you may only add new lines at the end. Mistakes are corrected by adding **compensating (reversal) entries**, never by editing history.

REAL-WORLD EXAMPLE

An accountant who makes an error in a bound ledger book does not tear out the page or use correction fluid; they write a new line reversing the wrong one and then the correct one. Anyone reading can follow exactly what happened.

INCORRECT IMPLEMENTATION

`UPDATE ledger_entries SET amount_paise = 50000 WHERE id = 5;` or `DELETE FROM ledger_entries WHERE id = 6;`

Problems: fraud becomes undetectable; balances no longer match history; statements already given to customers become lies; auditors cannot trust anything.

CORRECT APPROACH

Treat `transactions`, `ledger_entries` and `audit_logs` as **append-only**:

• No UPDATE/DELETE endpoints in the API. • The application's database user has no UPDATE/DELETE privilege on them. • A database trigger rejects any UPDATE/DELETE even from a bug.

To correct a mistake, create a **REVERSAL** transaction that posts the opposite entries and references the original.

✗ Editing history

```
UPDATE ledger_entries
   SET amount = 500
 WHERE id = 5;


DELETE FROM ledger_entries
 WHERE id = 6;
```

History rewritten, nobody can tell

✓ Compensating (reversal) transaction

1. Leave T3 exactly as it is.
2. Create NEW transaction T4 of type REVERSAL with reverses_transaction_id = T3.
3. T4 posts the opposite entries.
4. Audit log records WHO reversed it and WHY.

entry	txn	account	direction	amount	balance after	note
#5	T3	A	DEBIT	₹1,000	₹4,000	original transfer
#6	T3	B	CREDIT	₹1,000	₹3,000	original transfer
#7	T4	B	DEBIT	₹1,000	₹2,000	REVERSAL of T3
#8	T4	A	CREDIT	₹1,000	₹5,000	REVERSAL of T3

Net effect: A back to ₹5,000, B back to ₹2,000 — and the full story stays visible forever.

Figure 5.7 *— Left: forbidden edits. Right: the original transfer T3 stays; reversal T4 posts opposite entries and the net effect is zero.*

NUMERICAL EXAMPLE

T3: A −₹1,000 (→ ₹4,000), B +₹1,000 (→ ₹3,000). Admin finds it was the wrong beneficiary.

T4 (REVERSAL of T3): DEBIT B ₹1,000 (→ ₹2,000), CREDIT A ₹1,000 (→ ₹5,000). Debits = credits ✓

Final: A ₹5,000, B ₹2,000; four entries remain visible forever; audit log shows who reversed and why.

Reversal rules (PROPOSED)

- A transaction can be reversed **at most once** — enforce with a UNIQUE index on `transactions.reverses_transaction_id`.
- A reversal needs a **reason** and an **authorised role** (Admin in our proposal), and is audit-logged.
- The reversed account must still have enough money (or the business must decide the policy: allow overdraft for reversals? That is a product decision to confirm).
- A reversal is itself a transaction and cannot be edited; reversing a reversal is a *new* correction.

```
CREATE FUNCTION forbid_mutation() RETURNS trigger AS $$
BEGIN
  RAISE EXCEPTION '% on % is not allowed: financial records are append-only', TG_OP, TG_TABLE_NAME;
END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER ledger_entries_immutable BEFORE UPDATE OR DELETE ON ledger_entries
FOR EACH ROW EXECUTE FUNCTION forbid_mutation();
CREATE TRIGGER transactions_immutable   BEFORE UPDATE OR DELETE ON transactions
FOR EACH ROW EXECUTE FUNCTION forbid_mutation();
CREATE TRIGGER audit_logs_immutable     BEFORE UPDATE OR DELETE ON audit_logs
FOR EACH ROW EXECUTE FUNCTION forbid_mutation();
-- also block TRUNCATE (statement-level trigger) and REVOKE UPDATE, DELETE from the app's DB role

CREATE UNIQUE INDEX one_reversal_per_txn ON transactions (reverses_transaction_id)
WHERE reverses_transaction_id IS NOT NULL;
```

NOTE

Note: `accounts.balance_paise` **does** change (it is a cache). Immutability applies to the *records of events*, not to the running total. That is exactly why we verify the cache against the ledger.

5.6 Balance Conservation

DEFINITION

Balance conservation: money is never created or destroyed by a transfer. What leaves one account arrives in another. Deposits and withdrawals change the total only by exactly their amount, and that change is mirrored elsewhere in the books.

REAL-WORLD EXAMPLE

In a closed room with ₹7,000 in two wallets, passing ₹1,000 from one wallet to the other leaves ₹7,000 in the room. If you count ₹6,000 or ₹8,000 afterwards, someone made an error — or stole.

INCORRECT IMPLEMENTATION

`balance_A -= 1000` succeeds but `balance_B += 100` (typo) → ₹900 vanishes.
Or retry bug: `balance_B += 1000` runs twice → ₹1,000 appears from nowhere.
With single-entry storage, nothing alerts you.

CORRECT APPROACH

Use **integer paise** (no floats) so arithmetic is exact. Perform both sides from the same `amount_paise` variable. Use atomic transactions + balanced entries + idempotency. Run a **verification query** that checks the invariants and call it in tests and from the Auditor screen.

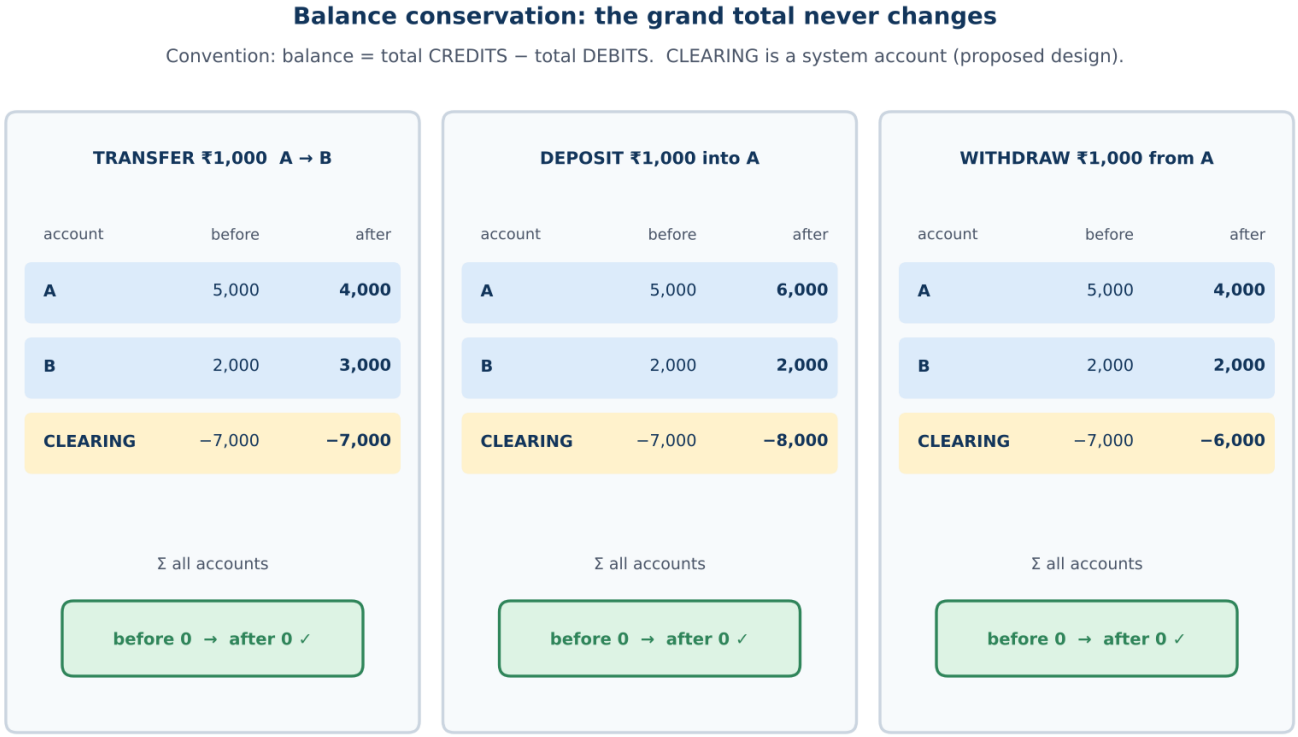


Figure 5.8 *— The grand total over all accounts including the system CLEARING account stays at ₹0 in every scenario.*

NUMERICAL EXAMPLE

Transfer: A+B = 7,000 before and after.
Deposit ₹1,000 into A: customers 7,000 → 8,000; CLEARING −7,000 → −8,000; **sum of all = 0** ✓
Why zero? With balance = credits − debits and every transaction balanced, the sum over all accounts equals total credits − total debits = 0.

Integrity queries (PROPOSED)

```
-- 1. Every transaction is balanced (expect 0 rows)
SELECT transaction_id,
       SUM(amount_paise) FILTER (WHERE direction='DEBIT')  AS debits,
       SUM(amount_paise) FILTER (WHERE direction='CREDIT') AS credits
FROM ledger_entries GROUP BY transaction_id
HAVING COALESCE(SUM(amount_paise) FILTER (WHERE direction='DEBIT'),0)
    <> COALESCE(SUM(amount_paise) FILTER (WHERE direction='CREDIT'),0);

-- 2. Stored balance equals ledger replay (expect 0 rows)
SELECT a.id, a.balance_paise,
       COALESCE(SUM(CASE e.direction WHEN 'CREDIT' THEN e.amount_paise ELSE -e.amount_paise END),0) AS replayed
FROM accounts a LEFT JOIN ledger_entries e ON e.account_id = a.id
GROUP BY a.id, a.balance_paise
HAVING a.balance_paise <> COALESCE(SUM(CASE e.direction WHEN 'CREDIT' THEN e.amount_paise ELSE -e.amount_paise END),0);

-- 3. Grand total over all accounts must be 0
SELECT SUM(balance_paise) FROM accounts;
```

5.7 Isolation

DEFINITION

Isolation is the “I” in ACID: concurrent database transactions should not interfere with each other. Ideally each behaves as if it were running alone. **Isolation levels** are settings that trade strictness against speed.

REAL-WORLD EXAMPLE

Two clerks update the same ledger book at once. If one can read the other’s half-written line (“A debited, B not yet credited”) and make decisions on it, chaos results. Isolation is the rule that stops this.

The problems isolation prevents

Anomaly	Meaning	Money example
Dirty read	Reading data another transaction has not committed (and may roll back).	You see A = ₹4,000 though the transfer later rolls back to ₹5,000.
Non-repeatable read	Reading the same row twice in one transaction gives different values.	Balance ₹5,000 at the start of a statement run, ₹4,000 at the end.
Phantom read	Re-running a query returns new rows that appeared meanwhile.	Statement total changes because a new entry appeared mid-report.
Lost update	Two transactions read, then both write; one overwrite hides the other.	Two withdrawals both subtract from the same stale ₹1,000.
Write skew	Two transactions each check a rule using data the other changes; both pass, jointly breaking the rule.	Two accounts together must keep ≥ ₹1,000; each withdraws, each checked the other still had money.

Which problems can happen at each isolation level? (PostgreSQL behaviour)

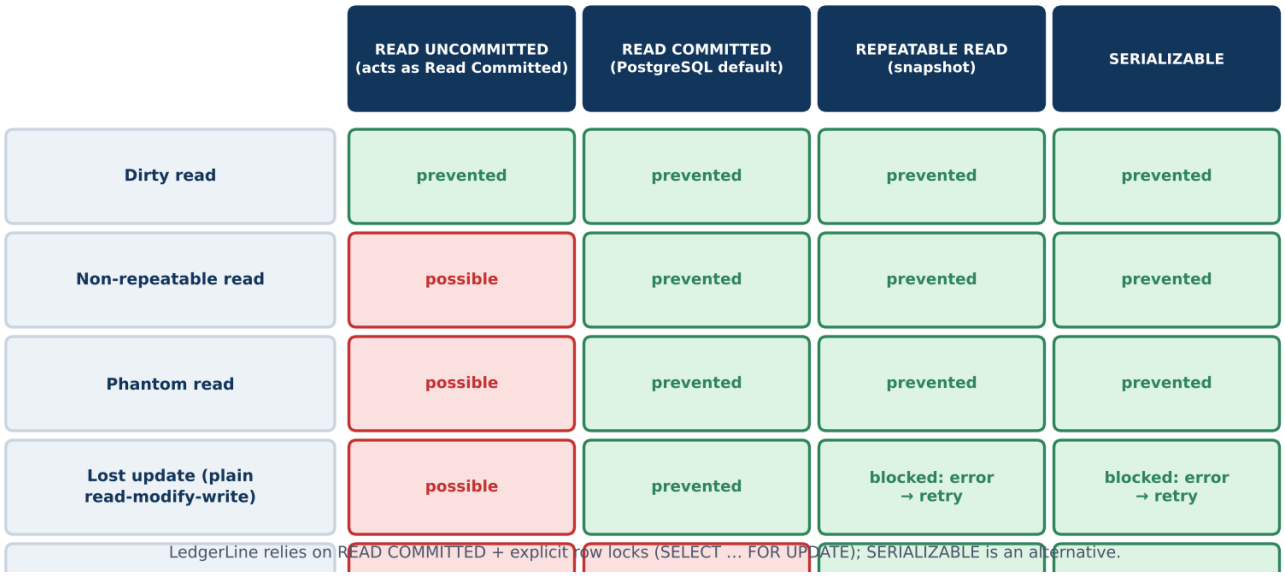


***Figure 5.9** — PostgreSQL isolation levels versus the anomalies each allows. Red = can happen, green = prevented.*

PostgreSQL's default is **READ COMMITTED**: each *statement* sees only data committed before it started. It prevents dirty reads but still allows lost updates in naïve read-modify-write code. That is why LedgerLine adds **explicit row locks** (Section 5.4). **REPEATABLE READ** gives a stable snapshot for the whole transaction (good for consistent statements/reports). **SERIALIZABLE** behaves as if transactions ran one after another; PostgreSQL may abort one with error `40001` (serialization failure) and the application must **retry** the whole transaction.

> **NUMERICAL EXAMPLE**
> Two withdrawals of ₹800 on ₹1,000 under READ COMMITTED **without** locking: both read ₹1,000 → lost update, final ₹200 ✗.
> Same under READ COMMITTED **with** `FOR UPDATE`: second waits and re-reads ₹200 → rejected ✓.
> Same under SERIALIZABLE without explicit locks: one transaction fails with 40001; the retry reads ₹200 and is rejected ✓ (needs retry code).

```
# choosing an isolation level per transaction (SQLAlchemy)
with session.begin():
    session.execute(text("SET TRANSACTION ISOLATION LEVEL REPEATABLE READ"))   # e.g. consistent statement report
    ...

# retry wrapper needed if you rely on SERIALIZABLE
def run_with_retry(fn, attempts=3):
```

```
    for i in range(attempts):
        try:
            return fn()
        except SerializationFailure:      # SQLSTATE 40001
            if i == attempts - 1: raise
```

5.8 Consistency

DEFINITION

Consistency (the “C” in ACID) means a transaction takes the database from one **valid** state to another valid state: all rules (constraints) hold before and after. If a change would break a rule, the database rejects it.

REAL-WORLD EXAMPLE

A bank rule says “customer balances can never be negative”. Even if a buggy screen asks for a withdrawal that would go negative, the database itself refuses to store it.

INCORRECT IMPLEMENTATION

Rules exist only in the UI or only in a single backend function. A second endpoint, a script, an admin tool or a future developer forgets the check → invalid data (negative balance, entry pointing to a deleted account, duplicate account number).

CORRECT APPROACH

Enforce each rule at **every** layer, with the database as the final guard: `NOT NULL`, `CHECK`, `UNIQUE`, `FOREIGN KEY`, enum types and constraint triggers.

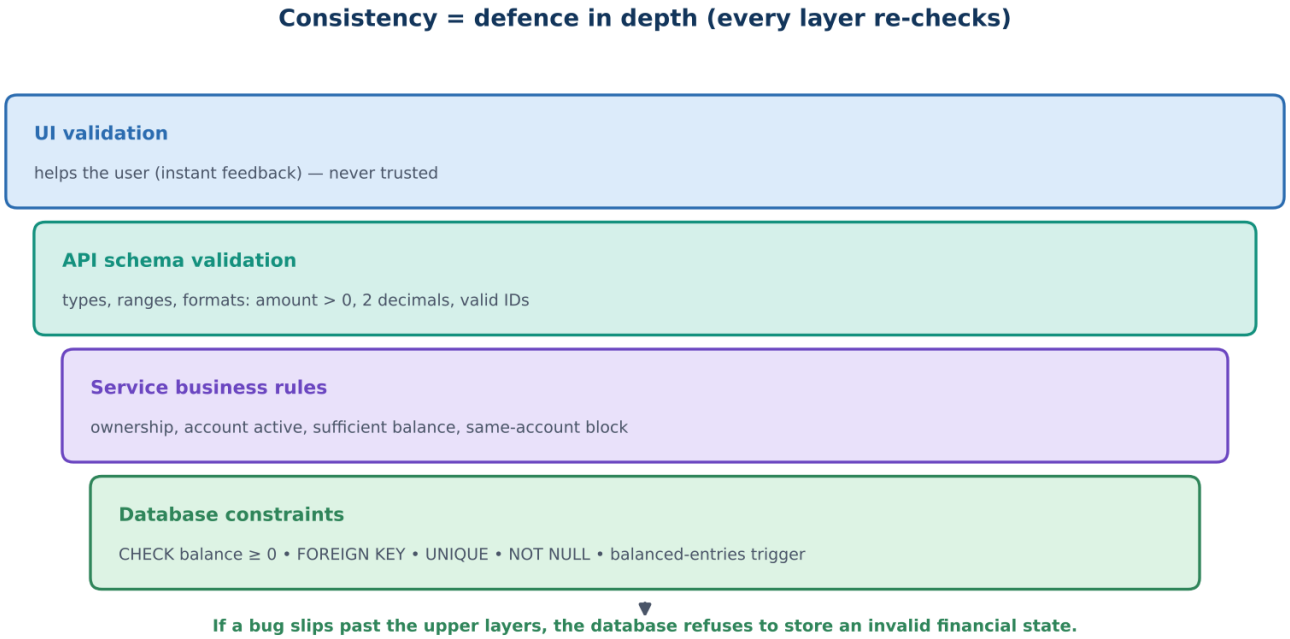


Figure 5.10 *— Defence in depth: UI → API validation → business rules → database constraints.*

Business rule	Constraint that enforces it (PROPOSED)
Amount must be positive	`CHECK (amount_paise > 0)` on transactions and ledger_entries
Customer accounts cannot be overdrawn	`CHECK (type = 'SYSTEM' OR balance_paise >= 0)` on accounts
Direction is only DEBIT or CREDIT	`CHECK (direction IN ('DEBIT','CREDIT'))`
Every entry belongs to a real transaction and account	`FOREIGN KEY (transaction_id)`, `FOREIGN KEY (account_id)`
Account numbers and emails are unique	`UNIQUE (account_number)`, `UNIQUE (email)`
One idempotency key per user	`UNIQUE (user_id, key)`
Roles come from a fixed list	`CHECK (role IN ('ACCOUNT_HOLDER','ADMIN','AUDITOR'))`

Business rule	Constraint that enforces it (PROPOSED)
Transaction entries balance	Deferred constraint trigger (Section 5.2)
History cannot be edited	Immutability triggers (Section 5.5)

```
ALTER TABLE accounts
  ADD CONSTRAINT balance_not_negative
  CHECK (type = 'SYSTEM' OR balance_paise >= 0);

-- Proof it works: this fails even if the app has a bug
UPDATE accounts SET balance_paise = balance_paise - 99999999 WHERE id = 'acc_A';
-- ERROR: new row for relation "accounts" violates check constraint "balance_not_negative"
```

> **NOTE**
>
> **Two meanings of “consistency”.** In ACID it means *obeying the database’s rules*. In distributed systems (“eventual consistency”) it means *all copies of data agree*. LedgerLine uses the first meaning. Do not mix them up in an interview.

6. Authentication and Security

CONFIRMED SECURITY REQUIREMENTS

Authenticate users before protected operations • enforce authorization **on the backend** • verify account ownership • apply Admin and Auditor permissions • validate all financial inputs • protect credentials and configuration • record security-sensitive operations.

6.1 Authentication versus authorization

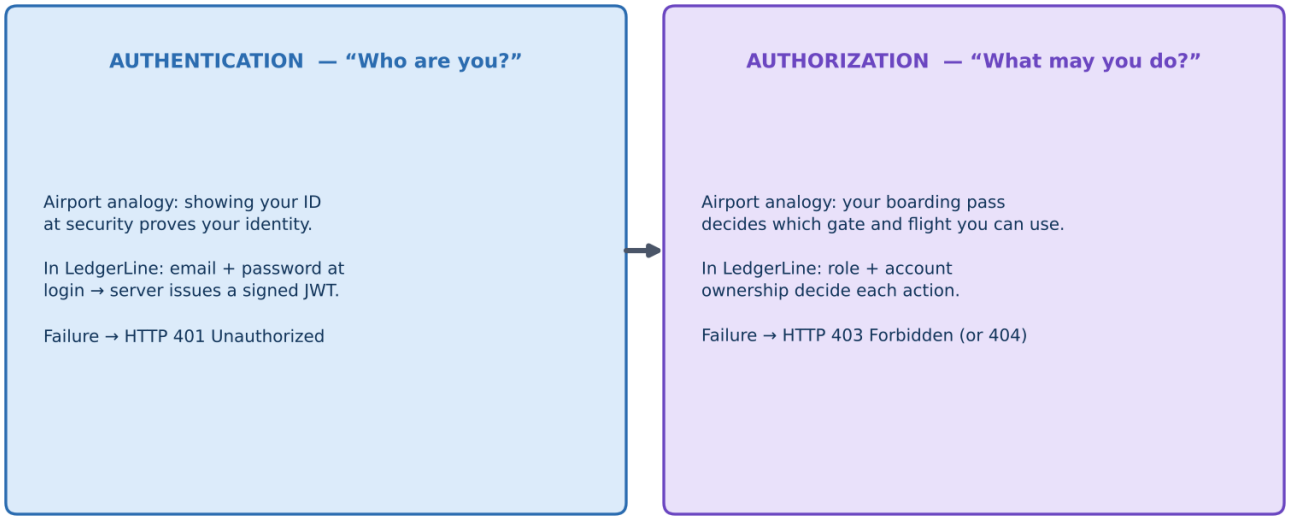


Figure 6.1 *— Two different questions. Authentication: “who are you?”. Authorization: “what are you allowed to do?”.*

	Authentication (AuthN)	Authorization (AuthZ)
Question	Who are you?	May you do this?
Proof	Password, token, OTP, fingerprint	Role, ownership, permissions, account status
When	At login, and token check on every request	On every protected action
Failure code	**401 Unauthorized** (really “unauthenticated”)	**403 Forbidden** (or 404 to hide existence)
LedgerLine example	Asha logs in with email + password	Asha may transfer from `LL0001` but not from `LL0002`

6.2 Login and password hashing

A website must never store your actual password. If its database leaks, every user would be exposed (and, since people reuse passwords, exposed elsewhere too). Instead it stores a **hash**: the output of a one-way function. You cannot compute the password back from the hash; the server only checks whether hashing what you typed gives the same result.

INCORRECT IMPLEMENTATION

Storing `password = "asha123"` in plain text.
Or using a fast general hash like plain SHA-256: attackers can test billions of guesses per second on a GPU.
Or hashing without a **salt**: identical passwords give identical hashes, and precomputed “rainbow tables” crack them instantly.

CORRECT APPROACH

Use a **slow, salted, purpose-built password hash** — **Argon2id** (modern default) or **bcrypt**. A *salt* is random data mixed in per password, so two users with the same password get different hashes. “Slow” on purpose: it makes mass guessing expensive.

```
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError
ph = PasswordHasher()                       # salted Argon2id with safe defaults

stored = ph.hash("asha-S3cure-passphrase")  # store this string in users.password_hash
# looks like: $argon2id$v=19$m=65536,t=3,p=4$<salt>$<hash>

def verify_login(email, password, db):
    user = db.get_user_by_email(email)
    try:
        ph.verify(user.password_hash if user else DUMMY_HASH, password)   # always do the work
    except VerifyMismatchError:
        user = None
    if not user or not user.is_active:
        audit(db, None, "LOGIN", "DENIED", {"email": email})
        raise HTTPException(401, "Invalid email or password")            # same message for both cases
    return user
```

Notes: return the **same generic message** whether the email or the password was wrong (otherwise attackers learn which emails exist). Add **rate limiting** or lockout after repeated failures. Require HTTPS everywhere so passwords are encrypted in transit.

6.3 JWT or session-based authentication

After login the server must recognise you on later requests without asking for the password every time. There are two common ways.

	Server-side session	JWT (JSON Web Token)
How	Server stores a session record; browser keeps a random session ID in a cookie	Server signs a small token containing user ID, role and expiry; client sends it back each time
Server state	Needs a session store	Stateless: verify the signature, no lookup needed
Logout / revoke	Easy: delete the session	Harder: token valid until it expires (use short expiry + refresh tokens or a block-list)
Typical risk	CSRF (when cookie-based)	Token theft (XSS if kept in localStorage)
Fit for LedgerLine	Perfectly valid choice	The problem statement says “JWT or session” — both are acceptable

> **PROPOSED DESIGN (not a confirmed requirement)**
>
> Proposed: **JWT access token with a short life (e.g., 15 minutes)** sent in the `Authorization: Bearer` header. A JWT has three Base64 parts: header (algorithm), payload (claims such as `sub`, `role`, `exp`) and signature. The payload is **readable by anyone** — never put secrets in it; the *signature* only prevents tampering.

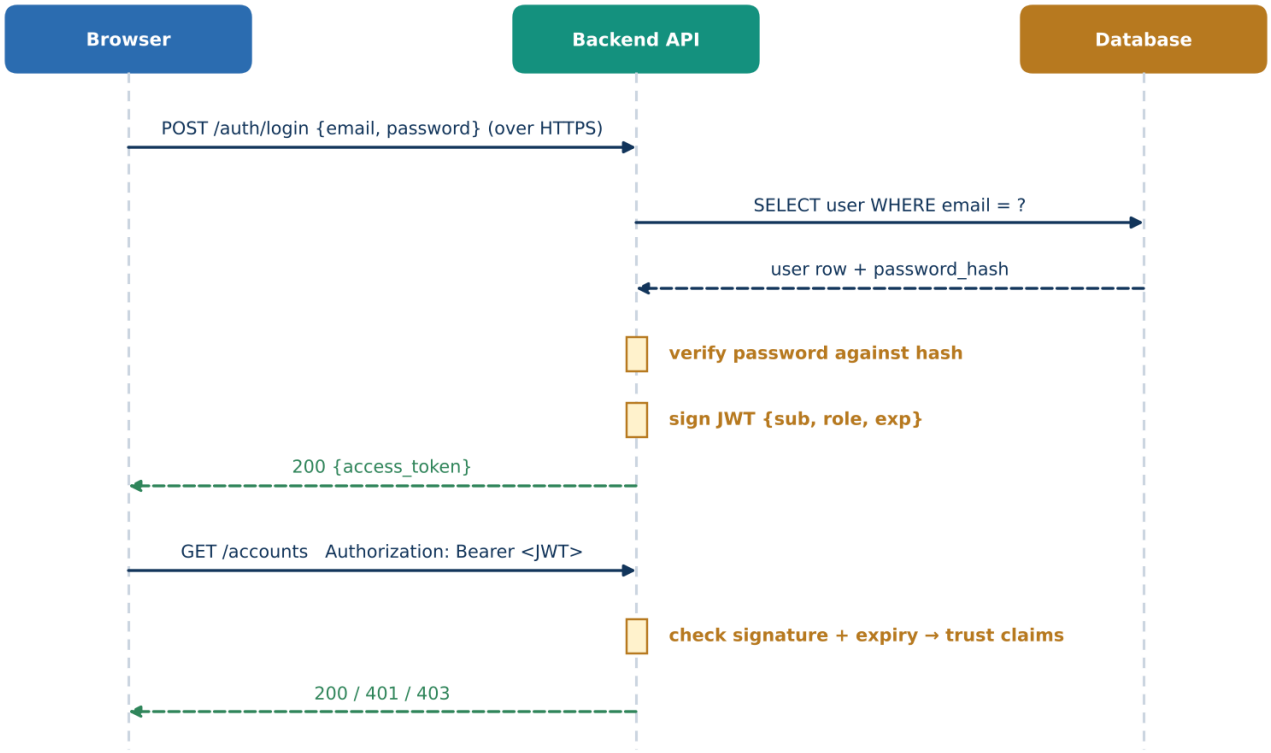


Figure 6.2 *— Login produces a signed token; every later request presents it and the server verifies signature and expiry.*

```
import jwt, datetime, os
SECRET = os.environ["JWT_SECRET"]                # long random value, never in Git

def make_token(user):
    now = datetime.datetime.now(datetime.timezone.utc)
    return jwt.encode({"sub": str(user.id), "role": user.role,
                       "iat": now, "exp": now + datetime.timedelta(minutes=15)},
                      SECRET, algorithm="HS256")

def current_user(token: str = Depends(oauth2_scheme), db=Depends(get_db)):
    try:
        claims = jwt.decode(token, SECRET, algorithms=["HS256"])   # checks signature AND expiry
    except jwt.PyJWTError:
        raise HTTPException(401, "Invalid or expired token")
    user = db.get_user(claims["sub"])                              # reload: role may have changed
    if not user or not user.is_active:
        raise HTTPException(401, "Account disabled")
    return user
```

WARNING

Always pass an explicit `algorithms=[…]` list when decoding. Never accept “alg: none”. Never decode a token and trust it without verifying the signature.

6.4 Role-based access control (RBAC)

RBAC assigns each user a **role**, and each role a set of permitted actions. The server looks up the role instead of deciding person by person. LedgerLine has three roles named in the problem statement.

Role-based access control matrix (PROPOSED — confirm with the task owner)

	Account Holder	Admin	Auditor
Log in / view own profile	✓	✓	✓
View OWN accounts & balance	✓	✗	✗
View ANY account	✗	✓	✓
Open an account for self	✓	✓	✗
Freeze / close any account	✗	✓	✗
Deposit / withdraw on OWN account	✓	✗	✗
Transfer from OWN account	✓	✗	✗
Transfer from SOMEONE ELSE's account	✗	✗	✗
View OWN transaction history	✓	✗	✗
View ALL ledger entries	✗	✓	✓
Reverse a transaction (compensating)	✗	✓	✗
Read audit logs / run integrity check	✗	✗	✓
Edit or delete a ledger entry	✗	✗	✗

"Nobody can move customers' money" is deliberately NO here — admins manage accounts; corrections go through audited reversals.

Figure 6.3 *— Proposed permission matrix. Treat it as a draft: confirm particularly whether Admin may reverse transactions or move money.*

```
def require_roles(*roles):
    def dependency(user = Depends(current_user)):
        if user.role not in roles:
            audit_denied(user, "ROLE_DENIED")
            raise HTTPException(403, "Not allowed")
        return user
    return dependency

@router.post("/transfers")                      # only account holders initiate transfers
def create_transfer(req: TransferIn, user = Depends(require_roles("ACCOUNT_HOLDER")), ...): ...

@router.get("/audit/ledger")                    # read-only for auditors (and admins)
def read_ledger(user = Depends(require_roles("AUDITOR", "ADMIN")), ...): ...
```

6.5 Account ownership

A role is not enough. Two account holders have the **same role**, yet Asha must never touch Bharat's account. So for account-holder operations we also verify **ownership**: `account.owner_id == current_user.id`. Missing this check is the most common real-world API vulnerability, called **IDOR** (Insecure Direct Object Reference) or *Broken Object-Level Authorization*.

```
def get_account_for_user(db, user, account_id, for_update=False):
    acc = db.get_account(account_id, for_update=for_update)
    # 404 (not 403) so strangers cannot learn which account IDs exist
    if acc is None:
        raise HTTPException(404, "Account not found")
    if user.role == "ACCOUNT_HOLDER" and acc.owner_id != user.id:
        audit_denied(user, "OWNERSHIP_VIOLATION", acc.id)
        raise HTTPException(404, "Account not found")
    return acc
```

REAL-WORLD EXAMPLE

IDOR attack, step by step: Asha opens `GET /accounts/acc_A/transactions`. She edits the URL to `acc_B` (Bharat's). If the server only checks "is logged in?", she sees Bharat's history. With the ownership check she receives 404 and an audit line "OWNERSHIP_VIOLATION asha → acc_B".

6.6 Admin and Auditor permissions

Role	Can	Cannot (proposed)	Why
Account Holder	See and use only own accounts; deposit, withdraw, transfer from own accounts; own history and statements	See others' data; act on others' accounts	Principle of least privilege
Admin	Create/freeze/unfreeze/close accounts; view all accounts and users; create audited reversals	Edit balances or ledger rows; delete anything; spend customers' money; read passwords	Admin power is the biggest insider risk — constrain and log it
Auditor	Read ledger, transactions, audit logs; run integrity verification	Change anything at all	Independence: the checker must not be able to alter what is checked

NOTE

Principle of least privilege: give each role the minimum rights needed. **Separation of duties:** the person who can move or fix money should not be the person who audits it.

6.7 Backend security checklist

Area	What to do	Attack it prevents
Transport	HTTPS only (TLS); HSTS in production	Eavesdropping, token/password theft
SQL	Always **parameterised queries** / ORM; never build SQL with f-strings from user input	SQL injection
Input	Validate type, range, length, format on the server (below)	Bad data, overflow, injection
Auth	Hash passwords, short-lived tokens, rate-limit login	Credential stuffing, brute force
AuthZ	Role + ownership check in every handler (use shared dependencies so none is forgotten)	IDOR, privilege escalation
CORS	Allow only your own frontend origin	Malicious sites calling your API from a victim's browser
Errors	Generic messages to clients; details only in server logs; never print SQL or stack traces	Information leakage
Logging	Log actions and IDs, **never** passwords, tokens or full card numbers	Secret leakage through logs
Dependencies	Pin versions; scan for vulnerabilities; update regularly	Known-vulnerability exploits
Headers	Security headers; do not expose server version	Clickjacking, sniffing
Audit	Record logins, denials, money moves, admin actions	Undetected misuse

6.8 Input validation

Never trust anything from the client, including the amount, IDs and even headers. Validate on the server using a schema library (Pydantic in the proposed stack). For money:

```
from decimal import Decimal, InvalidOperation, ROUND_UNNECESSARY
from pydantic import BaseModel, field_validator, constr
from uuid import UUID

MAX_PAISE = 10_00_00_000 * 100          # e.g. ₹10 crore per transaction (a business decision)

class TransferIn(BaseModel):
    from_account_id: UUID
    to_account_id: UUID
```

```
    amount: str                          # string, not float
    description: constr(max_length=140) = ""

    @field_validator("amount")
    @classmethod
    def valid_amount(cls, v):
        try:
            d = Decimal(v)
        except InvalidOperation:
            raise ValueError("amount must be a number")
        if not d.is_finite() or d <= 0:
            raise ValueError("amount must be positive")
        if d.as_tuple().exponent < -2:
            raise ValueError("at most 2 decimal places")
        if d * 100 > MAX_PAISE:
            raise ValueError("amount too large")
        return v

    def paise(self) -> int:
        return int(Decimal(self.amount) * 100)       # exact integer arithmetic from here on
```

Bad input	Why it is dangerous	Expected result
`-500`	Transferring a negative amount would pull money from the receiver	422
`0`	Pointless entry; may break “amount > 0” constraint	422
`10.999`	Rounding creates or loses paise	422
`1e999`, `NaN`, `Infinity`	Special float values break arithmetic	422
`"100; DROP TABLE accounts"`	SQL injection attempt	422 (and parameterised queries make it harmless anyway)
`from == to`	Self-transfer	422
missing `Idempotency-Key`	No retry safety	422

6.9 API security

- **Authenticate every protected route** by default (a global dependency), and make public routes the explicit exception (`/auth/login`, `/health`).
- **Use HTTP methods properly:** GET never changes money; money actions are POST.
- **Rate limit** login and money endpoints to slow brute force and abuse.
- **Limit request size** and response size (pagination for history).
- **Do not expose internal IDs or fields** the client does not need (never return `password_hash`).
- **Version the API** (`/api/v1/…`) so changes do not break clients.
- **Document it** with OpenAPI (FastAPI generates `/docs` automatically); this also helps the next developer.

6.10 Secrets and environment variables

A **secret** is anything that grants access: database passwords, the JWT signing key, API keys. Hard-coding a secret in source code and pushing to GitHub is one of the most common ways systems get breached — bots scan public repositories within minutes.

```
# .env  (local only — listed in .gitignore, NEVER committed)
DATABASE_URL=postgresql://ledger_app:S0me-long-pass@localhost:5432/ledgerline
JWT_SECRET=4f1c...64-random-hex-characters...
ACCESS_TOKEN_MINUTES=15

# .env.example  (committed — shows variable NAMES with fake values so others can set up)
DATABASE_URL=postgresql://user:password@localhost:5432/ledgerline
JWT_SECRET=change-me
ACCESS_TOKEN_MINUTES=15

# .gitignore
.env
```

- Production secrets live in the hosting platform's **environment variable / secret manager** settings, not in files.
- If a secret was ever committed, **rotate it** (create a new one) — deleting the file does not remove it from Git history.
- Use a **least-privileged database user** for the app: no `DROP`, and no `UPDATE`/`DELETE` on ledger tables.

6.11 Preventing unauthorized transactions — practical scenarios

Attack / mistake	Defence	Test (Section 10)
No token or forged token	JWT signature verification → 401	test_transfer_requires_authentication
Expired token reused	`exp` claim checked → 401	test_expired_token_rejected
Asha debits Bharat's account by sending his account ID	Ownership check → 404/403 + audit	test_cannot_transfer_from_others_account
Auditor sends a transfer	Role check → 403	test_auditor_cannot_move_money
Admin tries to edit ledger via API	No such endpoint; DB trigger rejects UPDATE	test_ledger_update_rejected
Client claims `"role":"ADMIN"` in the body	Role comes only from verified token/DB	test_role_escalation_ignored
Frozen account used for money	Business rule → 422	test_frozen_account_blocked
Someone replays an old captured request	Short token expiry + idempotency key replay returns the same result, no new effect	test_replay_has_no_effect

> **TECHNICAL IMPLEMENTATION (how we make it true)**
>
> **Audit logging of failures:** when authorization fails, the money transaction is never started, so you can simply insert an audit row. But for failures that happen *inside* a money transaction (and are rolled back) the audit row must be written in a **separate** short transaction, otherwise it would be rolled back with everything else. Otherwise you lose evidence of attempted abuse.

7. Database Design

> **PROPOSED DESIGN (not a confirmed requirement)**
> The six tables named in your brief (users, accounts, transactions, ledger_entries, idempotency_keys, audit_logs) are treated as the starting point. Their exact columns, names and types below are my PROPOSED design. The problem statement confirms the *needs* (accounts, ledger entries, idempotency, audit logs), not these specific schemas.

7.1 Database vocabulary from zero

Term	Meaning	Example
Table	A grid of data about one kind of thing	`accounts`
Row	One record in a table	Asha's account
Column	One attribute of every row, with a type	`balance_paise BIGINT`
Primary key (PK)	Column(s) that uniquely identify a row; never repeated or null	`accounts.id`
Foreign key (FK)	A column pointing to a row in another table; the database ensures the target exists	`accounts.owner_id → users.id`
Relationship	How tables connect: one-to-many (1—N), etc.	One user has many accounts
Constraint	A rule the database enforces automatically	`CHECK (amount_paise > 0)`
Index	A sorted lookup structure that makes searching fast, like a book's index	index on `ledger_entries(account_id, id)`
Migration	A versioned script that changes the schema; keeps every environment identical	Alembic migration `0001_initial.py`

7.2 How the tables connect

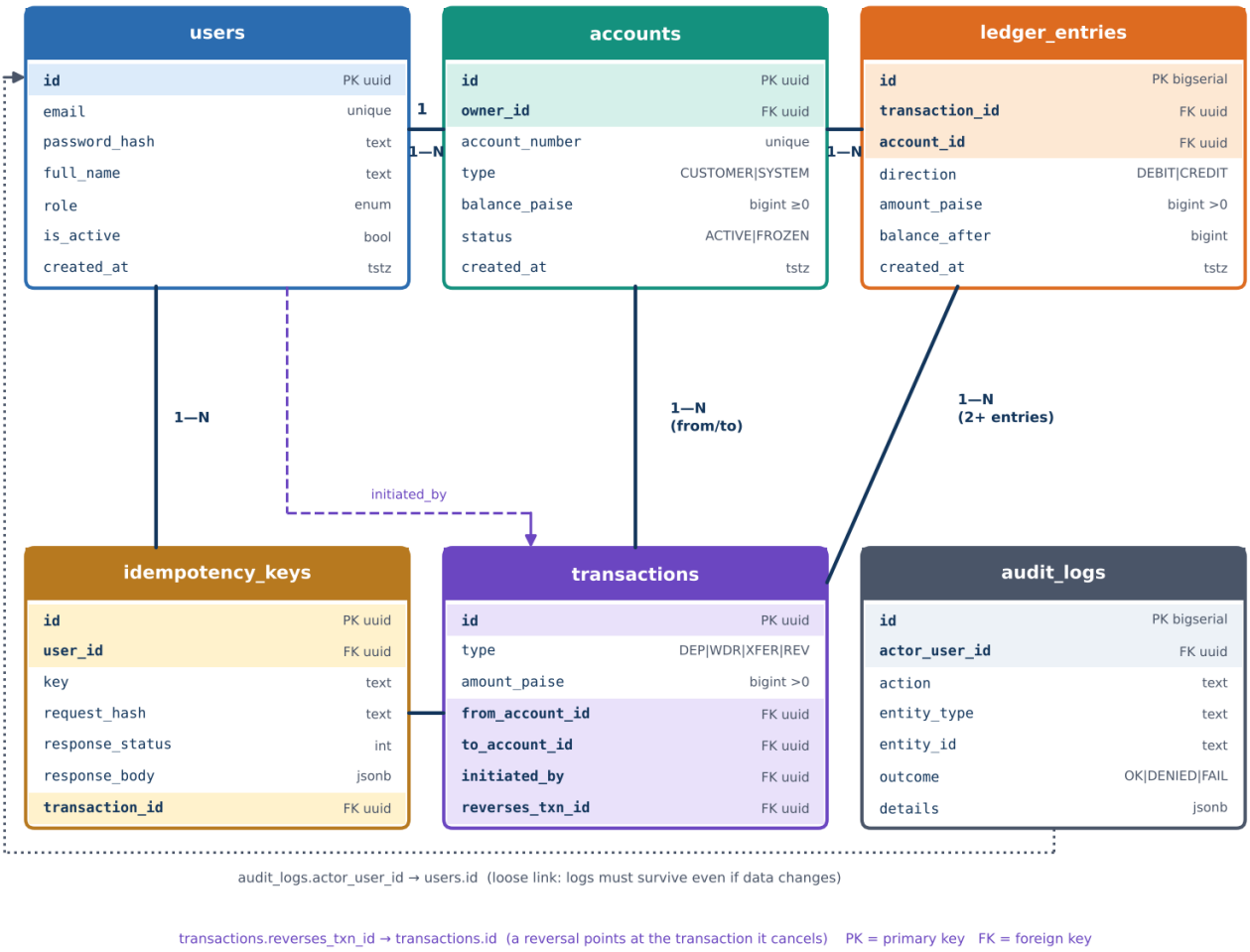


Figure 7.1 *— Entity-relationship (ER) diagram. "1—N" means one row on the left relates to many on the right.*

- **users → accounts (1—N):** a user owns zero or more accounts. System accounts (CLEARING) have no owner.
- **accounts → ledger_entries (1—N):** each entry belongs to exactly one account; an account's history is all its entries.
- **transactions → ledger_entries (1—N):** one financial transaction produces two or more entries that share its ID.
- **accounts → transactions:** a transaction names its source and destination account.
- **users → transactions:** who initiated it. **users → idempotency_keys:** keys are scoped per user.
- **transactions → transactions (self):** a reversal points to the transaction it cancels.
- **audit_logs:** references the acting user and an entity (account, transaction…), kept loosely so logs stay readable even when other data evolves.

7.3 Balance vs. transaction record vs. ledger entry

	accounts.balance_paise	transactions row	ledger_entries row
Purpose	Fast "current balance" (cache)	Header describing one money event	One debit or credit line for one account
Rows per ₹1,000 transfer	0 new (2 updated)	1 new	2 new
Mutable?	Yes (only via the engine)	No	No
Answers	"How much do I have now?"	"What happened and who started it?"	"How did each account's balance change, step by step?"

	accounts.balance_paise	transactions row	ledger_entries row
Proves correctness?	Not alone	Partly	Yes — replay all entries to recompute balances

7.4 Money as integers, time as UTC

INCORRECT IMPLEMENTATION

Floating-point numbers store most decimals approximately. In Python, `0.1 + 0.2` is `0.30000000000000004`. Over millions of transactions such errors become real money.

CORRECT APPROACH

Store money as an **integer count of the smallest unit**: **paise** (₹1 = 100 paise). ₹1,000.00 → `100000`. Convert only at the edges (parsing input, formatting output). Alternative: `NUMERIC(14,2)` (exact decimal) — also acceptable; **never** `FLOAT`/`DOUBLE`. Store timestamps as `TIMESTAMPTZ` (stored in UTC) and show local time (IST) in the UI.

7.5 Complete schema (PROPOSED, PostgreSQL)

```
CREATE TABLE users (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  full_name     TEXT NOT NULL,
  role          TEXT NOT NULL CHECK (role IN ('ACCOUNT_HOLDER','ADMIN','AUDITOR')),
  is_active     BOOLEAN NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE accounts (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id       UUID REFERENCES users(id),               -- NULL only for SYSTEM accounts
  account_number TEXT NOT NULL UNIQUE,
  type           TEXT NOT NULL DEFAULT 'CUSTOMER' CHECK (type IN ('CUSTOMER','SYSTEM')),
  currency       CHAR(3) NOT NULL DEFAULT 'INR',
  balance_paise  BIGINT NOT NULL DEFAULT 0,
  status         TEXT NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','FROZEN','CLOSED')),
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT balance_not_negative CHECK (type = 'SYSTEM' OR balance_paise >= 0),
  CONSTRAINT customer_has_owner   CHECK (type = 'SYSTEM' OR owner_id IS NOT NULL)
);
CREATE INDEX idx_accounts_owner ON accounts (owner_id);

CREATE TABLE transactions (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reference             TEXT NOT NULL UNIQUE,            -- human friendly: TXN-000003
  type                  TEXT NOT NULL CHECK (type IN ('DEPOSIT','WITHDRAWAL','TRANSFER','REVERSAL')),
  amount_paise          BIGINT NOT NULL CHECK (amount_paise > 0),
  from_account_id       UUID NOT NULL REFERENCES accounts(id),
  to_account_id         UUID NOT NULL REFERENCES accounts(id),
  initiated_by          UUID NOT NULL REFERENCES users(id),
  reverses_transaction_id UUID REFERENCES transactions(id),
  description           TEXT,
  created_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (from_account_id <> to_account_id)
);
CREATE UNIQUE INDEX one_reversal_per_txn ON transactions (reverses_transaction_id)
  WHERE reverses_transaction_id IS NOT NULL;
CREATE INDEX idx_txn_from ON transactions (from_account_id, created_at DESC);
CREATE INDEX idx_txn_to   ON transactions (to_account_id,   created_at DESC);

CREATE TABLE ledger_entries (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  transaction_id    UUID   NOT NULL REFERENCES transactions(id),
  account_id        UUID   NOT NULL REFERENCES accounts(id),
  direction         TEXT   NOT NULL CHECK (direction IN ('DEBIT','CREDIT')),
  amount_paise      BIGINT NOT NULL CHECK (amount_paise > 0),
  balance_after_paise BIGINT NOT NULL,                   -- running balance snapshot for statements
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
CREATE INDEX idx_ledger_account ON ledger_entries (account_id, id DESC);   -- statements
CREATE INDEX idx_ledger_txn     ON ledger_entries (transaction_id);

CREATE TABLE idempotency_keys (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID NOT NULL REFERENCES users(id),
  key             TEXT NOT NULL,
  request_hash    TEXT NOT NULL,
  response_status INT,
  response_body   JSONB,
  transaction_id  UUID REFERENCES transactions(id),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_idem UNIQUE (user_id, key)
);

CREATE TABLE audit_logs (
  id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  actor_user_id UUID REFERENCES users(id),                -- NULL for failed logins by unknown users
  action        TEXT NOT NULL,                            -- LOGIN, TRANSFER, REVERSE, ACCOUNT_FROZEN ...
  entity_type   TEXT, entity_id TEXT,
  outcome       TEXT NOT NULL CHECK (outcome IN ('SUCCESS','DENIED','FAILED')),
  details       JSONB,
  ip_address    INET,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_audit_actor  ON audit_logs (actor_user_id, created_at DESC);
CREATE INDEX idx_audit_entity ON audit_logs (entity_type, entity_id);
```

Plus the triggers from Sections 5.2 (balanced entries) and 5.5 (append-only). The CLEARING system account is created once by a migration/seed script.

7.6 Table-by-table explanation with example records

IDs are shortened for readability (`u_asha`, `acc_A`); the real database uses UUIDs. Amounts are shown both as stored (paise) and in ₹.

users

Purpose: everyone who can log in. **Key points:** `email` unique; `password_hash` holds the Argon2/bcrypt hash — never the password; `role` drives RBAC; `is_active` lets you disable a user without deleting history (deleting users would orphan records).

id	email	full_name	role	is_active
u_asha	asha@example.com	Asha Rao	ACCOUNT_HOLDER	true
u_bharat	bharat@example.com	Bharat Kumar	ACCOUNT_HOLDER	true
u_meera	meera@example.com	Meera Iyer	ADMIN	true
u_kiran	kiran@example.com	Kiran Das	AUDITOR	true

accounts

Purpose: one row per account with the cached balance. **Key points:** `CHECK` forbids negative customer balances; `owner_id` FK ties to a user; `status` supports freezing; index on `owner_id` makes “my accounts” fast.

id	owner_id	account_number	type	balance_paise	shown as	status
acc_A	u_asha	LL0001	CUSTOMER	400000	₹4,000.00	ACTIVE
acc_B	u_bharat	LL0002	CUSTOMER	300000	₹3,000.00	ACTIVE
acc_CLR	NULL	SYS-CLEARING	SYSTEM	−700000	−₹7,000.00	ACTIVE

transactions

Purpose: one header row per money event. **Key points:** never updated; `reference` for humans; `reverses_transaction_id` links a reversal to its original; `initiated_by` records who acted.

id	reference	type	amount_paise	from	to	by
T1	TXN-000001	DEPOSIT	500000	acc_CLR	acc_A	u_asha
T2	TXN-000002	DEPOSIT	200000	acc_CLR	acc_B	u_bharat
T3	TXN-000003	TRANSFER	100000	acc_A	acc_B	u_asha

ledger_entries

Purpose: the double-entry lines — the source of truth for money movements. **Key points:** two or more rows per transaction; `balance_after_paise` makes statements fast and audits easy; composite index `(account_id, id DESC)` serves “latest entries of this account”.

id	txn	account	direction	amount_paise	balance_after
1	T1	acc_CLR	DEBIT	500000	−500000
2	T1	acc_A	CREDIT	500000	500000
3	T2	acc_CLR	DEBIT	200000	−700000
4	T2	acc_B	CREDIT	200000	200000
5	T3	acc_A	DEBIT	100000	400000
6	T3	acc_B	CREDIT	100000	300000

idempotency_keys

Purpose: remember each request key and its saved answer. **Key points:** `UNIQUE (user_id, key)` is the guard; `request_hash` detects key misuse; `response_body` lets us replay the exact answer.

user_id	key	request_hash	response_status	transaction_id
u_asha	4e1f6c0a-...	a91f...	201	T3

audit_logs

Purpose: who did what, with what result. **Key points:** append-only; includes DENIED and FAILED outcomes; indexes by actor and entity support investigations.

id	actor	action	entity	outcome	details
1	u_asha	LOGIN	—	SUCCESS	{"ip":"49.x.x.x"}
2	u_asha	TRANSFER	T3	SUCCESS	{"amount":"1000.00"}
3	u_asha	TRANSFER	acc_B	DENIED	{"reason":"not owner"}

7.7 Indexes — why and which

Without an index the database reads every row to find matches (a **full table scan**). With millions of ledger lines, “show Asha’s last 20 transactions” would be painfully slow. An index like `(account_id, id DESC)` lets the database jump straight to her newest entries. Indexes cost a little write speed and storage, so add them for real query patterns: statements, lookups by owner, reversal lookups, idempotency checks (the `UNIQUE` constraint creates an index automatically), audit searches.

8. Technology Stack

CONFIRMED vs PROPOSED

No technology has been finalised in your problem statement. Everything in the right-hand columns below is a proposal made for a beginner-friendly, interview-friendly project. The left column is what the task actually requires. Pick your own stack if you prefer — the concepts in this guide are identical in Node, Java, Go or .NET.

Area	Confirmed requirement	PROPOSED choice	Reasonable alternatives
Frontend	Screens for login, dashboard, accounts, deposits/withdrawals, transfers, history, admin, auditor	React + Vite	Vue, Angular, Next.js, Flutter web
Backend	Server enforces auth, validation, rules	Python + FastAPI + Pydantic	Node (Express/NestJS), Django, Spring Boot
Database	Atomic, concurrent-safe, durable financial records	PostgreSQL	MySQL (InnoDB) — avoid document stores for the ledger
API	Frontend ↔ backend communication	REST over HTTPS with JSON (OpenAPI docs)	GraphQL (not recommended here)
Authentication	Authenticate; “JWT or session” per the brief’s wording; RBAC	JWT access tokens + Argon2 password hashes	Server sessions with cookies
Testing	Automated tests for the verification strategy	pytest + httpx + a real PostgreSQL test DB	unittest, Jest/Supertest, JUnit
Git & GitHub	Incremental commits; visible history; docs	Git + GitHub + GitHub Actions CI	GitLab
Deployment	Deployed and accessible; documented to run	Frontend on Vercel/Netlify; API + managed Postgres on Render/Railway/Fly.io	AWS/GCP/Azure; Docker on a VPS

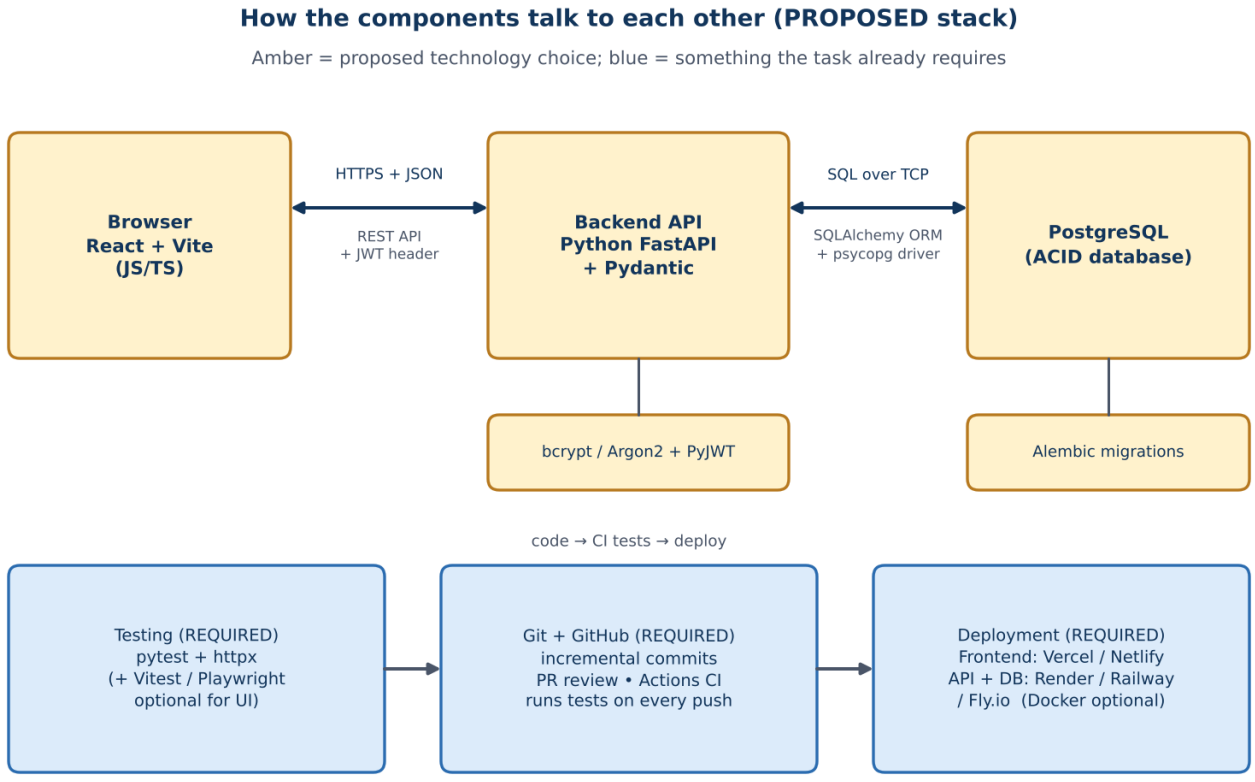


Figure 8.1 *— How the proposed components communicate. Amber = proposed technology; blue = areas the task requires.*

8.1 Why each choice (and how it communicates)

Frontend — React (+ Vite)

A **frontend** is the part that runs in the user's browser. React builds screens from reusable components and is widely used, which helps both learning and interviews. It talks to the backend using `fetch`/Axios, sending JSON over HTTPS and attaching the JWT in the `Authorization` header. It must treat all data as untrusted-in, and never be the only place a rule exists. Keep the token in memory (or an httpOnly cookie) rather than long-lived localStorage if you can; discuss the trade-off in THREATS.md.

Backend — FastAPI (Python)

The **backend** is the program on a server that holds the business rules. FastAPI is quick to start, gives automatic input validation (Pydantic) and auto-generated API documentation. Python is readable, so the *financial logic* — not framework ceremony — stays the focus. Any mainstream backend language can do this; choose one you can explain line by line.

Database — PostgreSQL

A **relational database** stores data in tables with strict rules and supports real ACID transactions. PostgreSQL provides row-level locks (`FOR UPDATE`), strong constraints, deferred constraint triggers, `JSONB`, and several isolation levels — everything Section 5 uses. A database that cannot give you atomic multi-row transactions and constraints is a poor fit for a ledger.

API — REST + JSON

An **API** is the agreed set of URLs and message formats between frontend and backend. Proposed endpoints:

Method & path	Who	Purpose
`POST /api/v1/auth/login`	anyone	Exchange email+password for a token
`GET /api/v1/accounts`	holder (own) / admin (all)	List accounts and balances
`POST /api/v1/accounts`	holder / admin	Open an account
`PATCH /api/v1/accounts/{id}/status`	admin	Freeze / unfreeze / close
`POST /api/v1/accounts/{id}/deposit`	holder (own)	Deposit (needs `Idempotency-Key`)
`POST /api/v1/accounts/{id}/withdraw`	holder (own)	Withdraw (needs `Idempotency-Key`)
`POST /api/v1/transfers`	holder (own source)	Transfer (needs `Idempotency-Key`)
`GET /api/v1/accounts/{id}/transactions`	holder (own) / admin / auditor	History with pagination
`GET /api/v1/accounts/{id}/statement?from=&to=`	holder (own) / auditor	Statement for a period
`POST /api/v1/transactions/{id}/reverse`	admin	Create compensating reversal
`GET /api/v1/audit/logs` • `GET /api/v1/audit/verify`	auditor	Read audit logs; run integrity checks

Authentication — JWT + Argon2

Covered in Section 6. Proposed because JWT keeps the API stateless and is a very common interview topic. Sessions are equally valid.

Testing — pytest + a real database

pytest is simple and powerful (fixtures, parametrised tests). Money tests must run against a **real PostgreSQL** — for example a Docker container started for the test run — because mocks cannot prove that locks, constraints and rollbacks work. `httpx` calls the API as a client would.

Git & GitHub

Git records history; GitHub hosts it and runs **CI (Continuous Integration)** — automatic tests on every push. The brief stresses a visible, incremental commit history (Section 11).

Deployment

Deploy = make the app run on the internet. Typical flow: push to GitHub → CI runs tests → the host builds and starts the app → run database **migrations** → set environment variables (database URL, JWT secret) in the host's dashboard → serve over HTTPS. Keep a `README` with exact setup and run steps so another developer can reproduce it (confirmed success criterion).

```
# docker-compose.yml  (PROPOSED, for local development & tests)
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ledger_app
      POSTGRES_PASSWORD: dev_password_only
      POSTGRES_DB: ledgerline
    ports: ["5432:5432"]
  # api: built from ./backend ; web: built from ./frontend  (added in later phases)
```

8.2 Decisions to confirm before coding

- Which backend language/framework and database will you finally use?
- Does the Admin role ever move money, or only manage accounts and create reversals?
- How are deposits/withdrawals "funded" — a simulated cash counter via the CLEARING account (proposed)?
- JWT or sessions? Token lifetime? Refresh tokens needed?
- Currency: INR only (proposed) — any multi-currency requirement?
- Limits and fees: maximum amount per transaction? daily limits? any transfer fee?
- Can accounts be closed with a balance? Can a reversal overdraw the receiver?
- Where will it be deployed and who needs access?

9. Development Phases

CONFIRMED REQUIREMENT

The eight phases and their names come directly from your problem statement. The file and module names below are PROPOSED and assume the suggested Python/React/PostgreSQL stack.

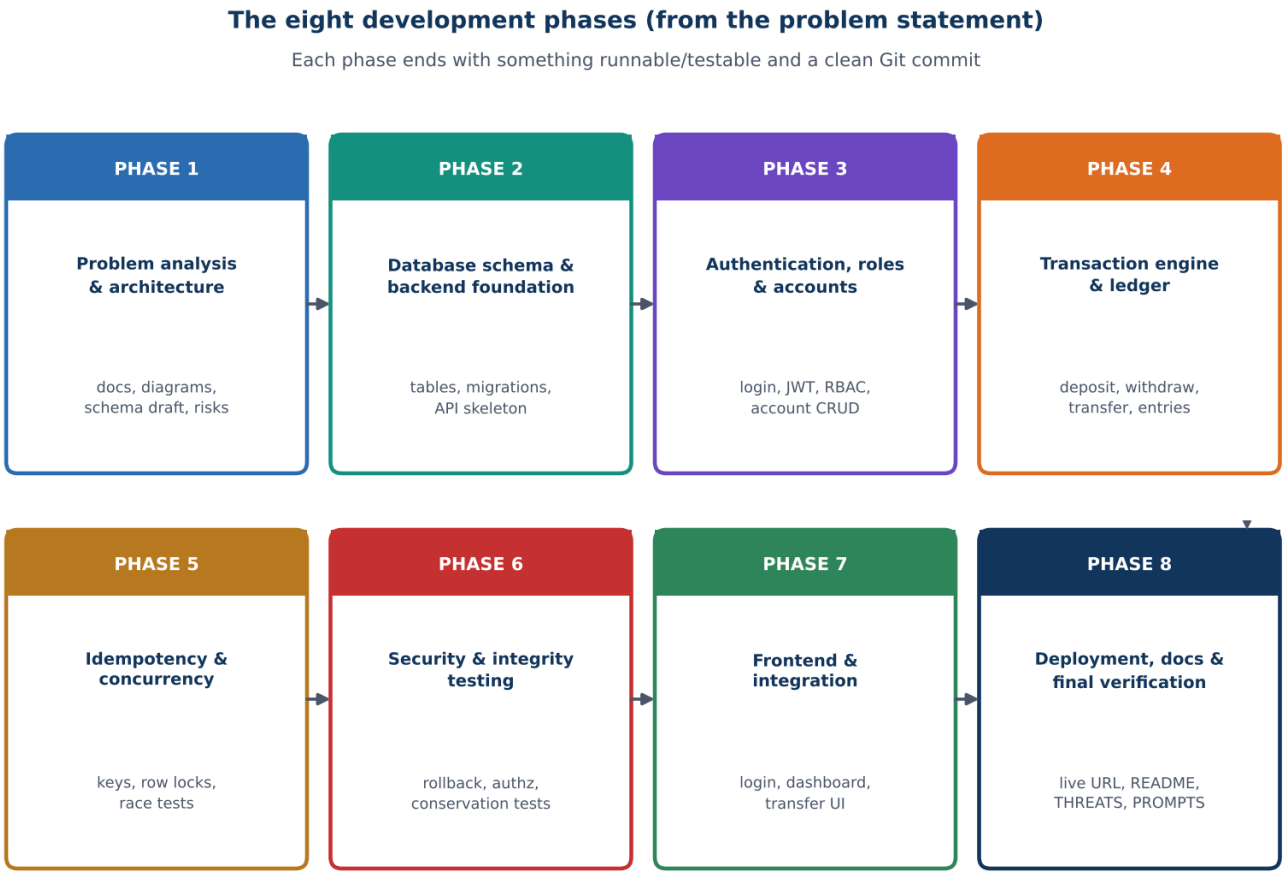


Figure 9.1 *— The eight phases at a glance.*

Proposed repository layout

```
ledgerline/
├── README.md   PROMPTS.md   THREATS.md   .env.example   .gitignore   docker-compose.yml
├── docs/                    problem understanding, architecture, ER diagram, API list
├── backend/
│   ├── app/
│   │   ├── main.py  config.py  db.py                app start, settings from env, DB session
│   │   ├── models/      user.py account.py transaction.py ledger_entry.py idempotency.py audit.py
│   │   ├── schemas/     auth.py account.py transfer.py          (request/response validation)
│   │   ├── api/         auth.py accounts.py transfers.py admin.py audit.py deps.py
│   │   ├── services/    transaction_engine.py ledger.py idempotency.py audit_service.py account_service.py
│   │   └── security/    passwords.py tokens.py permissions.py
│   ├── migrations/      (Alembic versions)
│   └── tests/           conftest.py test_auth.py test_transfer.py test_idempotency.py test_concurrency.py ...
└── frontend/src/        pages/ components/ api/ auth/
```

Phase 1 — Problem Analysis and Architecture

Aspect	Details
Objective	Understand exactly what must be built and why, before coding. Produce the shared mental model and the design.
Concepts to understand	Sections 1–5 of this guide: ledger, double-entry, atomicity, idempotency, concurrency, immutability; the three roles; the request flow.
Files / modules	`README.md` (stub), `docs/PROBLEM_UNDERSTANDING.md` (your problem-understanding document), `docs/ARCHITECTURE.md`, ER diagram, API endpoint list, first drafts of `THREATS.md` and `PROMPTS.md`.
What you implement	No application code. Repository set up with `.gitignore` (including `.env`). Documents describing requirements, risks, architecture, schema draft, proposed stack and the phase plan.
What you test	Explain-it-aloud test: can you narrate the ₹1,000 transfer including failure cases without notes? List 10 risks and one mitigation each.
Expected output	Committed docs; initial commit history shows understanding came before code.
Completion criteria	Open decisions (Section 8.2) answered or recorded as assumptions; diagrams drawn; every confirmed requirement mapped to a phase.
Common mistakes	Starting with the UI; copying requirements without understanding; hiding assumptions instead of labelling them; skipping the threat list.

Phase 2 — Database Schema and Backend Foundation

Aspect	Details
Objective	Create a trustworthy data layer and a running, configurable backend skeleton.
Concepts	Tables, keys, constraints, indexes, migrations (7.1); integer money (7.4); environment-based configuration (6.10); ACID basics (5.1, 5.8).
Files / modules	`docker-compose.yml`, `config.py`, `db.py`, `models/*`, Alembic migration `0001_initial`, `main.py` with `/health`, `tests/conftest.py`, seed script for the CLEARING account.
What you implement	All six tables with constraints and indexes; triggers for balanced entries and append-only tables; database session management; global error handler returning a consistent error JSON; health check.
What you test	Migrations apply cleanly from empty; negative customer balance is rejected by the database; an unbalanced set of entries fails at commit; `UPDATE`/`DELETE` on `ledger_entries` fails; duplicate email/account number rejected.
Expected output	Running `docker compose up` + migrations gives an empty but fully constrained database; `GET /health` returns 200; constraint tests pass.
Completion criteria	Schema documented; constraints proven by tests; configuration only via environment variables; no secrets in Git.
Common mistakes	Using FLOAT for money; forgetting indexes/foreign keys; putting all rules only in Python; hard-coding the database password; creating tables by hand instead of migrations.

Phase 3 — Authentication, Roles and Account Management

Aspect	Details
Objective	Know who is calling and what they may do; let users manage accounts safely.
Concepts	Authentication vs authorization (6.1); hashing (6.2); JWT (6.3); RBAC (6.4); ownership/IDOR (6.5); validation (6.8); audit of security events.
Files / modules	`security/passwords.py`, `security/tokens.py`, `security/permissions.py`, `api/deps.py`, `api/auth.py`, `api/accounts.py`, `api/admin.py`, `services/account_service.py`, `services/audit_service.py`.
What you implement	User creation (seed or admin-created), login, token issue/verify, `current_user` and `require_roles` dependencies, account create/list/get, ownership enforcement, admin freeze/unfreeze, audit entries for login success/failure and denied access.
What you test	Wrong password → 401; no/expired/forged token → 401; Account Holder reading another's account → 404/403; Auditor attempting a write → 403; Admin freeze works and is audited; password hashes never returned.

Aspect	Details
Expected output	A user can log in and see only their own accounts; admin and auditor see appropriate data.
Completion criteria	Every protected route has authentication; every account route has an ownership/role check; denied attempts appear in `audit_logs`.
Common mistakes	Trusting `user_id` from the request body; checking roles only in the frontend; different error messages for “no such user” vs “wrong password”; JWT secret in code; forgetting to check `is_active`.

Phase 4 — Financial Transaction Engine and Ledger

Aspect	Details
Objective	Implement correct money movement with atomic database transactions and a double-entry ledger.
Concepts	Debit/credit and double-entry (3.4–3.6, 5.2); atomicity (5.1); balance rules; reversals (5.5); conservation (5.6).
Files / modules	`services/transaction_engine.py`, `services/ledger.py`, `api/transfers.py`, `schemas/transfer.py`, statement/history queries.
What you implement	Deposit, withdraw and transfer functions that run inside one database transaction; header + balanced ledger entries + balance update + audit; `balance_after`; history and statement endpoints with pagination; admin-only reversal.
What you test	Successful transfer (balances and entries as in Section 4); insufficient balance rejected with no side effects; rollback on injected failure; entries always balanced; statement closing balance equals account balance.
Expected output	A working API where every transaction leaves exactly the expected rows.
Completion criteria	Conservation invariants hold after every test; no code path changes a balance without writing entries; no UPDATE/DELETE on ledger tables anywhere in the code.
Common mistakes	Committing in the middle; updating balances without ledger entries; using floats; forgetting that deposits/withdrawals also need entries; allowing amount ≤ 0; letting the client send the “new balance”.

Phase 5 — Idempotency and Concurrency Protection

Aspect	Details
Objective	Make duplicates and simultaneous requests harmless.
Concepts	Idempotency keys (5.3); race conditions, row locks, lock ordering, deadlocks (5.4); isolation levels (5.7).
Files / modules	`services/idempotency.py`, locking helper in `transaction_engine.py`, `tests/test_idempotency.py`, `tests/test_concurrency.py`.
What you implement	`Idempotency-Key` header required on money endpoints; key table with unique constraint and request hash; replay of stored response; same-key-different-body rejection; `SELECT … FOR UPDATE` in sorted ID order; (optionally) the conditional-update safeguard.
What you test	Same request twice → one transfer, identical response; two parallel identical requests → one transfer; two ₹800 withdrawals on ₹1,000 in parallel → exactly one success; opposite-direction transfers in parallel → no deadlock.
Expected output	Repeated runs of the concurrency tests (e.g., 50 iterations) always give the same correct outcome.
Completion criteria	Tests are deterministic and pass repeatedly; final balances and ledger totals equal expectations; no unexplained 500 errors under load.
Common mistakes	Check-then-act without locks; locking accounts in request order (deadlock); idempotency check outside the money transaction; keys that depend on amount/time; running concurrency tests against a mocked database.

Phase 6 — Security and Financial Integrity Testing

Aspect	Details
Objective	Attack your own system and prove the guarantees with automated evidence.
Concepts	Threat modelling (11.3); defence in depth (5.8); integrity queries (5.6); least privilege (6.6).
Files / modules	`tests/test_security.py`, `tests/test_integrity.py`, `services/integrity.py` + `/audit/verify`, finished `THREATS.md`.
What you implement	Gaps found by testing; an integrity verification service (balanced transactions, balance = ledger replay, no negative customers, grand total 0); rate limiting; consistent error handling; least-privilege DB role.
What you test	The full matrix in Section 10: success, insufficient funds, duplicates, concurrency, unauthorized access, rollback, ledger imbalance, invalid input, ownership violations, DB failure.
Expected output	Green test suite plus a short test report/coverage summary; updated threat register.
Completion criteria	Every row of THREATS.md has a mitigation and a test; the verification endpoint reports PASS on a seeded random workload.
Common mistakes	Testing only the happy path; asserting only HTTP codes (not balances/ledger); fixing a test instead of the bug; leaving verbose error messages that leak internals.

Phase 7 — Frontend Development and Integration

Aspect	Details
Objective	Provide screens for every role that call the proven backend correctly.
Concepts	HTTP/JSON client; token handling; generating and reusing an idempotency key per intention; displaying server-provided balances; loading/error states.
Files / modules	`frontend/src/pages/{Login,Dashboard,Accounts,DepositWithdraw,Transfer,History,Admin,Auditor}.jsx`, `api/client.js`, `auth/AuthContext.jsx`, route guards, shared components.
What you implement	The screens in Section 12; a route guard per role (cosmetic — the server is the real guard); forms with inline validation; clear display of API errors; disabled-while-submitting buttons; the Idempotency-Key created when the form opens and reused on retry.
What you test	Manual walkthrough of each role; optional component/E2E tests: login → transfer → balance updates; error banner for insufficient funds; double-click sends one effective request.
Expected output	A usable web app against the local backend.
Completion criteria	All required functions reachable from the UI; unauthorized pages redirect; no business logic only in the UI.
Common mistakes	Computing the new balance in the browser; generating a **new** idempotency key on every retry; hiding admin buttons and assuming that is security; storing tokens carelessly; showing raw server errors.

Phase 8 — Deployment, Documentation and Final Verification

Aspect	Details
Objective	Make the project live, reproducible and demonstrably correct.
Concepts	Environment configuration; migrations in production; CI; HTTPS; documentation for another developer.
Files / modules	`.github/workflows/ci.yml`, deployment config, final `README.md`, `THREATS.md`, `PROMPTS.md`, `docs/`.
What you implement	CI running the test suite on every push; deployment of API, database and frontend; production environment variables; seed/admin bootstrap; README with setup, run, test and architecture sections; final integrity check run on the deployed system.
What you test	Smoke test on the live URL (login, deposit, transfer, history, auditor verify); clone-from-scratch test: follow your own README on a clean machine.
Expected output	Public URL, green CI badge, complete docs, tagged release.
Completion criteria	Matches the problem statement's "Definition of Success" (all eight points).
Common mistakes	Committing secrets; forgetting to run migrations after deploy; using development settings in production; README that omits a step; last-minute untested changes.

10. Testing and Failure Scenarios

Testing a money system is different from testing a normal app. You are not just checking that pages work — you are **proving invariants**: facts that must always be true no matter what happens. After every test that moves money, assert the **state** (balances, entries, audit), not only the HTTP status code.

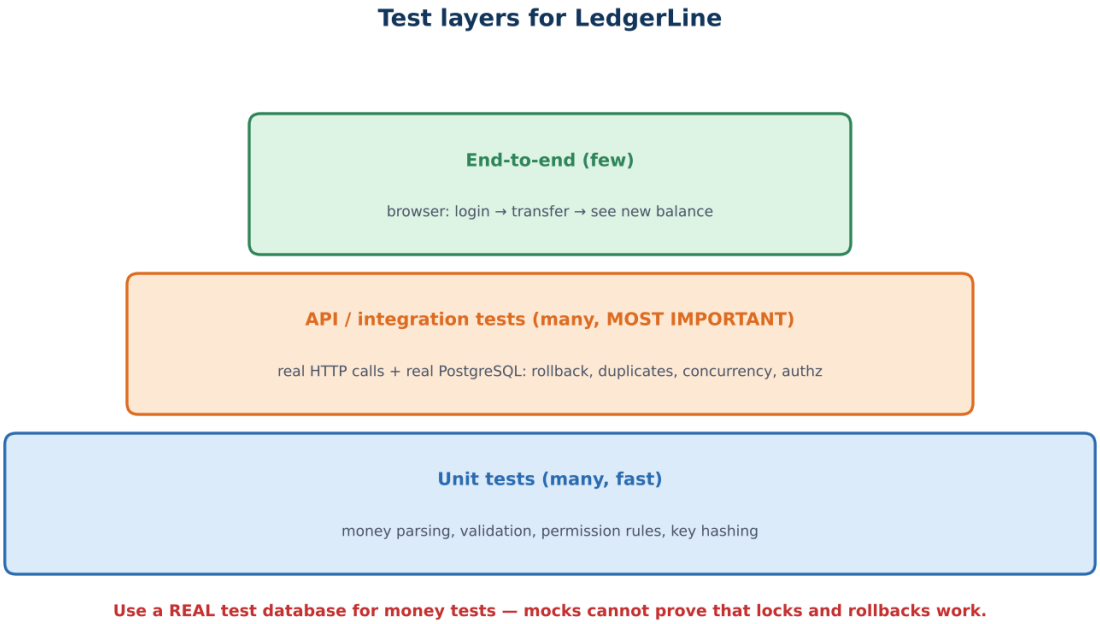


Figure 10.1 *— Test layers; the integration layer with a real database is where the money guarantees are proven.*

Invariants to assert after tests

- Σ debits = Σ credits for every transaction.
- Each account's stored balance = replay of its ledger entries.
- No customer balance < 0.
- Sum of all balances (including CLEARING) = 0; customers' total changes only by deposits/withdrawals.
- Number of transactions/entries equals what the test intended (no duplicates, no leftovers after rollback).

```
# tests/conftest.py (sketch)
@pytest.fixture
def world(db):
    """Fresh database: users asha, bharat, meera(admin), kiran(auditor); A=₹5,000, B=₹2,000."""
    ...

def assert_ledger_invariants(db):
    assert db.scalar("SELECT COUNT(*) FROM (<unbalanced-transactions query from 5.6>) q") == 0
    assert db.scalar("SELECT COUNT(*) FROM (<balance-vs-replay query from 5.6>) q") == 0
    assert db.scalar("SELECT COALESCE(SUM(balance_paise),0) FROM accounts") == 0
    assert db.scalar("SELECT COUNT(*) FROM accounts WHERE type='CUSTOMER' AND balance_paise < 0") == 0
```

10.1 Scenario table

#	Scenario	Setup (A=₹5,000, B=₹2,000)	Action	Expected result
1	Successful transfer	Asha logged in	Transfer ₹1,000 A→B	201; A=₹4,000, B=₹3,000; 2 new entries (DEBIT A, CREDIT B); 1 audit row; invariants hold
2	Insufficient balance	A = ₹5,000	Transfer ₹6,000 A→B	422 INSUFFICIENT_FUNDS; balances unchanged; **no** entries; audit FAILED/DENIED

#	Scenario	Setup (A=₹5,000, B=₹2,000)	Action	Expected result
3	Duplicate request	Key K unused	Send the same transfer twice with K	Both responses identical (201); A=₹4,000 (**one** transfer); 1 transaction row
3b	Key reuse, different body	Key K used for ₹1,000	Send ₹2,500 with K	422; no new transfer
4	Concurrent withdrawals	A = ₹1,000	Two parallel withdrawals of ₹800 (different keys)	Exactly one 201 and one 422; final A=₹200
4b	Opposite transfers (deadlock check)	A=B=₹5,000	Parallel A→B ₹100 and B→A ₹100, repeated 50×	All succeed or fail cleanly; no deadlock errors; total unchanged
5	Unauthorized access	No token / Auditor token	POST /transfers	401 / 403; nothing changes
6	Database failure mid-transaction	Inject exception when inserting ledger entries	Transfer ₹1,000	500; A and B unchanged; no transaction row; no partial entries
7	Transaction rollback	Force error after balance update	Transfer	Everything rolled back (same as 6 but after the UPDATEs)
8	Ledger imbalance	Attempt INSERT of DEBIT 100 + CREDIT 90	Commit	Database raises “unbalanced”; nothing stored
9	Invalid input	—	amount = −5, 0, 10.999, "abc", 1e999; same account; bad UUID	422 each; nothing stored
10	Ownership violation	Asha logged in	Transfer from B’s account; GET B’s history	404/403; audit row OWNERSHIP_VIOLATION
11	Immutability	—	Try UPDATE/DELETE a ledger row via DB session	Exception from trigger; row unchanged
12	Reversal	T3 exists	Admin reverses T3	New REVERSAL txn with opposite entries; balances back to ₹5,000/₹2,000; second reversal rejected

10.2 Example test code (sketches)

Successful transfer

```
def test_successful_transfer(client, world, db):
    r = client.post("/api/v1/transfers", headers=auth(world.asha, key="k1"),
                    json={"from_account_id": world.A, "to_account_id": world.B, "amount": "1000.00"})
    assert r.status_code == 201
    assert balance(db, world.A) == 400_000 and balance(db, world.B) == 300_000      # paise
    entries = ledger_for(db, r.json()["transaction_id"])
    assert sorted((e.account_id, e.direction, e.amount_paise) for e in entries) == sorted([
        (world.A, "DEBIT", 100_000), (world.B, "CREDIT", 100_000)])
    assert_ledger_invariants(db)
```

Insufficient balance

```
def test_insufficient_balance(client, world, db):
    r = client.post("/api/v1/transfers", headers=auth(world.asha, key="k2"),
                    json={"from_account_id": world.A, "to_account_id": world.B, "amount": "6000.00"})
    assert r.status_code == 422 and r.json()["error"]["code"] == "INSUFFICIENT_FUNDS"
    assert balance(db, world.A) == 500_000 and balance(db, world.B) == 200_000
    assert count(db, "ledger_entries", txn_type="TRANSFER") == 0
```

Duplicate request

```
def test_duplicate_request_executes_once(client, world, db):
    h = auth(world.asha, key="same-key")
    body = {"from_account_id": world.A, "to_account_id": world.B, "amount": "1000.00"}
    r1 = client.post("/api/v1/transfers", headers=h, json=body)
    r2 = client.post("/api/v1/transfers", headers=h, json=body)          # retry with same key
    assert r1.status_code == 201 and r2.status_code == 201
```

```
    assert r1.json()["transaction_id"] == r2.json()["transaction_id"]    # identical replayed response
    assert balance(db, world.A) == 400_000                               # moved ONCE
    assert count(db, "transactions", type="TRANSFER") == 1
```

Concurrent withdrawals

```
from concurrent.futures import ThreadPoolExecutor

def test_concurrent_withdrawals_cannot_overdraw(live_server_url, world_with_1000, db):
    # NOTE: use real parallel HTTP calls (or separate DB sessions), not one shared session
    def withdraw(i):
        return httpx.post(f"{live_server_url}/api/v1/accounts/{world_with_1000.A}/withdraw",
                          headers=auth(world_with_1000.asha, key=f"w-{i}"), json={"amount": "800.00"})
    with ThreadPoolExecutor(max_workers=2) as pool:
        results = list(pool.map(withdraw, [1, 2]))
    assert sorted(r.status_code for r in results) == [201, 422]      # exactly one success
    assert balance(db, world_with_1000.A) == 20_000                  # ₹200
    assert_ledger_invariants(db)
```

Tip: to make races likely, use a **barrier** so both threads fire at the same instant, and repeat the test many times in a loop; also temporarily remove the `FOR UPDATE` and confirm the test then **fails** — a test that cannot fail proves nothing.

Rollback / database failure (fault injection)

```
def test_failure_after_balance_update_rolls_back(client, world, db, monkeypatch):
    def boom(*a, **k): raise RuntimeError("simulated crash while writing ledger")
    monkeypatch.setattr("app.services.transaction_engine.insert_ledger_entries", boom)
    r = client.post("/api/v1/transfers", headers=auth(world.asha, key="k9"), json=VALID_BODY(world))
    assert r.status_code == 500
    assert balance(db, world.A) == 500_000 and balance(db, world.B) == 200_000   # untouched
    assert count(db, "transactions") == 2                                         # only the two seed deposits
```

Ledger imbalance and immutability (database-level tests)

```
def test_db_rejects_unbalanced_entries(db, world):
    with pytest.raises(Exception, match="unbalanced"):
        with db.begin():
            t = insert_transaction(db, type="TRANSFER", amount=10_000, ...)
            insert_entry(db, t, world.A, "DEBIT", 10_000)
            insert_entry(db, t, world.B, "CREDIT", 9_000)          # wrong on purpose -> fails at COMMIT

def test_db_rejects_ledger_update(db, world):
    with pytest.raises(Exception, match="append-only"):
        db.execute(text("UPDATE ledger_entries SET amount_paise = 1 WHERE id = 1")); db.commit()
```

Unauthorized access, ownership and invalid input

```
def test_requires_authentication(client, world):
    assert client.post("/api/v1/transfers", json=VALID_BODY(world)).status_code == 401

def test_auditor_cannot_move_money(client, world):
    assert client.post("/api/v1/transfers", headers=auth(world.kiran, key="x"), json=VALID_BODY(world)).status_code == 403

def test_cannot_transfer_from_someone_elses_account(client, world, db):
    body = {"from_account_id": world.B, "to_account_id": world.A, "amount": "100.00"}   # Asha uses Bharat's account
    r = client.post("/api/v1/transfers", headers=auth(world.asha, key="y"), json=body)
    assert r.status_code in (403, 404) and balance(db, world.B) == 200_000
    assert audit_exists(db, action="OWNERSHIP_VIOLATION", actor=world.asha)

@pytest.mark.parametrize("bad", ["-5", "0", "10.999", "abc", "1e999", "NaN", ""])
def test_invalid_amounts_rejected(client, world, bad):
    r = client.post("/api/v1/transfers", headers=auth(world.asha, key=f"i-{bad}"), json={**VALID_BODY(world), "amount": bad})
    assert r.status_code == 422
```

10.3 How to know your tests are good

- Each test asserts **state** (balances, entries, audit rows), not only status codes.
- Break the code on purpose (remove the lock, drop the transaction, disable the trigger) and confirm the relevant test fails (**mutation thinking**).
- Run concurrency tests repeatedly (e.g., 20–100 loops) to expose flaky logic.
- Keep tests independent: clean the database between tests (truncate tables or recreate a schema).
- Add a regression test for every bug you fix and mention it in the commit message.

11. Engineering Workflow

CONFIRMED REQUIREMENT

The problem statement confirms: incremental development; each feature implemented, tested, reviewed and committed independently; AI tools may assist but output must be reviewed and verified; AI mistakes, rejected approaches and corrections documented in **PROMPTS.md**; threats and mitigations documented in **THREATS.md**; a README so another developer can run the project.

11.1 Git commits and incremental development

Git is a version-control tool: it records snapshots ("commits") of your project so you can see what changed, when and why, and go back if needed. **GitHub** hosts the repository online. Reviewers read your commit history like a diary of your thinking: a project built in 2–3 giant commits looks rushed or copied; a project built in many small, meaningful commits shows understanding and discipline.



Figure 11.1 *— An example commit history for LedgerLine (illustrative). Note how documentation came first, tests live beside features, and bug fixes are visible.*

Rules for good commits

- **One logical change per commit.** "Add idempotency key table and replay logic", not "stuff".
- **Present-tense, specific messages** with a type prefix: `feat:`, `fix:`, `test:`, `docs:`, `refactor:`, `chore:`.
- **The project must build and tests must pass at every commit** (run tests *before* committing).
- **Never commit secrets, `.env`, build output or large generated files.**
- **Do not rewrite shared history to hide mistakes.** A visible `fix:` commit is a sign of honesty; use `git revert` rather than deleting history.
- **Commit the test with the feature** (or before it). Example sequence for one feature: `test: add failing test for duplicate transfer` → `feat: implement idempotency key replay` → `docs: describe idempotency in README`.

```
git switch -c feat/idempotency          # work on a branch per feature
# ... write test, write code ...
pytest -q                               # tests MUST pass before you commit
```

```
git add backend/app/services/idempotency.py backend/tests/test_idempotency.py
git commit -m "feat(idem): replay stored response for repeated Idempotency-Key"
git push -u origin feat/idempotency     # open a Pull Request on GitHub, review, then merge
```

11.2 Code review

A **pull request (PR)** proposes merging a branch. **Code review** means reading the change critically before merging — even if you review your own PR, walk through it as a skeptic. A money-specific checklist:

Question to ask about every PR	Why
Does every balance change create matching ledger entries in the same database transaction?	Atomicity + auditability
Is any money value a float?	Rounding errors
Are accounts locked in a consistent order before the balance check?	Races and deadlocks
Is there an ownership/role check on this route?	IDOR / privilege escalation
Can the client influence anything it shouldn't (user_id, role, balance)?	Trust boundary
Is there an UPDATE/DELETE on ledger tables?	Immutability
Are errors generic to clients and detailed in logs (without secrets)?	Leakage
Is there a test that would fail if this line were removed?	Test quality

11.3 AI coding assistance, PROMPTS.md and THREATS.md

AI tools can speed you up, but they confidently produce plausible, subtly wrong code — especially for concurrency, security and money. The rule from your problem statement: **AI may assist; you must review and verify**. The skill reviewers look for is *engineering judgement*: noticing what the AI got wrong.

PROMPTS.md — an honest log of AI collaboration

Record meaningful prompts, what the AI produced, whether you accepted it, and **especially mistakes and corrections**. Example entry:

```
## 2026-11-12 — Transfer function (Phase 4)
**Prompt:** "Write a transfer function that moves money between two accounts in PostgreSQL."
**AI output:** Read both balances, checked `balance >= amount`, then two UPDATEs; no locking.
**Problem found:** Check-then-act race — two parallel withdrawals could overdraw (see test_concurrent_withdrawals).
**Correction:** Added SELECT ... FOR UPDATE on both rows in sorted-ID order; added CHECK (balance_paise >= 0).
**Rejected approach:** Python threading.Lock — only works inside one process, useless with multiple servers.
**Verification:** Concurrency test fails without the lock and passes with it (50 repeats).
**Commit:** fix(txn): lock accounts in id order before balance check
```

THREATS.md — a living threat register

A **threat model** lists what could go wrong, how, and what you did about it. Keep a table; update it as you build and link each row to a test.

ID	Threat	Example attack / failure	Impact	Mitigation	Test
T1	Duplicate execution	Double-click or network retry	Money moved twice	Idempotency-Key + unique index	test_duplicate_request
T2	Race condition	Two simultaneous withdrawals	Overdraft	Row locks + CHECK constraint	test_concurrent_withdrawals
T3	Partial failure	Crash between debit and credit	Money lost	Atomic DB transaction	test_rollback
T4	Broken authorization (IDOR)	Change account ID in URL	Data theft / theft of funds	Ownership check, 404, audit	test_ownership
T5	Privilege escalation	Send role=ADMIN in body	Admin powers	Role from verified token only	test_role_escalation

ID	Threat	Example attack / failure	Impact	Mitigation	Test
T6	History tampering	UPDATE ledger rows	Undetectable fraud	Append-only + triggers + revoked privileges	test_ledger_immutable
T7	Credential theft	DB leak; weak hashing	Account takeover	Argon2/bcrypt + salt; HTTPS	test_password_hashing
T8	Secret leakage	.env pushed to GitHub	Full compromise	.gitignore, env vars, rotate	secret scan in CI
T9	Injection	SQL in a text field	Data loss/theft	Parameterised queries, validation	test_sql_injection_input
T10	Brute force	Guess passwords	Account takeover	Rate limit, lockout, audit	test_login_rate_limit
T11	Insider misuse	Admin abuses power	Fraud	No edit endpoints, reversal needs reason, audit	test_admin_actions_audited

11.4 README and documentation

Your success criteria say another developer must be able to **understand and run** the project from its documentation. A good README contains:

1. **What it is** (2–3 sentences) and a screenshot or diagram.
2. **Features and roles.**
3. **Architecture** (the diagram from Section 2) and the key design decisions (and why).
4. **How to run locally:** prerequisites, `cp .env.example .env`, start database, run migrations, start API and frontend — exact commands.
5. **How to run tests**, including the concurrency tests.
6. **API overview** or link to the generated OpenAPI docs.
7. **Financial guarantees** explained briefly (atomicity, ledger, idempotency, concurrency, immutability) and where each lives in code.
8. **Security notes** and pointer to THREATS.md; **AI usage** and pointer to PROMPTS.md.
9. **Deployed URL** and demo credentials (never real ones).

11.5 Test before you commit — a daily routine

1. Pull latest; create a branch for one small feature.
2. Write or update the test first (watch it fail).
3. Write the smallest code to pass it.
4. Run the whole test suite and the linter.
5. Read your own diff line by line (“would I approve this?”).
6. Commit with a clear message; push; open a PR; merge when CI is green.
7. Add notes to PROMPTS.md / THREATS.md if you learned something.

12. Complete Project Demonstration

> **PROPOSED DESIGN (not a confirmed requirement)**
> These are illustrative mockups of the expected final application, drawn by me to show the intended user experience. The final look, wording and layout are yours to design. Amounts follow the running example (A = ₹4,000 after the transfer).

Demo script (about 5 minutes) for an interview or review: log in as Asha → show dashboard → transfer ₹1,000 to Bharat → show history and balance_after → click Transfer twice quickly (one effect) → try to transfer more than the balance (clean error) → log in as Auditor → run integrity check → log in as Admin → freeze an account and create a reversal → show the audit log and the test suite output.

12.1 Login page

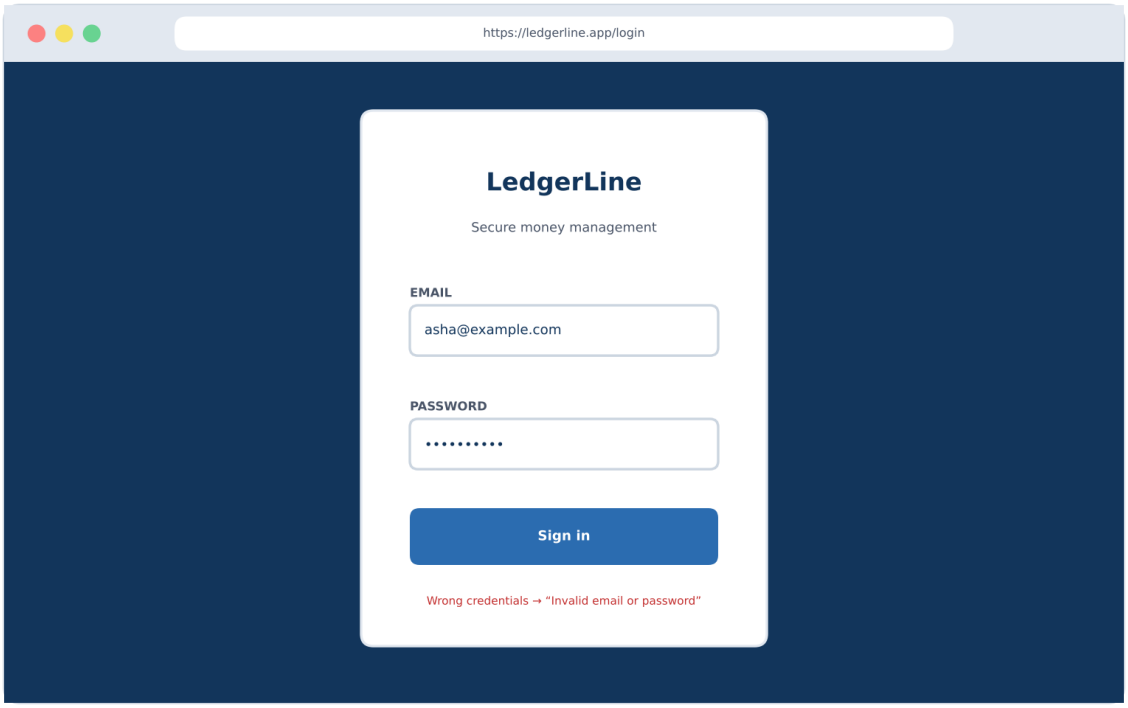


Figure 12.1 *— Login: email + password. Generic error on failure; no hints about which field was wrong.*

12.2 Dashboard

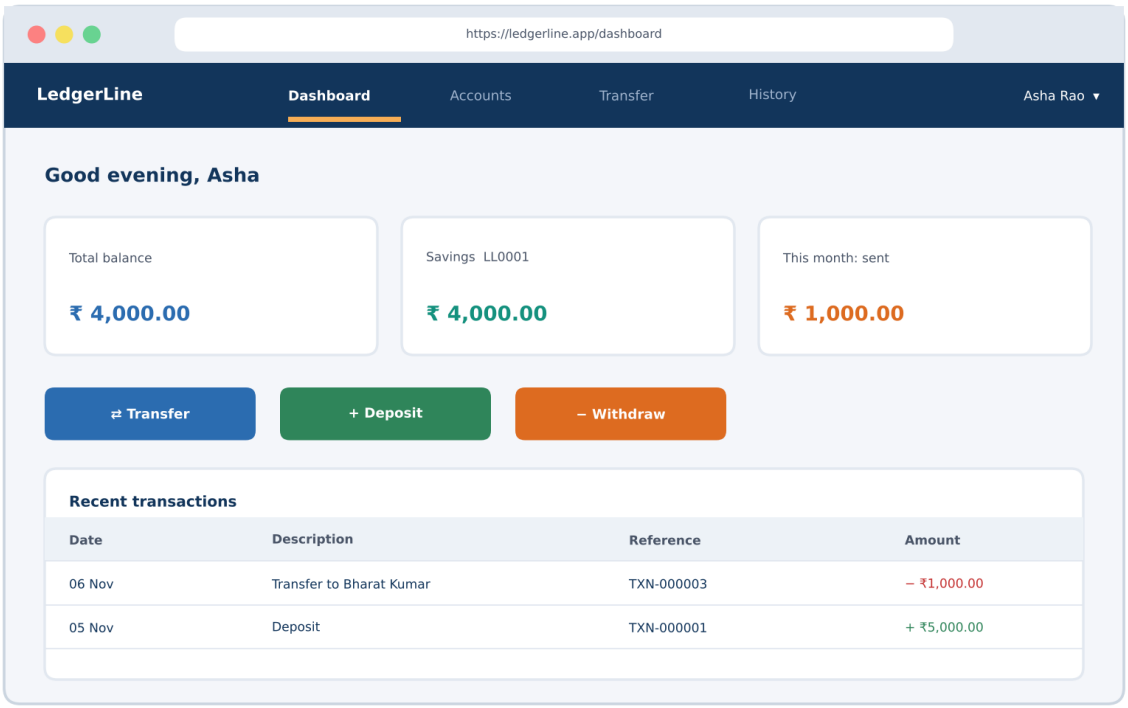


Figure 12.2 *— Dashboard: balances, quick actions, recent transactions.*

12.3 Account management

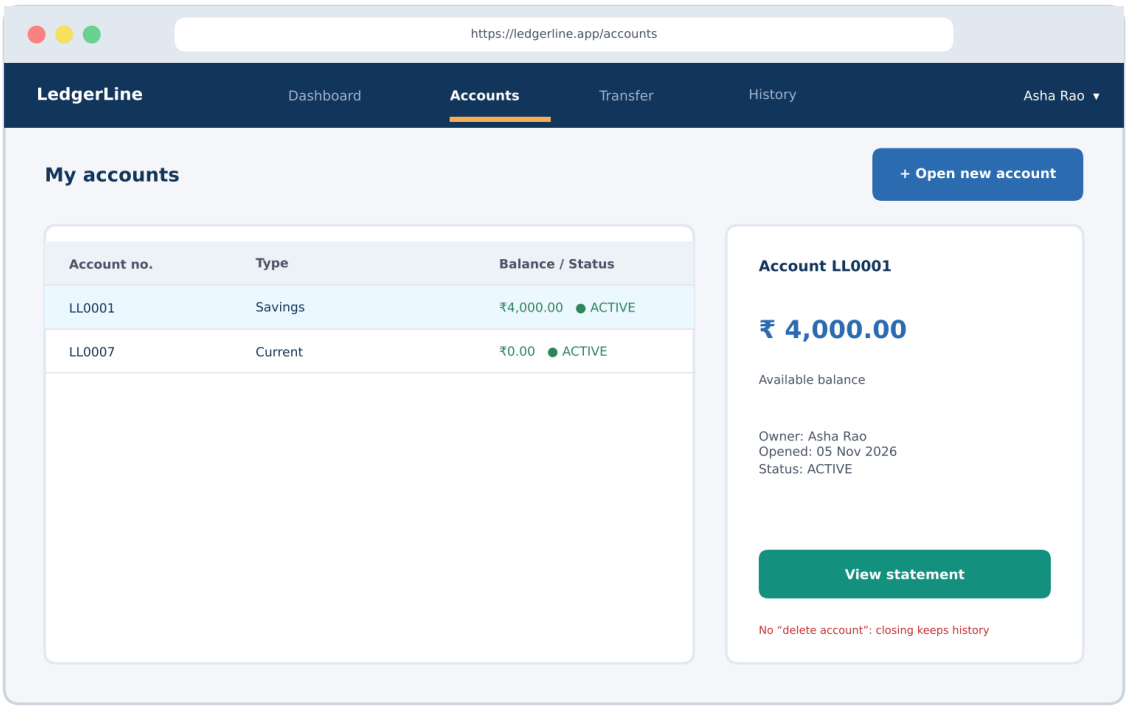


Figure 12.3 *— Accounts list and detail. There is no “delete account” button: records persist.*

12.4 Deposit and withdrawal screens

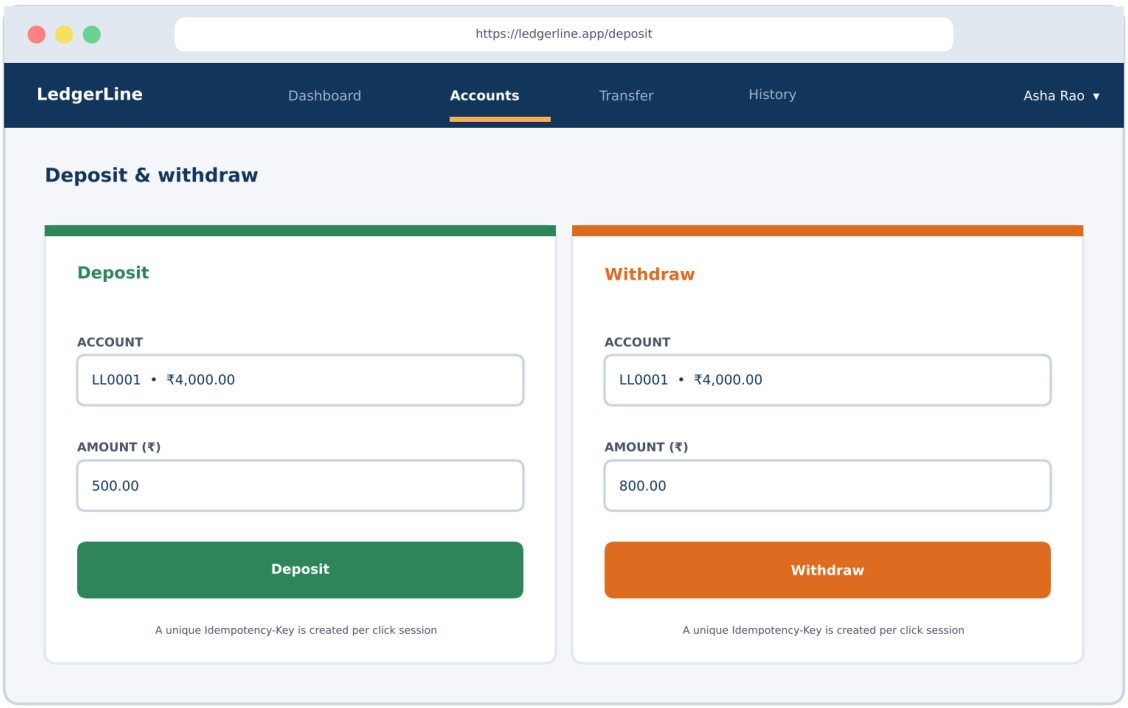


Figure 12.4 *— Deposit and withdraw cards. Each submission carries an Idempotency-Key.*

12.5 Transfer screen

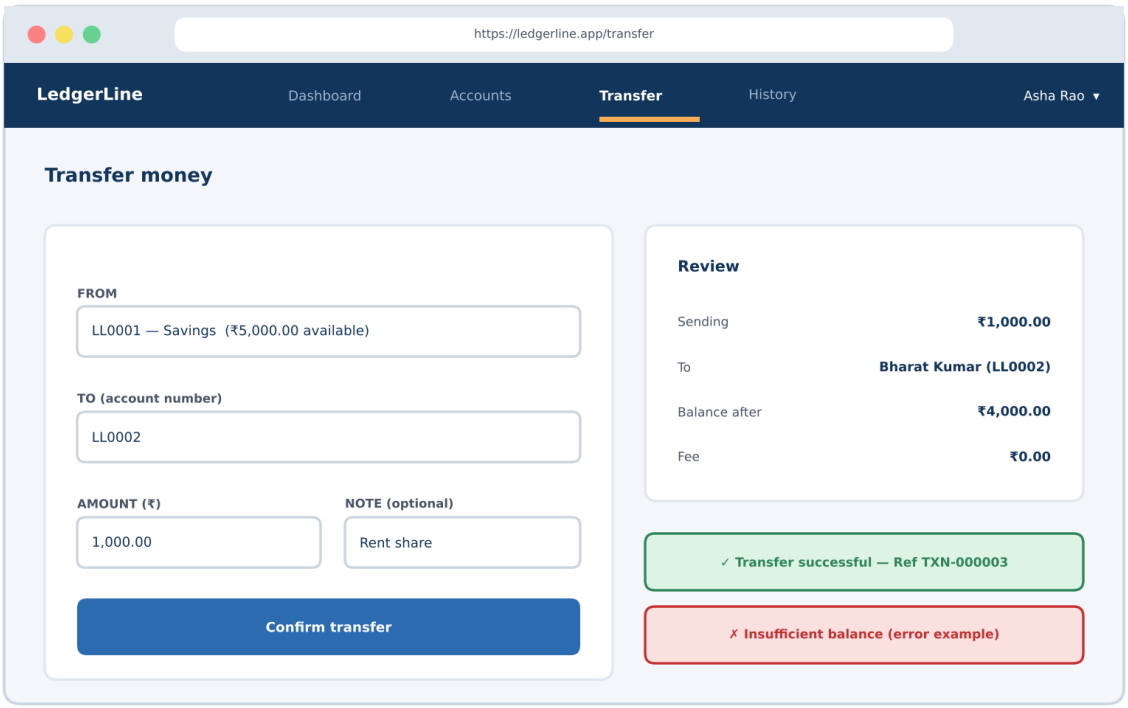


Figure 12.5 *— Transfer form with a review panel and examples of success and error messages.*

12.6 Transaction history

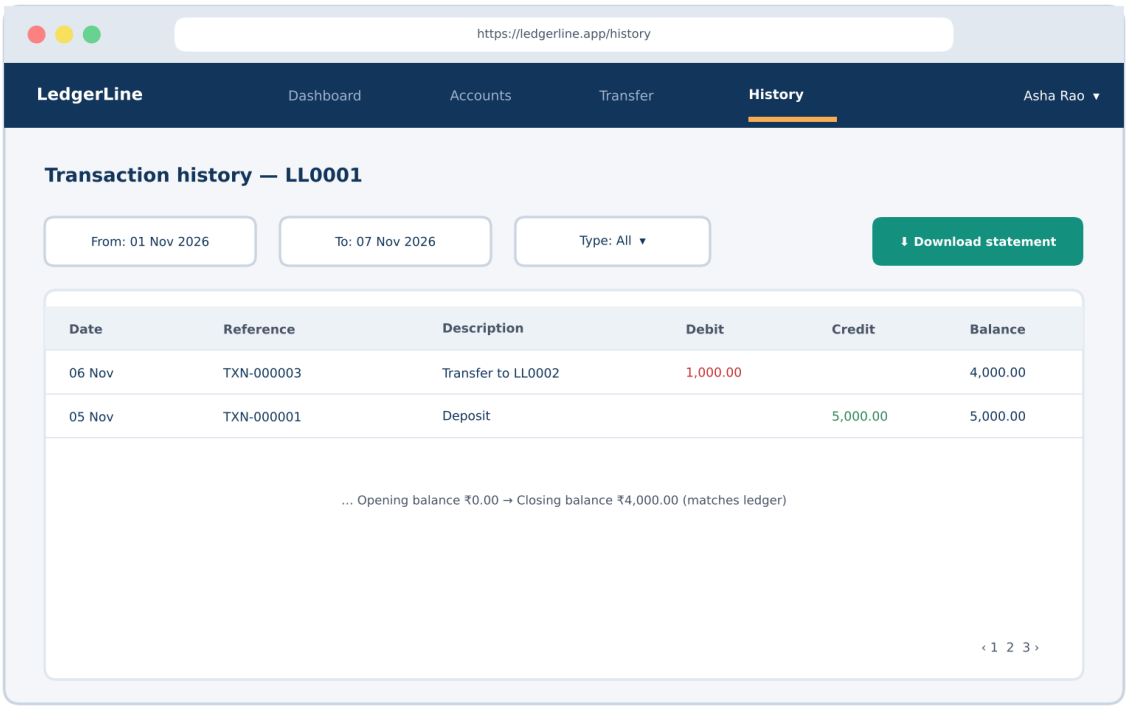


Figure 12.6 *— History with debit/credit columns and running balance; statements can be downloaded.*

12.7 Admin interface

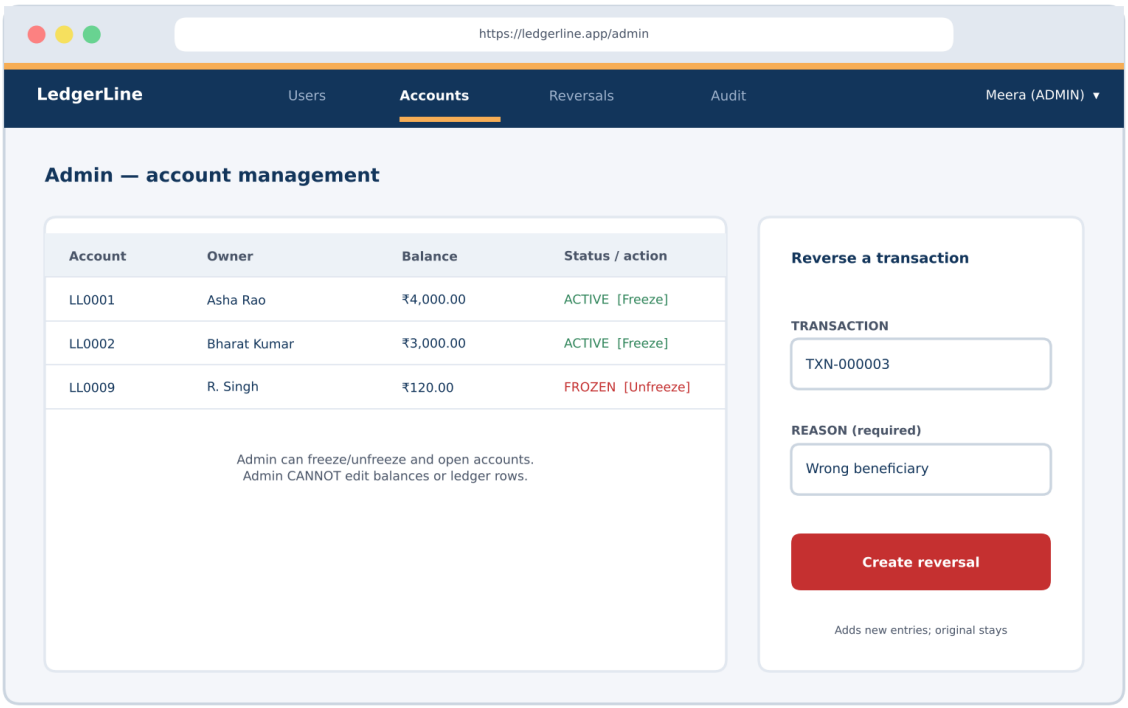


Figure 12.7 *— Admin: account status management and audited reversals. No way to edit balances.*

12.8 Auditor interface

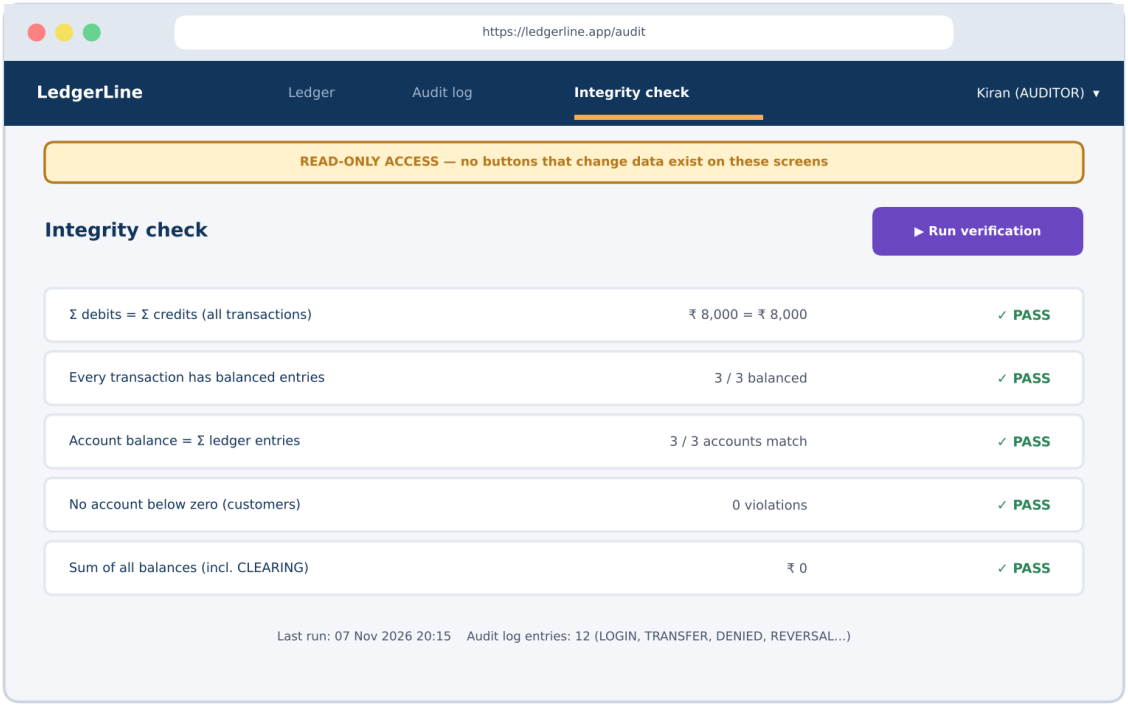


Figure 12.8 *— Auditor: read-only screens and an integrity verification report.*

13. Interview Preparation

For each question you will find (1) an answer in **simple words** to make sure you understand it, and (2) an **interview-ready** answer using correct technical terms. Practise saying them aloud, then add your own project examples.

13.1 Architecture

Q. Describe LedgerLine’s architecture.

In simple words: It has three layers: the web screens, the backend that applies all the rules, and the database that keeps the records. The backend is the strict guard; the database is the final safety net.

Interview-ready answer: LedgerLine is a three-tier application: a React SPA, a stateless REST API (FastAPI) and a PostgreSQL database. Requests pass input validation, JWT authentication, role and ownership authorization, then a transaction engine that, inside one ACID database transaction, performs idempotency handling, row locking, balance checks, balance updates, double-entry ledger inserts and audit logging. Rules are enforced in the API and again as database constraints (defence in depth).

Q. Why must business rules be enforced on the backend and not the frontend?

In simple words: Anyone can change what runs in their own browser or call the API directly. So the browser can only be helpful, never trusted.

Interview-ready answer: The client is outside our trust boundary; any validation or authorization performed there can be bypassed with curl or dev tools. The server must independently validate input, authenticate, authorize and enforce invariants; frontend checks only improve UX.

13.2 Transaction processing and double-entry

Q. Walk me through a ₹1,000 transfer.

In simple words: The user clicks; the server checks the request and who is asking; it locks both accounts so nobody else changes them; checks the money is there; moves it; writes two ledger lines; saves everything together; replies. If anything fails, nothing is saved.

Interview-ready answer: POST /transfers with a JWT and an Idempotency-Key. Validate payload; authenticate; authorize role and ownership; begin a DB transaction; claim the idempotency key; lock both account rows ordered by ID with SELECT … FOR UPDATE; verify status and balance; insert the transaction header; update both balances; insert a DEBIT and a CREDIT ledger entry with balance_after; write the audit row and the stored response; commit. Any exception triggers rollback, leaving state unchanged.

Q. What is double-entry bookkeeping and why use it in software?

In simple words: Every money movement is written down twice — where it left and where it arrived — and both amounts must match. If they ever don’t, you know there is a bug.

Interview-ready answer: Each financial event produces balanced debit and credit entries so Σ debits = Σ credits. It gives a self-checking, auditable trail: balances are derivable from entries, imbalances are detectable, and the whole ledger can be verified with simple queries. We enforce balance at the application level and with a deferred constraint trigger.

Q. What is the difference between an account balance, a transaction and a ledger entry?

In simple words: The balance is “how much now”. The transaction is “what happened”. Ledger entries are “the lines saying which account went up or down”.

Interview-ready answer: accounts.balance_paise is a mutable cached summary; transactions are immutable headers per financial event; ledger_entries are immutable debit/credit lines (≥2 per transaction). The ledger is the source of truth; the balance must equal the replay of entries, which we verify.

Q. Why store money as integers or decimals instead of floats?

In simple words: Computers can’t store numbers like 0.1 exactly in float form, so tiny errors appear and add up.

Interview-ready answer: IEEE-754 binary floating point cannot represent most decimal fractions exactly (0.1 + 0.2 ≠ 0.3), causing rounding drift. Use integer minor units (paise) or exact NUMERIC/Decimal types; convert only at the I/O boundary and reject more than two decimal places.

13.3 Atomicity and database transactions

Q. What is atomicity? How do you guarantee it?

In simple words: All steps happen or none do. We wrap them in a database transaction; if anything fails, the database undoes everything.

Interview-ready answer: Atomicity is the A in ACID: a transaction’s effects are all committed or none are. We perform the balance updates, ledger inserts, audit row and idempotency record inside one BEGIN…COMMIT block; on any exception the driver issues ROLLBACK and PostgreSQL discards the uncommitted row versions. I avoid autocommit and never swallow exceptions inside the block.

Q. Explain ACID.

In simple words: Four promises databases make: all-or-nothing (A), rules stay true (C), people do not disturb each other (I), saved data survives crashes (D).

Interview-ready answer: Atomicity: all-or-nothing. Consistency: transactions move the database between states that satisfy constraints. Isolation: concurrent transactions do not observe each other’s intermediate states, to the degree of the chosen isolation level. Durability: once committed, data survives crashes, via write-ahead logging.

Q. What happens if the server crashes right after COMMIT but before responding?

In simple words: The money did move, but the user never heard back. If they try again, the server must recognise the retry and just repeat the old answer.

Interview-ready answer: The transaction is durable, so balances are correct, but the client sees a timeout. Because the stored response is committed with the transaction, a retry with the same Idempotency-Key replays the original 201 rather than executing again.

13.4 Idempotency

Q. What is idempotency and how did you implement it?

In simple words: Doing the same request again doesn’t change the result more than the first time. We give each request a unique ticket number; if the same ticket shows up again, we return the old answer.

Interview-ready answer: The client sends a UUID Idempotency-Key per intent. A table with UNIQUE(user_id, key) stores a request hash and the final response. The insert is the first statement of the money transaction with ON CONFLICT DO NOTHING; if the key exists and the hash matches, we return the stored response; if the hash differs we reject. Concurrent duplicates serialise on the unique index.

Q. Why store a request hash with the key?

In simple words: To catch someone reusing an old ticket for a different request, which would otherwise get the wrong answer.

Interview-ready answer: The hash binds the key to its original payload. Without it, a client bug could reuse key K with a different amount and receive the cached response for the first request, silently dropping the second. We return 422 on mismatch.

13.5 Concurrency and race conditions

Q. Explain a race condition with two withdrawals.

In simple words: Two people check the same balance at once, both see enough money, both withdraw. Together they take more than exists.

Interview-ready answer: Check-then-act without mutual exclusion: both transactions read balance ₹1,000, both pass balance ≥ 800, both write ₹200 — a lost update. Fix with SELECT … FOR UPDATE (pessimistic row lock) so the second transaction waits and re-reads ₹200, or an atomic conditional UPDATE … WHERE balance ≥ amount, backed by CHECK (balance ≥ 0).

Q. How do you avoid deadlocks?

In simple words: A deadlock is two people each holding what the other needs. Always grab accounts in the same order so it can’t happen.

Interview-ready answer: Deadlocks occur when transactions acquire locks in opposite orders (A→B and B→A). I lock both rows in ascending ID order with a single SELECT … ORDER BY id FOR UPDATE, keep transactions short, and rely on PostgreSQL’s deadlock detector only as a backstop (retrying the transaction).

Q. Pessimistic vs optimistic locking — which and why?

In simple words: Pessimistic: “lock first, then work”. Optimistic: “work, then check nobody changed it, else retry”.

Interview-ready answer: Pessimistic locking suits high-contention, correctness-critical updates like balances: simple reasoning, no retries. Optimistic locking (version column) avoids waiting and suits low-contention data, but needs retry logic and wastes work under contention. I chose pessimistic row locks plus a CHECK constraint.

Q. What are isolation levels and which do you use?

In simple words: They decide how much one running operation can see of another’s unfinished work. Higher levels are safer but slower.

Interview-ready answer: READ COMMITTED (PostgreSQL default) prevents dirty reads; REPEATABLE READ gives a transaction-wide snapshot; SERIALIZABLE emulates serial execution and may abort with 40001 requiring retry. I use READ COMMITTED with explicit row locks; I would use REPEATABLE READ for consistent statements and could use SERIALIZABLE with a retry loop as an alternative.

13.6 Immutability, conservation and consistency

Q. Why can’t you delete or edit ledger entries? How are mistakes fixed?

In simple words: Because history must be trustworthy. A mistake is fixed by adding a new opposite entry, like a bank’s correction line.

Interview-ready answer: Append-only records preserve auditability and prevent silent tampering. Enforcement: no mutate endpoints, revoked UPDATE/DELETE privileges for the app role, and BEFORE UPDATE/DELETE triggers. Corrections are compensating REVERSAL transactions that reference the original, post opposite balanced entries, are restricted to one per original, require a reason and are audit-logged.

Q. How do you prove no money is created or lost?

In simple words: We add everything up and check the totals match.

Interview-ready answer: Invariants verified by queries and tests: each transaction's debits equal credits; each account's stored balance equals the signed sum of its entries; no customer balance is negative; and the sum of all account balances including the CLEARING account is zero. These run in the automated suite and as an Auditor verification endpoint.

Q. What does "consistency" mean?

In simple words: The database never stores something that breaks the rules.

Interview-ready answer: In ACID, consistency means each committed transaction preserves declared invariants — NOT NULL, CHECK, UNIQUE, FOREIGN KEY, triggers. It is not the same as "eventual consistency" in distributed systems.

13.7 Authorization and security

Q. Authentication vs authorization?

In simple words: Authentication: who are you? Authorization: what are you allowed to do?

Interview-ready answer: Authentication verifies identity (password at login, JWT signature and expiry on each request → 401). Authorization decides permission using role and resource ownership (→ 403/404). Both are enforced server-side on every protected route.

Q. What is IDOR and how do you prevent it?

In simple words: Changing an ID in a request to peek at someone else's data. We always check that the thing belongs to the logged-in user.

Interview-ready answer: Insecure Direct Object Reference / broken object-level authorization: the API trusts a client-supplied identifier without checking the caller's right to it. I derive identity only from the verified token, load the resource, compare owner_id, return 404 on mismatch to avoid account enumeration, and write an audit entry.

Q. How do you store passwords?

In simple words: Never as text. We store a slow, salted scrambled version that can't be reversed.

Interview-ready answer: Argon2id (or bcrypt) with a per-password random salt and tuned cost; never fast hashes like plain SHA-256. Login compares via the library's verify, uses constant generic error messages, and is rate limited.

Q. JWT vs sessions?

In simple words: Both remember who you are. JWT is a signed pass the client carries; sessions are a record kept by the server.

Interview-ready answer: JWTs are stateless and scale easily but are hard to revoke before expiry, so use short lifetimes (+ refresh tokens), explicit algorithm lists and secure storage. Sessions allow instant revocation but need shared storage and CSRF protection when using cookies.

Q. How do you keep secrets safe?

In simple words: Passwords and keys live outside the code, in settings on the server, never on GitHub.

Interview-ready answer: Configuration via environment variables / secret manager; .env in .gitignore with a committed .env.example; rotate any leaked secret since Git history retains it; least-privilege database user; secret scanning in CI.

Q. How do you prevent SQL injection?

In simple words: Never glue user text into database commands; let the database treat user input as plain data.

Interview-ready answer: Use parameterised queries or an ORM so input is bound as data, validate and constrain inputs, and give the DB account minimal privileges.

13.8 Testing and engineering

Q. How did you test concurrency?

In simple words: I sent two withdrawals at exactly the same time and checked only one worked. I also removed the lock to see the test fail.

Interview-ready answer: Parallel HTTP calls (thread pool with a barrier) against a real PostgreSQL: two ₹800 withdrawals on ₹1,000 must yield one 201 and one 422 and final balance ₹200; repeated many times; plus opposite-direction transfers to check for deadlocks. I verified the test fails when FOR UPDATE is removed.

Q. How did you use AI tools responsibly?

In simple words: I let AI draft, but I checked everything and wrote down where it was wrong.

Interview-ready answer: AI output was treated as untrusted code: reviewed, tested and corrected. PROMPTS.md records prompts, flaws found (e.g., unlocked read-modify-write), rejected approaches and verification. Critical code (locking, auth, ledger) was understood line by line.

Q. What would you improve with more time?

In simple words: Make it handle more users and more features safely.

Interview-ready answer: Refresh tokens and MFA; per-account daily limits and fraud rules; outbox/event stream for notifications; read replicas and partitioned ledger tables for scale; scheduled reconciliation jobs; formal chart of accounts; observability (metrics, tracing, alerts); property-based and chaos tests.

14. Final Learning Roadmap

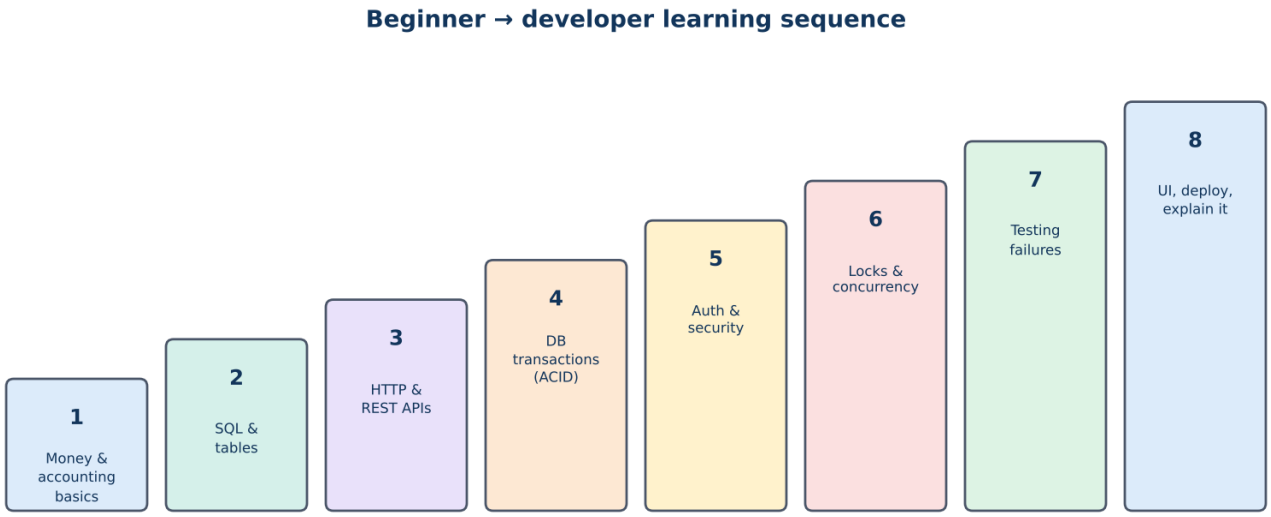


Figure 14.1 *— A beginner-to-developer learning sequence.*

14.1 What you must understand BEFORE coding

- What an account, balance, transaction, debit, credit and ledger are (Section 3).
- Why LedgerLine differs from CRUD, and the five failure types (Section 1).
- The full life of a ₹1,000 transfer including every failure branch (Section 4).
- The meaning of atomicity, idempotency, race conditions and immutability in your own words (Section 5).
- The difference between authentication and authorization; roles vs ownership (Section 6).
- The six tables and how they relate (Section 7).
- Which parts of the design are confirmed and which are your decisions (Sections 0 and 8).

14.2 What you will learn DURING implementation

Phase	You will learn by doing
2	SQL DDL, constraints, indexes, migrations; connecting an app to PostgreSQL; configuration via environment variables
3	Password hashing, JWT, dependency-based authorization, writing audit logs
4	Database transactions in application code; building ledger entries; designing statements; reversal logic
5	Row locks, lock ordering, unique-index tricks, writing parallel tests; reading PostgreSQL docs on isolation
6	Threat modelling; property-style invariants; fault injection; test design
7	API consumption from React; state and token handling; error UX
8	CI pipelines, deployment, production configuration, documentation

14.3 What you must be able to explain AFTER completion

- Draw the architecture and the transfer sequence diagram from memory.
- Explain each of the eight financial rules with a numerical ₹ example and show **where in your code** it is implemented.
- Demonstrate a test that fails when you remove the lock, and one that fails when you remove the transaction.
- Explain why your ledger can prove balances and how to run the integrity checks.
- Justify every proposed design decision (stack, schema, CLEARING account, token choice) and state what you would change.
- Describe three AI mistakes you caught and how you verified fixes.

14.4 A beginner-to-developer learning sequence

Step	Topic	Practice task	Done when you can…
1	Accounting basics	Write 5 transactions on paper in debit/credit form	Show Σ debits = Σ credits for each
2	SQL fundamentals	Create users/accounts tables; INSERT, SELECT, JOIN, GROUP BY	Compute a balance from ledger rows with SQL
3	HTTP & REST	Build a tiny API with two endpoints; call with curl/httpx	Explain methods, status codes, headers, JSON
4	Database transactions	In psql: BEGIN; update; ROLLBACK; open two sessions and watch locks	Demonstrate atomicity and a blocking lock by hand
5	Auth & security	Add login with hashing and JWT; build a role guard	Show a 401, a 403 and an ownership denial
6	Concurrency	Run two psql sessions with SELECT … FOR UPDATE; then script parallel requests	Reproduce the race, then fix it
7	Testing	pytest basics, fixtures, parametrisation, fault injection	Write a failing test first and make it pass
8	Frontend & deploy	React forms calling your API; deploy a hello-world	Ship the full system and explain it end to end

> **NOTE**
> A great hands-on exercise before writing any app code: open **two terminal windows with psql**, start a transaction in each on the same account row, and watch the second one wait. Ten minutes of this teaches more about concurrency than hours of reading.

Appendix A — Glossary

Term	Plain meaning
ACID	Four database transaction guarantees: Atomicity, Consistency, Isolation, Durability
Account	Container tracking one holder’s money
API	Defined set of requests a program accepts from another program
Append-only	Data may be added but never changed or removed
Atomicity	All steps happen or none do
Audit log	Append-only record of who did what and with what result
Authentication / Authorization	Who you are / what you may do
Balance	Current money in an account; equals credits minus debits
CHECK constraint	Database rule that every row must satisfy a condition
Clearing account	(Proposed) system account representing money entering or leaving the system
Commit / Rollback	Make a database transaction permanent / undo all its changes
Compensating transaction	New transaction that cancels the effect of an earlier one without editing it
Concurrency	Several operations running at overlapping times
CRUD	Create, Read, Update, Delete
Credit / Debit	In LedgerLine: credit raises a customer balance; debit lowers it
Deadlock	Two transactions each waiting for a lock the other holds
Double-entry	Each event recorded as equal debits and credits
Foreign key	Column that must match a row in another table
Hash (password)	One-way scrambled form of a password
Idempotency key	Unique token that lets a server recognise a retry of the same request
IDOR	Accessing others’ data by changing an identifier the server fails to authorize
Index	Lookup structure that speeds up queries
Isolation level	Setting controlling how much concurrent transactions see of each other
JWT	Signed token carrying identity claims
Ledger	Permanent ordered record of money movements
Lock (row lock)	Mechanism making others wait while you modify a row
Migration	Versioned script that changes the database schema
Paise	1/100 of a rupee; money is stored as integer paise
Primary key	Unique identifier of a row
Race condition	Bug where the result depends on unlucky timing
RBAC	Role-Based Access Control
Reversal	Compensating transaction that cancels a previous transaction
Salt	Random data added before hashing so identical passwords hash differently
UUID	Long random unique identifier

Appendix B — Quick Reference Cheat Sheet

Rule	Remember	Mechanism
Atomicity	All or nothing	BEGIN … COMMIT / ROLLBACK
Double-entry	Debits = credits	Balanced entries + deferred trigger
Idempotency	Same key, same result	UNIQUE(user_id,key) + stored response + request hash
Concurrency	Take turns on shared rows	SELECT … FOR UPDATE in sorted ID order
Immutability	Never edit history	Triggers, revoked privileges, reversal transactions
Conservation	Total is constant	Integer paise + invariants + integrity queries
Isolation	Transactions don’t see each other’s half-work	READ COMMITTED + locks (or SERIALIZABLE + retry)
Consistency	Database refuses invalid states	CHECK / FK / UNIQUE / NOT NULL
Authentication	Prove identity	Argon2/bcrypt + JWT
Authorization	Check permission on the server	RBAC + ownership check
Audit	Record everything important	Append-only audit_logs, including denials

> **NOTE**
> Final reminder: this guide separates **what the project requires** from **what I propose**. When you present LedgerLine, make that distinction yourself — it shows engineering maturity.